# Rubik’s Cube Solver

[redacted]

# Contents

Contents ..... 2
Description ..... 6
Analysis – TO BE FINISHED BY 15/9/23 ..... 9
Analysis/Research Plan ..... 9
Existing Systems ..... 9
ruwix.com ..... 9
rubiks-cube-solver.com ..... 10
Type of Application ..... 10
How to represent the cube in code ..... 11
Class diagram ..... 11
Which Algorithm to Solve? ..... 12
Kociemba's Algorithm ..... 12
How does the algorithm work? ..... 12
Scrambling ..... 12
Graphics ..... 13
UI ..... 13
Moves ..... 14
Dialogue with the end-user ..... 14
Interview ..... 14
Analysis of user ideas ..... 15
User requirements ..... 16
Objectives ..... 17
Design – TO BE FINISHED BY 13/10/23 ..... 18
Overall System Design ..... 18
Underlying structure ..... 18
Moves and Rotations ..... 20
UI ..... 20
Designing the UI ..... 21
Explanation of the UI design ..... 23
Palette ..... 23
Algorithms ..... 23
Resetting the cube (meets objective 7) ..... 23
CheckSolvable ..... 24
CheckSolved ..... 26
Scrambling Algorithm ..... 26
Solving Algorithm (Kociemba's Algorithm) ..... 27

Implementation – TO BE FINISHED BY 17/11/23 ..... 28
Face ..... 28
Facelet ..... 28
Cubie ..... 28
Property CubieType ..... 29
Property Colours ..... 29
Property Permutation ..... 29
Moves ..... 30
reset ..... 30
AssignCubiesArray ..... 30
L ..... 31
R ..... 31
U ..... 32
D ..... 32
F ..... 33
B ..... 33
E ..... 34
S ..... 34
M ..... 35
La ..... 35
Ra ..... 36
Ua ..... 36
Da ..... 37
Fa ..... 37
Ba ..... 38
Ma ..... 38
Ea ..... 39
Sa ..... 39
x ..... 40
y ..... 41
z ..... 42
xa ..... 43
ya ..... 44
za ..... 45
CheckSolvable ..... 46
checkface ..... 47
SOLVE ..... 47
PaletteChanges ..... 48

PaletteY_click ..... 48
PaletteW_click ..... 48
PaletteB_click ..... 49
PaletteG_click ..... 49
PaletteR_click ..... 50
PaletteO_click ..... 50
faceletclicked ..... 51
Form1 ..... 51
Scramble_Click ..... 52
Form1_Load ..... 54
btnReset_Click ..... 55
L_Click ..... 55
R_Click ..... 55
U_Click ..... 56
D_Click ..... 56
F_Click ..... 56
B_Click ..... 56
M_Click ..... 56
E_Click ..... 56
S_Click ..... 56
btnLa_Click ..... 56
btnRa_Click ..... 56
btnUa_Click ..... 57
btnDa_Click ..... 57
btnFa_Click ..... 57
btnBa_Click ..... 57
btnMa_Click ..... 57
btnEa_Click ..... 57
btnSa_Click ..... 57
btnx_Click ..... 57
btny_Click ..... 57
btnz_Click ..... 57
btnxa_Click ..... 58
btnya_Click ..... 58
btnza_Click ..... 58
btnHelp_Click ..... 58
ArrangeButtons ..... 59
CheckSolved ..... 61

ShowLog ..... 61
PaletteY_Click ..... 61
PaletteW_Click ..... 62
PaletteB_Click ..... 62
PaletteG_Click ..... 62
PaletteR_Click ..... 62
PaletteO_Click ..... 62
btnSolve_Click ..... 63
checkEditingMode ..... 63
Testing – TO BE FINISHED BY 8/12/23 ..... 64
Test Log ..... 64
Objectives Testing ..... 78
Evaluation – TO BE FINISHED BY 19/12/23 ..... 84
Evaluating objectives ..... 84
Feedback from user ..... 86
Improvements ..... 87
Actually use the underlying cube (objectives 1 and 4.1) ..... 87
Arrangement of buttons (objectives 3 and 5.7) ..... 87
Box around palette reflects the selected colour ..... 87
Animation (objectives 5.4 and 5.5) ..... 88
The algorithm to solve the cube (Kociemba's algorithm) (objective 10) ..... 88
Checking if the cube is solved ..... 88
Links ..... 89

# Description

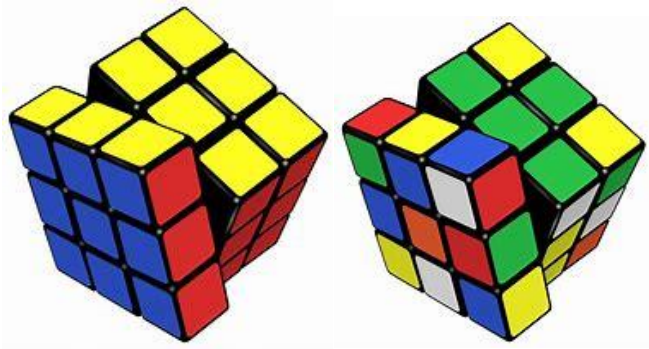
## Description of the problem to be investigated

I am going to be making my own Rubik's Cube program, in order to investigate how a Rubik's Cube can be stored on a computer and to experiment with the algorithms used.

## The Rubik's Cube

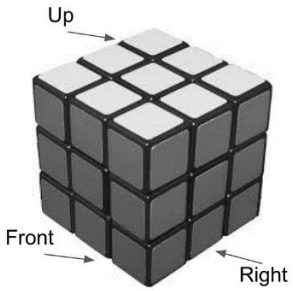
The classic Rubik's Cube was developed by Erno Rubik when he was looking for a solution to get his students to understand 3D architecture. It took him a month to solve it when he scrambled it for the first time. He originally called it the "Magic Cube".

Original state: Scrambled state:



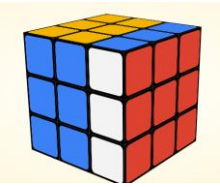
The classical Rubik's Cube consists of a 3x3x3 physical grid of 27 'cubies' which can move around. There are 6 'faces' which consist of 9 'facelets' each, meaning there are 6x9=54 facelets in total. Each face is one of 6 colours from white, yellow, blue, green, red and orange. The yellow and white centres are opposite each other, the blue and green centres are opposite each other, and the red and orange centres are opposite each other. By default, in the Rubiks community, the yellow centre is on top and the white centre is on the bottom, although people could use green and blue or red and orange for fun.

There are notations used by the Rubiks community to name the possible moves that can be done on a Rubiks cube. Each face is moved in relation to how it looks as if it was being looked at. Each face is denoted by its initial, that is (R)ight, (L)eft, (F)ront, (B)ack, (U)p and (D)own.




If we want to move the front face clockwise, we will notate that move with the front face's initial (F). If we want to turn the face anticlockwise, we add an apostrophe after the face's initial (F'). If we want to turn the face 180°, we put the face's initial and a 2 to denote that we turn it 90° twice (F2). There are also lowercase moves (f, r, f2, r2, f', r', etc.) which denote turning the two columns on that side. This also means turning the cube so that for example, with the move r, the front face becomes the up face and so on.

R performed on the cube:



There are 43 quintillion possible cube states. This is because of all the possible positions of the corner and edge cubies, although we do have to filter out impossible orientations and permutations. This is because there are cube states which cannot be solved, where the edge and corner orientations and permutations do not add up correctly, and it would be as if someone twisted either a corner or an edge piece to make a prank. However, if a correct number of cubies were twisted, then the cube would be solvable again as the permutations would cancel each other out.

$$\frac{(3^8 * 8!)(2^{12} * 12!)}{3 * 2 * 2} \approx 4.3 * 10^{19}$$

In 2010, Tomas Rokicki, Herbert Kociemba, Morley Davidson, and John Dethridge reduced the 43 quintillion possible combinations by a factor of 48 using mirrored states and symmetry. There are 24 ways a Rubik's cube can be oriented in space and this can be multiplied by two to deal with mirrored states. After this reduction, 55,882,296 cube states remain. Using Google's computers, they were able to generate the numbers shown in Table 2. Calculating these numbers would take a good desktop computer 35 CPU-years.

Table 2: Scramble Distance Possible Positions

| Distance | Total Possible Positions |
|---|---|
| 0 | 1 |
| 1 | 18 |
| 2 | 243 |
| 3 | 3,240 |
| 4 | 43,239 |
| 5 | 574,908 |
| 6 | 7,618,438 |
| 7 | 100,803,036 |
| 8 | 1,332,343,288 |
| 9 | 17,596,479,795 |
| 10 | 232,248,063,316 |
| 11 | 3,063,288,809,012 |
| 12 | 40,374,425,656,248 |
| 13 | 531,653,418,284,628 |
| 14 | 6,989,320,578,825,350 |
| 15 | 91,365,146,187,124,300 |
| 16 | ~ 1,100,000,000,000,000,000 |
| 17 | ~ 12,000,000,000,000,000,000 |
| 18 | ~ 29,000,000,000,000,000,000 |
| 19 | ~ 1,500,000,000,000,000,000 |
| 20 | ~ 490,000,000 |

From this, we can conclude that it is possible to solve any cube state in a maximum of 20 moves (Rubiks Cube's God's number)! A puzzle's "God's number" can be calculated by working out the least possible number of moves to solve the puzzle in any randomly generated state. Any algorithm which works out how to find these optimal moves is therefore called a "God's algorithm".

There are many different algorithms that can be used. There are "human algorithms", which can be solved by a human, usually intuitively. Some are more intuitive than others, which follow more of a flowchart, such as the beginner's method. Then there are also "computer algorithms", which can be

used by computers. Some are quicker than others based on the programming language and the data structure used and how complicated the skills to program them are.

# Analysis – TO BE FINISHED BY 15/9/23

## Analysis/Research Plan

First of all, I need to interview a user who will use my Rubik's cube solver as this will define any requirements that I will have.

I have made a list of things that I will research:

- How to display the cube on screen to the user.
- Which algorithm is the best to use in relation to which data structure I am going to use.
- How to represent the cube in code in conjunction with and considering which algorithm I am going to use.

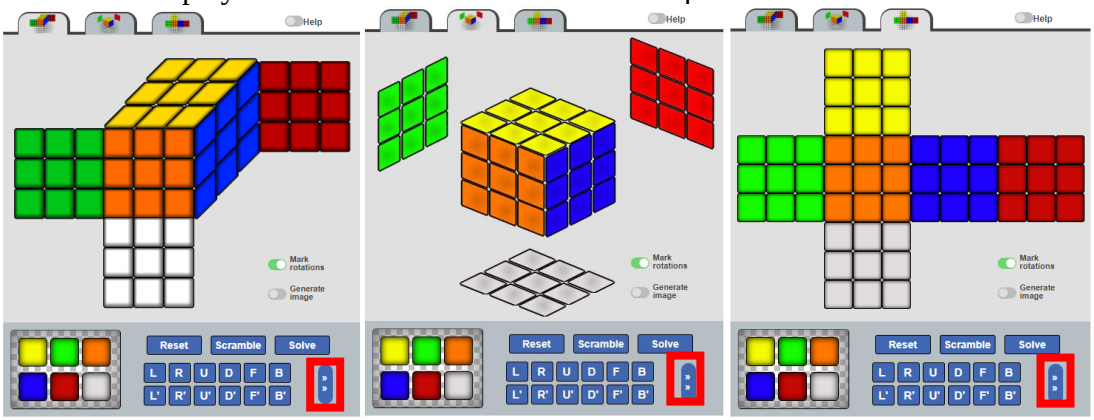
There are some things that I will need to consider before others. I will look at different data structures that others have used first so that I can get a strong foundation when I look at different algorithms. I learnt that the algorithm used will greatly define which internal data structure to use. Finally, I will look at how to represent the cube onscreen for users and for myself to test if the cube is working.

## Existing Systems

I searched for existing Rubiks cube solvers to get some inspiration. Therefore, they will all be web applications but they are similar in user-interface to a form application.

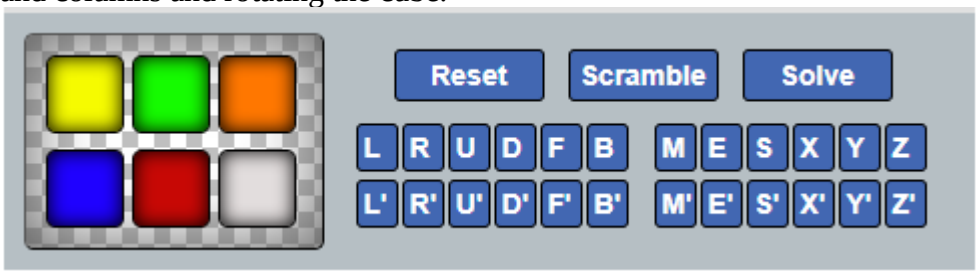
### ruwix.com

Online Rubik's Cube Solver App (ruwix.com) had an option to choose the layout of the cube and how it was displayed on screen. This included three options as shown below.




The third looks easier to program to display on screen, as all of the facelets are of the same type, whereas for the other two there would have to be three different types of facelet class as the three types look different on screen. This will require object-oriented programming and composition, as the faces will be composed of a 2D 3x3 array of facelets. The issue with this approach is that it will be significantly harder to visualize the cube.

The buttons are ordered so that the anticlockwise moves are underneath the clockwise moves. When clicking on the arrows to the right (in the red boxes), more moves appear, including the middle rows and columns and rotating the cube.

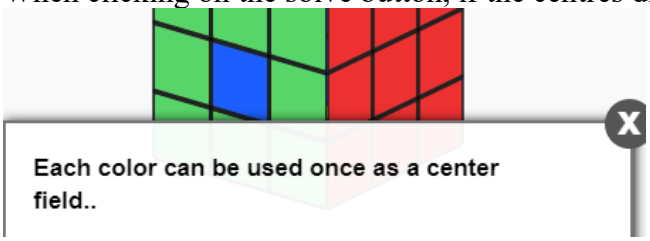


I am also going to have a palette of colours. However, I think I will put the extra moves under the basic moves.

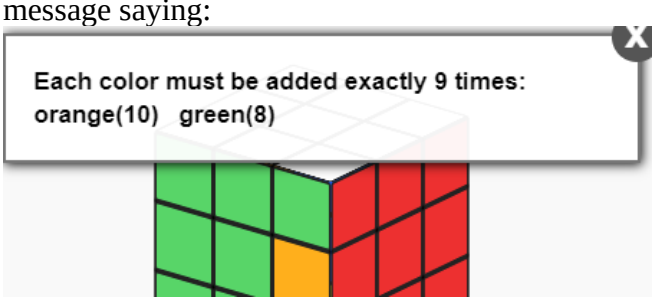
### rubiks-cube-solver.com

This one also had the three layouts and also had animation, but I will not use 3D animation. It had arrows to show the rotations, but I don't know how to do that and I will stick to the letter annotations.

When clicking on the solve button, if the centres didn't match up, it showed a message saying:

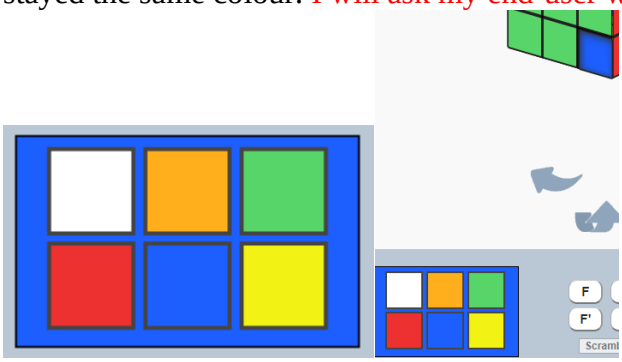



If the centres were alright but the total number of facelets of each colour didn't match up, it showed a message saying:




Whilst also specifying how many facelets were of each colour that didn't have 9 total facelets of that colour.

It had a box around the palette which changed its colour to that of the selected palette colour. It sometimes changed its colour to nothing when a facelet on the cube was clicked, and sometimes stayed the same colour. I will ask my end-user what he thinks about this.




## Type of Application

There are a few options, of which a console application, a web application, and a form application. I have never made a web application, as I would need a domain name to show it to other people, which costs money. I would also have to learn HTML which I do not have the time to do. A form application uses OOP in the interface, and it will be easier to display buttons. A console application would require a command line interface where the user types in commands (which will have to be remembered), however I think that the experience would be more enjoyable if buttons were involved. A form application also looks better. The examples online all use a web application, which looks like a form application, so mine will look better if it looks like the others available online. However, I

have never seen a Rubik's cube solver online that uses a command line, so if I use a console application, it will stand out, and give some variety to users who are looking for a Rubik's cube solver.

I will use a Windows Form rather than a Console Application as it uses OOP more, I can output the faces more easily and it will be easier to animate.

## How to represent the cube in code

This StackOverflow discussion ([data structures - How would you represent a Rubik's Cube in code? - Stack Overflow]) suggests a lot of good ways to represent the cube, such as binary, a 6x3x3 2D array and a 3x3x3 3D array. The 3D array seems more intuitive, as that is what a Rubiks Cube is. The cube will be a 3x3x3 array of cubies. A cubie will have a CubieType, which will dictate how many colours it is allowed to have. I will also need to store their permutations for each cubie. For simplicity, I will denote each cubie's position as (U|D)(F|B)(L|R)

There was also a solution using binary, which stores the six faces in six strings. It was deemed to be much faster, as moves and rotations would be represented by bitwise operations. To store a face, only the outer 8 facelets would be stored as the middle facelet wouldn't move. This means that a face would be represented by 8 3-bit patterns, as there are only 6 colours. For example, the face "

```
WGR
G B
WYO
```

" will be represented in code for example as "00000000 00000001 00000010 00000011 00000100 00000101 00000000 00000001". It starts at the top left facelet and goes clockwise around the face. To turn the face clockwise, a bit rolling operation turns it into "00000000 00000001 00000000 00000001 00000010 00000011 00000100 00000101" which is equivalent to "

```
WGW
Y G
OBR
```

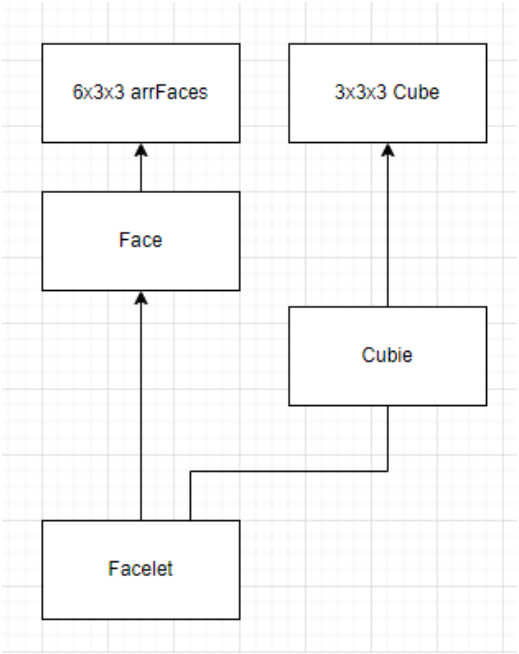
". Any adjacent facelets affected could be manipulated manually using trivial manipulation. I thought of reducing it to 3 bits per facelet.
I had the idea to not store it in binary, but instead as a string of colour initials (still going clockwise around the face), as it would be easier to read it when looking at the code.

The 3x3x3 array will make rotation easier to visualize, despite apparently being slow to move around. I could try the 6x3x3 array of faces, although this will mean that the faces' position in the array reflects their positions as seen from the user. This means that every time the cube is rotated, the faces will all have to switch positions and some will be reordered, which is very confusing, however I think it is worth a try.

After some reflection, I have decided that I will use both a 3x3x3 array of cubies and a 6x3x3 list of faces, so that I can experiment and see which one is easier to move its elements around.

## Class diagram

The 3x3x3 cube will be composed of cubies, and a face is composed of nine facelets. This is worthy of a simple class diagram. When making it, I realised I could allocate facelets to a cubie, which would make it easier when updating the onscreen cube net.




## Which Algorithm to Solve?

I will obviously need to have a solve button which will call a subroutine which will work out how to solve the cube from the current state. Therefore, I will need a solving algorithm.

I had to find out which algorithm was going to be the best one to use in my coursework. I was well aware that there were "human algorithms" and "computer algorithms", but the problem was going to be to identify which one I would use and explain why.

I quickly learnt that the underlying data structure and programming language used aren't that important for human algorithms, however they have to be carefully chosen for computer algorithms.

I chatted with my end-user and he said that he would like a computer algorithm as it is more impressive and also what's the point of implementing a human algorithm when the user can just do it themselves. Therefore, I will use a computer algorithm.

## Kociemba's Algorithm

After some research, I read about Kociemba's algorithm, an algorithm developed by Herbert Kociemba and which was later developed upon by Koch, who made the 20-move God's algorithm for the Rubik's cube. It follows the Two-Step Approach. I got all of my inspiration from Kociemba's own website: Solve Rubik's Cube with Cube Explorer (kociemba.org).

### How does the algorithm work?

The algorithm follows a two-step approach. The first step tries to get the fully scrambled cube to a subset of cubes (called G1). Step 2 uses just the moves U, D,

## Scrambling

Obviously, there will need to be a scrambling function so that the user has the challenge to solve it. I wondered if the scrambling function would continue scrambling from the current state of the cube, or from the default position. It would be more sensible to continue from the current state, because if the

user wants the cube to be scrambled from the default position, they can just press the Reset button and then scramble.

The program will generate a random series of moves and execute them.

It would be nice to have a textbox in which the user can input how many moves they would like the program to execute to scramble the cube. To prevent errors, the textbox will be an up-down control with up and down arrows, which the user can press if they want more or less moves in the scramble. There could also be a tick box if the user doesn't care about how many moves there will be in the scrambling process. I asked my brother what he thought about that and what range he would like, and he said _ to __. The program will choose a random number between 5 and 20.

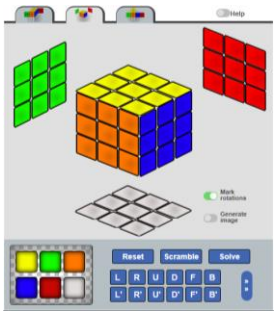
I will keep a score of the moves needed to solve the cube. Each time the user makes a move, I will add it on to the beginning of the list of moves needed to solve the cube. When hitting 'solve', I will count the number of moves required to solve the cube and compare it with __, which is the maximum number of moves which Kociemba's algorithm solves any cube in. If the number of moves in the score exceeds __, a new solving process will begin.

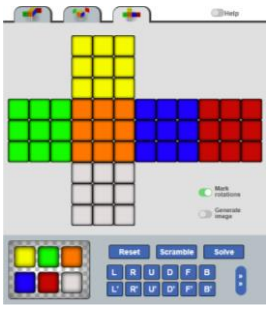
I don't know yet if the scrambling process will be animated or if the facelets will be updated all in one go. I will ask my end-user.

## Graphics

There are a few options, of which one I really liked from ruwix.com (see below) as it shows the non-visible faces like you are looking at them like an X-ray and it is easier to make the links between which facelet is on the same cubie as another facelet. However, a classic net would be easier to represent on a Windows Form. If I have time after making the rest of the fundamentals of the program, such as the moves, rotations and the solving algorithm, I could experiment with the graphics. I will put the yellow face at the top and the white face at the bottom, as that is what is considered the 'default' position in the Rubiks community.

From ruwix.com: Classic net:




Clearly, each face will need to be comprised of nine facelets in a 3x3 grid, no matter which design I choose. Therefore, I will use object-oriented programming in order to do this and create a class called Facelet.

## UI

I could choose between a 3D animation using Java, or I could use a cube net.

| **3D** | **Cube net** |
|---|---|
| • More advanced<br>• I will need to use another IDE than I am used to and learn a new language (Java). | • Simpler to implement.<br>• I can implement it using the skills I already have. |

Overall, I think I will use a simple cube net, as I feel like the 3D animation would be outside of the scope of what I am trying to achieve: I am focusing on the implementation of a Rubik's Cube and the algorithms involved.

I will have a palette of colours, and when a facelet is clicked, it will change colour. However, this must be done in an editing mode, therefore I will introduce an Editing Move tickbox.

The move buttons will need to be neatly arranged to avoid confusion and make it obvious which buttons are the opposites of which.

## Moves

For executing the moves, I will make six subroutines for R, L, F, B, U, and D. I would like to prevent the Cube from being public. Therefore, the moves will also be functions, returning the new state of the cube.
When a 180° turn is required, I will just call the relevant move two times. I will not bother putting buttons for 180° rotations, as online Rubiks websites do not have them, and the user can press either of the clockwise and anticlockwise buttons twice. For an anticlockwise move, I will create separate move subroutines as the animation flows more easily, rather than call the move 3 times. I decided to keep each move stationary, that is to say relative to the forward-facing face. For M(iddle) and E (turning the middle layer 90° to the left), I will implement the algorithm that I am in the process of such that it will compliment the new state of the cube. For example, when an M happens, the centre at the front will go to the top centre spot. Then I will switch each rotation such that for example a U (top layer 90° clockwise) turns into B (back layer 90° clockwise).

## Dialogue with the end-user

### Interview

I asked my brother if he would be willing to use my Rubik's Cube solver. Before I showed him anything, I asked him what he would like the program to do. He put the following down:

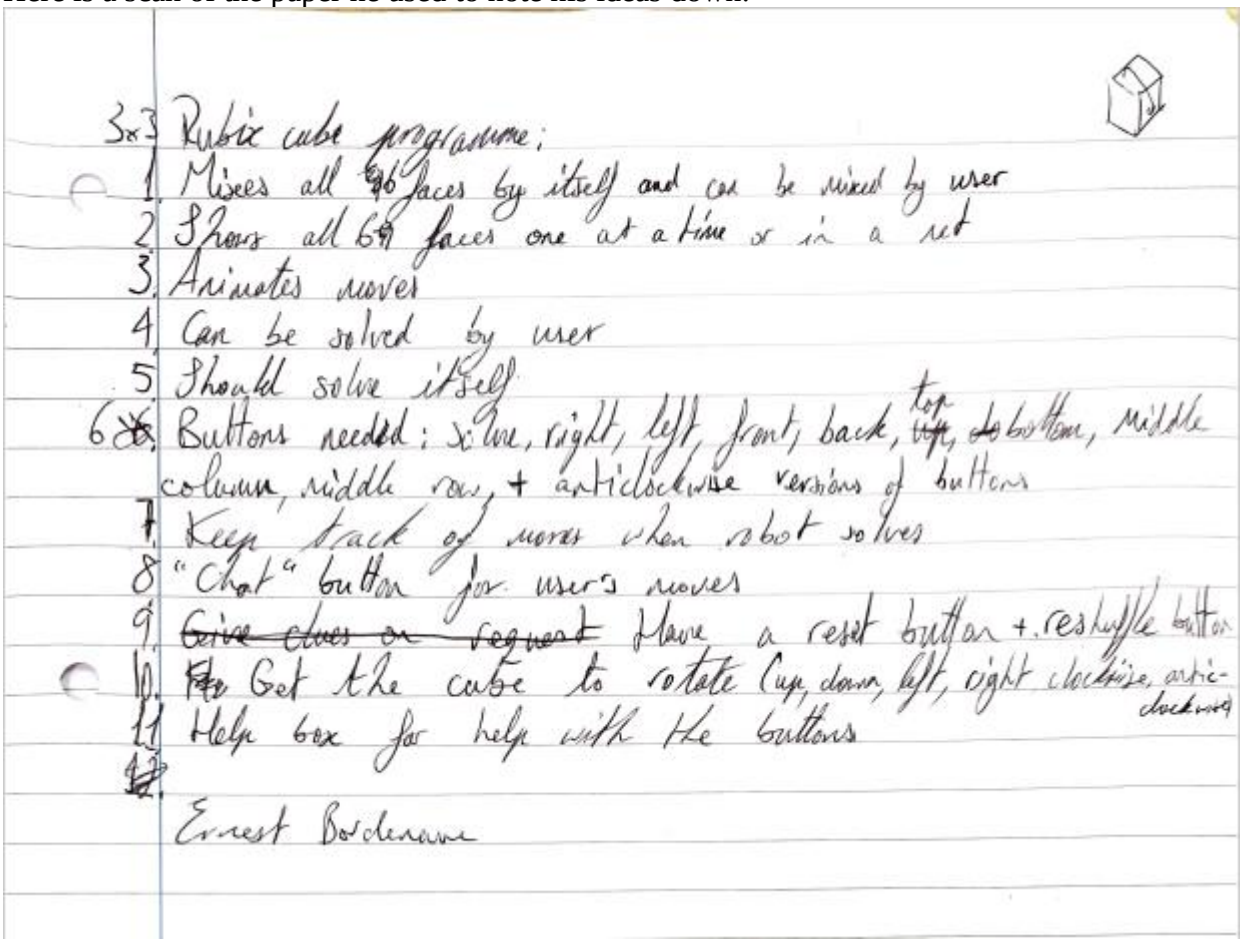
1. "Mixes all 6 faces by itself and can be mixed by user"
2. "Shows all 6 faces one at a time or in a net"
3. "Animates moves". He wanted it to show a 3D cube, but I said that I will not be doing that as it would be time wasted on animation. We decided on the end that the moves will be animated (in the direction that the cubies are moving in) on the net that I had already put on the Windows Form.
4. "Can be solved by user"
5. "Should solve itself". He elaborated on this afterwards that it should use an algorithm with as few moves as possible, so that it is more impressive and otherwise he could have done it himself.

Then I showed him how far I had come at that point, and I asked him if he had any further ideas from inspiration.

6. "Buttons needed: solve, right, left, front, back, top, bottom, middle column, middle row". I asked him if he wanted anticlockwise buttons for each of the moves and he said yes, so he added "+ anticlockwise versions of buttons". He also stressed on the fact that the buttons needed to be more explicit.
7. I thought of adding a textbox to the right under where the solve button was going to go, as there was space on the right of the Windows Form. He said yes and added "Keep track of moves when robot solves".
8. "'Chat' button for user's moves". I was confused, so he explained that it was going to keep track of what has happened, such as the user's moves, scrambles, and the moves to do when the solve button is pressed.

9. He put down "Clues on request", that is to say the program would output in the chat what the next move to solve the cube was. I said that that was a bit useless, because the user will probably press the clue button loads of times, and that would just be the same as clicking the solve button once.
10. I showed him the reset button that I put on, to put the cube back to its original state. He was a bit confused as to why, and I said it was to consider users that can't solve the cube, and the user could do their own moves on a solved cube straightaway if they wanted to. He put down "Have a reset button + reshuffle button". The reshuffle button is to re-scramble the cube.
11. Get the cube to rotate (up, down, left, right, clockwise, anticlockwise).
12. Help box for help with the buttons. I suggested putting a help button that when it is pressed, another window will appear, explaining what all the buttons do, for example for a user who is new to the Rubik's Cube or has never been on an online Rubik's application.

Here is a scan of the paper he used to note his ideas down:


3x3 Rubix cube programme:
1. Mixes all 6 faces by itself and can be mixed by user
2. Shows all 6 faces one at a time or in a net
3. Animates moves
4. Can be solved by user
5. Should solve itself
6. Buttons needed: solve, right, left, front, back, top, bottom, middle column, middle row, + anticlockwise versions of buttons
7. Keep track of moves when robot solves
8. "Chat" button for user's moves
9. ~~Give clues on request~~ Have a reset button + reshuffle button
10. Get the cube to rotate (up, down, left, right clockwise, anticlockwise)
11. Help box for help with the buttons
Ernest Bordenave

## Analysis of user ideas

- Number 1 is just scrambling. I am wondering whether I will use a computer algorithm (Kociemba's algorithm), or scramble with random moves. I will ask my end-user whether he wants to have a textbox which allows him to input how many moves he wants to scramble the cube with.
- For user requirement 3, I will be using the sleep function, although I will need to experiment with how many milliseconds will need to be slept on.
- User requirement 5 is a good point, and I did not think about that. I will need to use a computer algorithm (I have chosen Kociemba's algorithm) instead of a human algorithm.
- In regards to user requirement 6, I will line the buttons up such that the anticlockwise buttons will be underneath their respective clockwise rotations. There were only the moves' initials

shown on the buttons when I first showed my program, so I was thinking that I will need to put "Right clockwise" or "Up anticlockwise". For anticlockwise moves, I think I will need to make separate functions for them, as the animation will be more suitable, instead of animating a clockwise move 270°.

- For number 7, I think I will erase the moves in the solve box one by one once each move has been executed.
- Number 8 is just a log to keep track of what has happened since the program was opened. I am thinking of making a text file which will only be able to be added to. It will include everything that has happened in the Rubiks Cube solver from the moment the log file has been created and works. There will be a variable which will mark the beginning of the current session, which will allow the user to see only what has happened in the current session, or even everything that has happened ever in the Rubik's Cube solver.
- There is no need for number 9. The Reset button is a good idea, as it allows a user to reset the cube if they don't know how to solve the cube. The reshuffle is just a scrambling function.
- I thought for a moment that I could rotate the cube in 3D space, by rotating the 3x3x3 Cube and adding to the permutations of the corner and edge pieces. I mentioned it to my brother and he said yes, that was cool. The net on the Form will also be updated. When animating, I will start from the left- or right- or up- or down-most face, depending on which rotation is being executed, and animate with a wave of the appropriate Facelet colours going in the relevant direction. I will also need to make clockwise and anticlockwise buttons for each of the three ways a Rubiks Cube can be orientated in.
- I will add a Help button which when it is pressed, I will instantiate a new window with a pre-made text, which will be stored in a constant string variable.

## User requirements

1. A reset button to reset the cube back to its original state was already made but is in the user requirements. A reshuffle button is the same as a scrambling button, which will call a scrambling function. The scrambling function will either use a computer algorithm (Kociemba's algorithm) or generate moves at random, depending on how many moves the user wants to scramble the cube with.
2. Animate the moves, both clockwise and anticlockwise, as well as rotations of the whole cube.
3. Have a solve button which calls a solving function which makes use of Kociemba's algorithm.
   3.1 Have a solve box to show the user which moves lead to solving the cube.
   3.2 Erase the moves in the solve box one by one as they are executed by the user.
4. The buttons have to be more explicit in terms of the text that is shown on the buttons, and they will be lined up according to which face of the cube they are moving and in which direction. This also goes with whole cube rotations.
5. There will be a log window which will pop up when a History button is pressed. It will include things like what moves the user has made, any scrambles, and when the solve button has been pressed.
6. Rotate the whole cube in 3D space. This will need to include relevant buttons for each of the three axes in which the cube can be rotated about, both clockwise and anticlockwise. The rotations will also be animated.
7. A Help button is going to be made. When it is pressed, a new window will appear, containing instructions on what the different buttons do.

## Objectives

1. The underlying cube
   1.1 3x3x3 array of cubies
      1.1.1 Make a Cubie class with a CubieType property. Make sure each Cubie has the right number of colours for its CubieType. Preferably instantiate the cubies at run-time.
      1.1.2 Store the cubies' permutations.
      1.1.3 Make a private Cube made of 27 Cubies and preferably instantiate at run time.
   1.2 6x3x3 array of faces
2. Create the clockwise and anticlockwise moves as functions and return the new cube state, which will prevent the Cube from being public.
3. The buttons on the form have to be neatly arranged and explicit, and there will be a button for each clockwise and anticlockwise move function.
4. Make six rotation functions, a clockwise and an anticlockwise rotation for each of the three axes.
   4.1 The underlying cube will be updated first before any animation happens.
5. UI
   5.1 Make a Face class, which will be composed of 9 Facelets in a 3x3 2D array.
   5.2 Display six Faces in a classic net on a Windows Form.
      5.2.1 Make sure that yellow is above and white is on the bottom, so as to not confuse anyone.
      5.2.2 (Optional) Create the faces at run-time.
      5.2.3 (Optional) Recreate Option 2 from ruwix.com.
   5.3 Make a default position where each face is only one colour and show it when the Reset button is clicked.
   5.4 Animate each move, both clockwise and anticlockwise. For 180° rotations, I will call and animate the clockwise move twice. Use the sleep subroutine when animating moves.
   5.5 Animate the rotations. The facelets on the far side will be updated first, and the wave will continue in the direction that the cube is being rotated in. Any faces on the axis of rotation will be animated in a clockwise or anticlockwise manner, and will be animated along with the rest of the cube as it rotates.
   5.6 Make a colour palette so that the user can input individual colours.
      5.6.1 Have a tickbox so the user can say that they are in Editing Mode.
   5.7 Have the move buttons in neat order. Align the clockwise and anticlockwise move buttons with their respective partner.
6. Make a Reset button which will call an algorithm that resets the state of the cube and facelets.
7. Make a scramble function. It will work in conjunction with a textbox(?) in which the user can input how many moves they want the scrambling function to scramble with.
8. A History button will be made.
   8.1 When it is pressed a new window will appear, which will contain a log of everything that has happened with the cube so far in the session.
   8.2 A text file is to be created to keep track of what has happened.
   8.3 An integer variable will contain the line number of the file which starts the current session.
9. A Help button will be created.
   9.1 When it is pressed, a new window will appear containing instructions on how each button works and describe their relevant moves.
   9.2 The text string in the Help window will be stored in a constant variable.
10. The algorithm to solve the cube (Kociemba's algorithm)

# Design – TO BE FINISHED BY 13/10/23

## Programming Diagram

trfdfg

## Overall System Design

This table shows the inputs, processes, storage and outputs of the program.

| Input | Processes | Storage | Output |
|---|---|---|---|
| -Colour palette<br>-Editing Mode button | -Solving algorithm<br>-Reset algorithm<br>-Scrambling algorithm | -3x3x3 cube of cubies<br>-6 faces with 9 facelets | -Classic net<br>-Solving box<br>-Readable log |
| Solve button | Solving algorithm | -state of cube at the start of scrambling<br>-current state of cube. | Moves required to solve the cube in the solve box |
| Move buttons | A function per button | -3x3x3 cube<br>-6x3x3 faces | New state of cube |
| Scrambling button and up-down control | Scrambling algorithm | -state of cube at start of scrambling<br>-state of cube as moves are executed | New state of cube |
| Palette (6 colours) | Change colour of facelets | | Change colour of facelets |

## Underlying structure

A Face and a Facelet class will be made. Each Face will be comprised of 9 Facelets in a 3x3 2D array (objective 5.1). I will instantiate the faces and facelets at run-time (objective 5.2.2). If I go for option 2 from ruwix.com, there will need to be 3 different types of facelet, and I do not yet know how I will connect them with the Cube and animating the moves.

The Cube will be instantiated at run-time, which will be a 3x3x3 3D array of 27 Cubies (objective 1.1.3), which will also be instantiated at run time. A Cubie also has a CubieType, which identifies whether it is a corner, edge, centre, or the middle. Each Cubie will have a Colours array, but depending on its CubieType, a different number of colours will be in the array (objective 1.1.1). The face centres will be CubieType 1; the edges will be CubieType 2; the corners will be CubieType 3, and the unseen middle Cubie as CubieType 0. This is so a Cubie's CubieType is the same as how colours it is allowed to have. This will make it easier in the future when assigning colours, so that the program can check if a cubie has the right number of colours allowed. I don't think it matters in what order the colours are stored, as I can use the Linear Search algorithm on the colour arrays that have a maximum of 3 elements. There will be an integer Permutation variable in the cubie class, which will satisfy objective 1.2.

Cubie class:

| Data item | Property name | Variable type | Purpose |
|---|---|---|---|
| Number of colours | Type | Integer (iCubieType) | This variable will also be used to check whether the cubie is a corner, edge, centre or the middle cubie. It will define how many colours a cubie is allowed to have. |

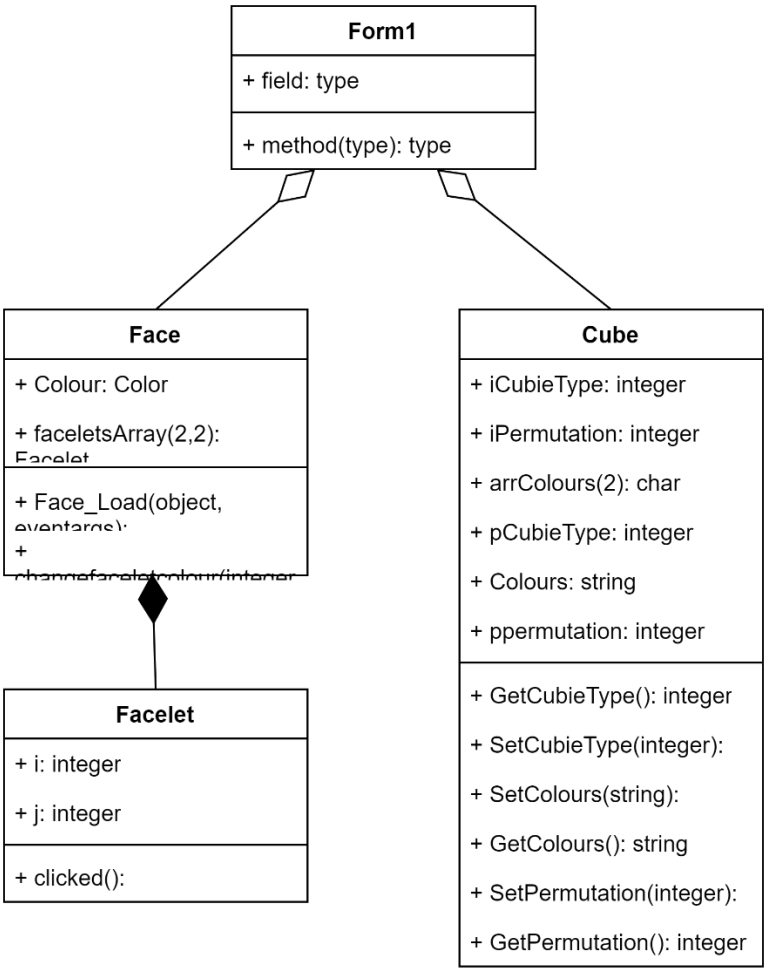
| | | | |
|---|---|---|---|
| The cubie's colours | Colours | An array of characters (arrColours) | This stores the cubie's colours and will have the number of elements as dictated by the Type variable. |
| Permutation | Permutation | Integer (iPermutation) | |

There will also be a 6x3x3 array of faces (objective 1.2), where each face is a 3x3 array of characters which represent the colours. I will order the faces in the following order:

| **Position** | **Face position** |
|---|---|
| 0 | Front |
| 1 | Back |
| 2 | Left |
| 3 | Right |
| 4 | Up |
| 5 | Down |

I will store the facelet colours from left to right and from top to bottom for each face as if I was looking at it.

## Class Diagram



## Moves and Rotations

For the moves, I will first rotate the facelets on screen all at once so that the move is static on-screen, and when the time comes, I will rearrange the lines of code and add sleep lines so that it animates the moves (objective 5.4). I will need to group facelets that will change colour at the same time.

The moves subroutines will all be public and in a Moves module. This will make sure the program satisfies the Excellent Coding style category.

In VB.net, the apostrophe symbol (') is used to comment lines of code. Therefore, I cannot use this symbol when naming my anticlockwise buttons and subroutines. I will replace it with a lowercase 'a' to stand for anticlockwise. When implementing the anticlockwise subroutines, I will simply swap the lines of code of 'opposite' facelets.

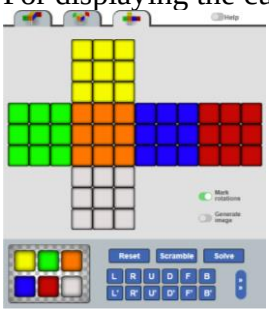
For rotating, I have decided to do the whole rotation in one go. For example, x could also be done with the M, R and L moves in any order. However, for the simplicity of things and to make sure the animation works, I have decided that the rotations will be implemented as their own subroutines.

The problem with testing these is that the facelets on the face being rotated don't change if the move being tested is the first one executed. Therefore, I will have to use a combination of moves and have a physical 3x3x3 Rubik's Cube next to me or compare against a website so I can see if the facelets match.

In general for the moves, I have moved the facelets on the face being rotated, then the neighbouring facelets of the neighbouring faces, in the order top, middle, bottom, or left, middle, right depending on which direction the face is facing.

## UI

For displaying the cube on screen, I will use a simple classic net (objective 5.2):

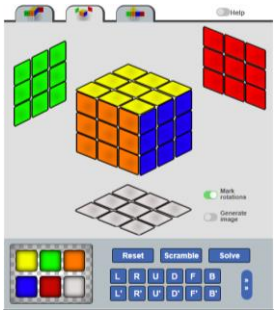



(screenshot from ruwix.com)

I will also put the yellow face on top and the white face on the bottom (objective 5.2.1).

When the program opens, the cube will be reset to the default position. Hence, the default position will occur only when the program opens and when the Reset button is pressed.

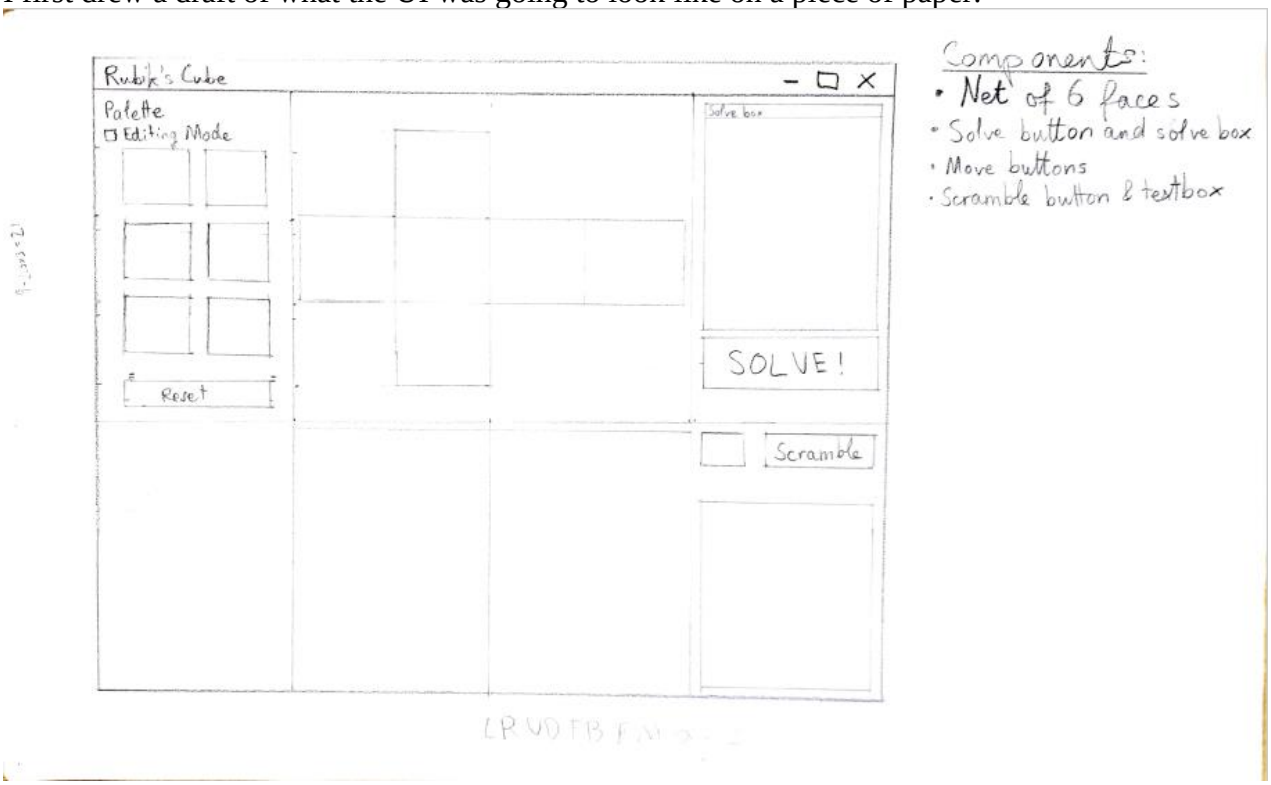
However, if I have enough time after having completed all of the other requirements/objectives of the program, I will try doing option 2 from ruwix.com (objective 5.2.3).



(screenshot from ruwix.com)

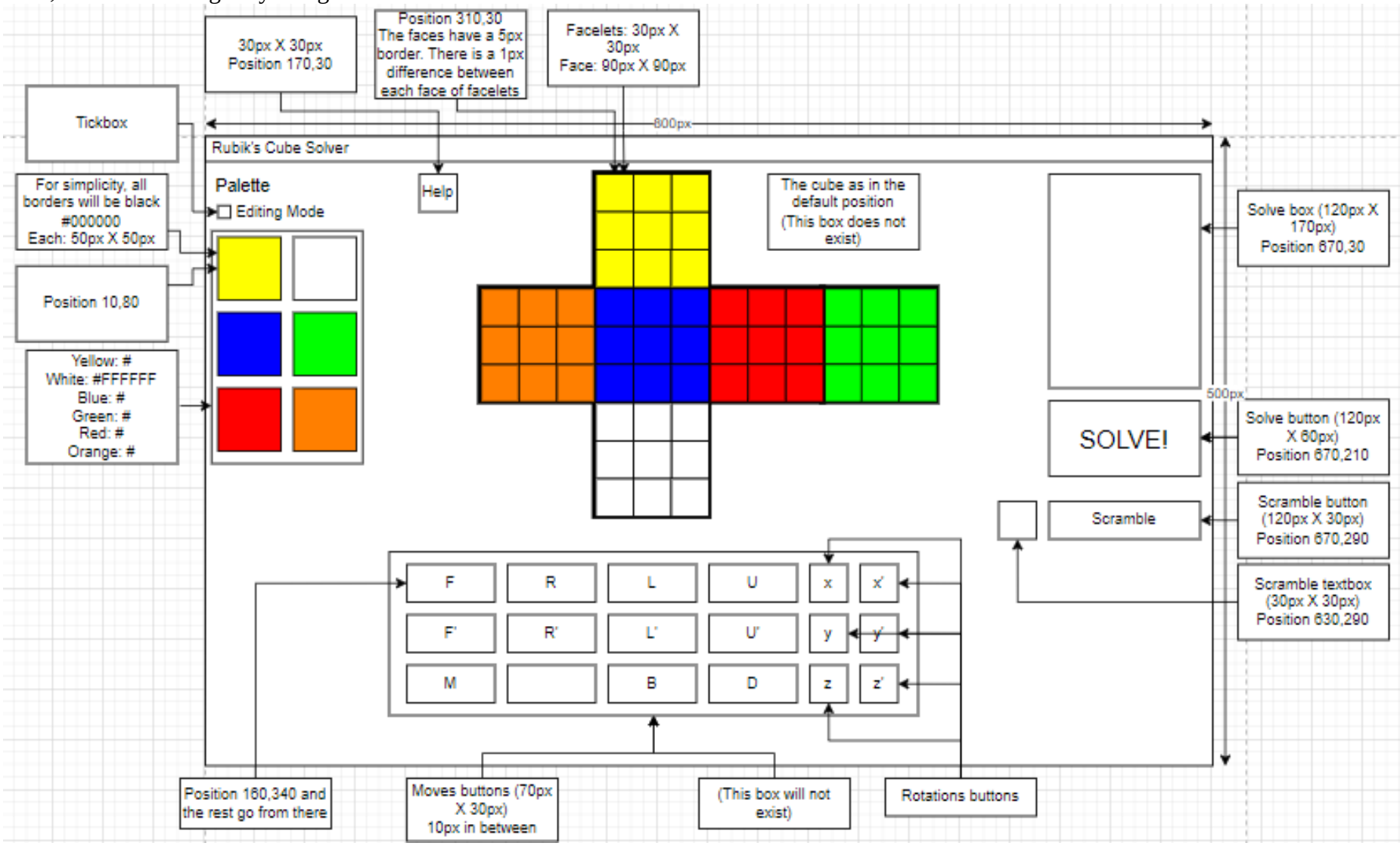
## Designing the UI

I first drew a draft of what the UI was going to look like on a piece of paper:



3x3x3 Rubik's Cube Solver

Next, I recreated it digitally using draw.io and added labels:



## Explanation of the UI design

| Component | Position coordinates and size | Purpose |
|---|---|---|
| Face | Size:90x90 | |
| Facelet | Size:30x30 | |
| Cube net | Positions from top-left corner<br>Y:310,30<br>W:310,210<br>B:310,120<br>G:490,120<br>R:400,120<br>O:220,120 | Show the current state of the cube |
| Solve button | Gdf | ghd |

## Palette

For the palette, I want it to be clear when a palette colour has been clicked. Therefore, the facelets will have to be buttons.
When clicking on a palette colour, if there is no facelet selected, the palette's colour will be stored in memory. If a facelet was selected just before selecting the palette colour, the colour of the facelet will be changed to the palette's colour.
When a facelet is clicked and no palette colour was selected as the last event before the facelt was clicked, the facelet will not change colour, as otherwise it will become too complicated.
If a move happens, all colours and selected facelets will be forgotten.

## Algorithms

### Resetting the cube (meets objective 7)

After some thought, I have realised that I will need a list of all the cubies in order, which will be a constant string array (CubiesColours). I will start with the top left cubie, go from left to right and up to down, and go from front to back. i.e. [UFL, UF, UFR (top front row), FL, F, FR, DFL, DF, DFR (front face), UL, U, UR, L, M, R, DL, D, DR (middle column), UBL, UB, UBR, BL, B, BR, DBL, DB, DBR (back face)]. I will declare the array in the Reset subroutine but I will make a function which assigns the colours of the cubies and call it from the Reset subroutine to keep it tidy.

```
SUB Reset(arrFaces, Cube)
  CubiesColours ← AssignCubiesArray(CubiesColours)

  'Put all cubies in cube
  FOR cubienumber ← 0..26
    FBcolumn ← i/9 'front, middle or back
    LRcolumn ← i mod 3 'L, M or R
    UDrow ← (i mod 9)/3 'U, E or D
    'this prevents 3 nested iteration loops inside each other and uses logic instead
    Cube(i, j, k).Colours ← CubiesColours(i, j, k)

  'Reset all facelets
  FOR facecount ← 0..5:
    FOR i ← 0..2
      FOR j ← 0..2
        CASE SELECT facecount
          1: arrFaces(facecount, i, j) ← "W"
          2: arrFaces(facecount, i, j) ← "Y"
          3: arrFaces(facecount, i, j) ← "B"
          4: arrFaces(facecount, i, j) ← "G"
          5: arrFaces(facecount, i, j) ← "R"
          6: arrFaces(facecount, i, j) ← "O"
```

### CheckSolvable

This algorithm checks if the cube is solvable when the Solve button is pressed. This prevents any unnecessary use of the Solving algorithm. Instead of returning a Boolean value, this function will return a string containing the error in the cube which will then outputted if not empty. This function does not need the cube of cubies as a parameter as it will only check the facelets. It firstly checks if the cube is already solved, then it checks if there ae any centres in the wrong places, then adds up all of the facelets to check if there are enough facelets of each colour.

```
FUNCTION CheckSolvable()
  booleanarray() ← {0, 0, 0, 0, 0, 0}
  solved ← Form1.CheckSolved()
  IF solved = True THEN
    RETURN "The cube has already been solved."
  ENDIF
  IF FaceU.faceletsArray(1, 1).BackColor <> Yellow THEN
    booleanarray(0) ← true
  ELSEIF FaceD.faceletsArray(1, 1).BackColor <> White THEN
    booleanarray(1) ← true
  ELSEIF FaceF.faceletsArray(1, 1).BackColor <> Blue THEN
    booleanarray(2) ← true
  ELSEIF FaceB.faceletsArray(1, 1).BackColor <> Green THEN
    booleanarray(3) ← true
  ELSEIF FaceR.faceletsArray(1, 1).BackColor <> Red THEN
    booleanarray(4) ← true
  ELSEIF FaceL.faceletsArray(1, 1).BackColor <> Orange THEN
    booleanarray(5) ← true
  ENDIF
  iWrongCentres ← 0
  FOR i = 0 TO 5
    IF booleanarray(i) = True THEN
      iWrongCentres ← iWrongCentres + 1
    ENDIF
  NEXT
  i ← 0
  IF iWrongCentres <> 0 THEN
    returnstring = "The "
    WHILE iWrongCentres > 0
      IF booleanarray(i) = True THEN
        SELECT CASE i
          CASE 0
            returnstirng ← returnstring + "yellow"
          CASE 1
            returnstirng ← returnstring + "white"
          CASE 2
            returnstirng ← returnstring + "blue"
          CASE 3
            returnstirng ← returnstring + "green"
          CASE 4
            returnstirng ← returnstring + "red"
          CASE 5
            returnstirng ← returnstring + "orange"
        END SELECT
        IF iWrongCentres > 1 THEN
          returnstring ← returnstring + ", "
        ENDIF
      ENDIF
      iWrongCentres ← iWrongCentres – 1
      i ← i + 1
    ENDWHILE
    returnstring ← returnstirng + " centres are in the wrong place."
    RETURN returnstring
  ENDIF
  RETURN ""
```

## CheckSolved

This algorithm checks if the cube has been solved after every move. It will be in its own subroutine in the Form's code and returns a Boolean value. I would rather have the Boolean cubesolved variable begin as true and turn false, as the majority of the time when the function is called, it will be after a move, so less facelets will be checked before one is spotted to be an incorrect colour. It will not be called when the cube is reset. The subroutine has to check all facelets of all facelets in order to know that the cube is solved.

```
FUNCTION CheckSolved()
  cubesolved ← True
  faceletnumber ← 0
  WHILE cubesolved = True
    i ← faceletnumber / 3
    j ← faceletnumber mod 3
    IF FaceW.faceletsArray(i, j).BackColor <> White THEN
      cubesolved ← False
    ELSEIF FaceY.faceletsArray(i, j).BackColor <> Yellow THEN
      cubesolved ← False
    ELSEIF FaceB.faceletsArray(i, j).BackColor <> Blue THEN
      cubesolved ← False
    ELSEIF FaceG.faceletsArray(i, j).BackColor <> Green THEN
      cubesolved ← False
    ELSEIF FaceR.faceletsArray(i, j).BackColor <> Red THEN
      cubesolved ← False
    ELSEIF FaceO.faceletsArray(i, j).BackColor <> Orange THEN
      cubesolved ← False
    ENDIF
    faceletnumber ← faceletnumber + 1
  ENDWHILE
  RETURN cubesolved
```

## Scrambling Algorithm

### Scrambling

Obviously, there will need to be a scrambling function so that the user has the challenge to solve it. I wondered if the scrambling function would continue scrambling from the current state of the cube, or from the default position. It would be more sensible to continue from the current state, because if the user wants the cube to be scrambled from the default position, they can just press the Reset button and then scramble.

The program will generate a random series of moves and execute them.

It would be nice to have a textbox in which the user can input how many moves they would like the program to execute to scramble the cube. To prevent errors, the textbox will be an up-down control with up and down arrows, which the user can press if they want more or less moves in the scramble. There could also be a tick box if the user doesn't care about how many moves there will be in the scrambling process. I asked my brother what he thought about that and what range he would like, and he said _ to __. The program will choose a random number between 5 and 20.

I will keep a score of the moves needed to solve the cube. Each time the user makes a move, I will add it on to the beginning of the list of moves needed to solve the cube. When hitting 'solve', I will count the number of moves required to solve the cube and compare it with __, which is the maximum number of moves which Kociemba's algorithm solves any cube in. If the number of moves in the score exceeds __, a new solving process will begin.

I don't know yet if the scrambling process will be animated or if the facelets will be updated all in one go. I will ask my end-user.

There will be an up-down control in which the user can enter how many moves they would like to be executed by the program to scramble the cube. It is an up-down control instead of a normal textbox to prevent errors form occuring if the user types in text. I found that the NumericUpDown control in the Visual Studio Windows Forms Application has a maximum limit of 100. Therefore, I have decided to put my own maximum limit of 30 so that my program doesn't have to do lots of moves if the user puts a large number in. It is also possible to enter a numeric value manually.

For the scramble function, the program will generate a random series of moves and execute them on the current cube's state. I will also need to keep track of the permutations of the cubies since the start of the scrambling process.

I have realised with the way the subroutines are ranked, I will have to implement this subroutine in the Form's code instead of in the Moves module, as the moves subroutines are privately declared in the Form.

It will keep a score of the moves needed to solve the cube. Each time the user makes a move, it will be added on to the beginning of the list of moves needed to solve the cube. When hitting 'solve', I will count the number of moves required to solve the cube and compare it with __, which is the maximum number of moves which Kociemba's algorithm solves any cube in. If the number of moves in the score exceeds __, a new solving process will begin.
[FLOWCHART]

### Solving Algorithm (Kociemba's Algorithm)

The solving process will use Kociemba's algorithm. When the solve button is pressed, a function containing Kociemba's algorithm will be called. This function will be contained in the Moves module.

When solving, I will need to store the cube's state at the start of being solved, and not update what is being outputted to the screen as the moves in the solving process are being explored. If the state is in G1, it will check if the cube is compatible with G2. If it is in G2, it will call the normal CheckSolved function.

There is going to be a read-only solve box which will output which moves are needed to solve the cube. The moves in the solve box will be separated by spaces, and will be outputted into the box using an iteration loop which adds a space after each move. It doesn't matter if there is a space after the last move as it will not be seen, because the solve box will be read-only. If the next move executed by the user matches the first move in the solve box, the move will be erased from the solve box along with the space that follows it. If the user goes off track, the solve box will reset its contents and go empty.

## Implementation

I will first put the code for the Face, Facelet and Cubie classes, and then the Moves and PaletteChanges modules, and finally for the Form.

### Face

```
1   Public Class Face
2       Dim colour As Color
3       Public faceletsArray(2, 2) As Facelet
        0 references
4       Private Sub Face_Load(sender As Object, e As EventArgs) Handles MyBase.Load
5           Dim i As Integer
6           Dim j As Integer
7           For faceletnumber = 0 To 8
8               i = faceletnumber Mod 3
9               j = Math.Floor(faceletnumber / 3)
10              faceletsArray(i, j) = New Facelet
11              faceletsArray(i, j).i = i
12              faceletsArray(i, j).j = j
13              faceletsArray(i, j).Left = i * 30
14              faceletsArray(i, j).Top = j * 30
15              Me.Controls.Add(faceletsArray(i, j))
16          Next
17      End Sub
18
        1 reference
19      Public Sub changefaceletcolour(i, j)
20          faceletsArray(i, j).BackColor = PaletteChanges.currentpalettecolour
21      End Sub
22  End Class
```

Line 3: This is an example of composition, which is a Group A

Line 10: The facelets are only instantiated when the face is instantiated (Group A).

### Facelet

```
1   Public Class Facelet
2       Public i As Integer
3       Public j As Integer
        0 references
4       Public Sub clicked() Handles Me.Click
5           Dim parentface As Face = Me.Parent
6           If Form1.checkEditingMode() = True Then
7               parentface.changefaceletcolour(i, j)
8           End If
9       End Sub
10  End Class
```

Line 5 makes use of object-oriented programming and gets the instance of the parent class which the current facelet instance is part of (Group A).

This IF statement going from lines 6 to 8 checks if the Editing Mode tickbox has been ticked before changing the facelet's colour (OOP Group A)

### Cubie

This class makes excellent use of defensive programming (Excellent Coding style), as inputs are passed into a subroutine, then the property, then the variable, instead of directly into the variable. This is to check that the input meets the standards.

```
    29 references
1   Public Class Cubie
2       Dim iCubieType As Integer '0:middle 1:centre 2:edge 3:corner
3       Dim iPermutation As Integer
4       Dim arrColours(2) As Char
```

## Property CubieType

```
2 references
Property pCubieType
    Set(value)
        If 0 <= value And value < 4 Then
            iCubieType = value
        Else
            MsgBox("Hello I'm going to annoy you bc you entered an incorrect cubie type.")
        End If
    End Set
    Get
        Return iCubieType
    End Get
End Property
0 references
Public Function GetCubieType()
    Return pCubieType
End Function
4 references
Public Sub SetCubieType(type As Integer)
    pCubieType = type
End Sub
```

This is an example of defensive programming, as a cubietype can only be one of three types.

## Property Colours

```
2 references
Property Colours As String
    Set(value As String)
        For i = 1 To value.Length
            arrColours(i - 1) = Mid(value, i, 1)
        Next
    End Set
    Get
        Dim returnstring As String = ""
        For i = 1 To arrColours.Length
            returnstring = returnstring + arrColours(i - 1)
        Next
        Return returnstring
    End Get
End Property

1 reference
Public Sub SetColours(value As String)
    If value.Length <> iCubieType Then
        MsgBox("Incorrect number of colours entered for this cubie")
    Else
        Colours = value
    End If
End Sub

0 references
Public Function GetColours()
    Return Colours
End Function
```

The array arrColours receives the number of colours that is passed into the property.

This returns the colours stored in the arrColours array.

## Property Permutation

```
2 references
Property ppermutation As Integer
    Set(value As Integer)
        If value < iCubieType And value >= 0 Then
            If value > 2 Then
                value = value - 3
            End If
            iPermutation = value
        End If
    End Set
    Get
        Return iPermutation
    End Get
End Property

1 reference
Public Sub SetPermutation(value As Integer)
    If value < iCubieType And value >= 0 Then
        If value > 2 Then
            value = value - 3
        End If
        ppermutation = value
    Else
        MsgBox("An error occured.")
    End If
End Sub

0 references
Public Function GetPermutation()
    Return ppermutation
End Function
End Class
```

## Moves

The majority of subroutines in this module involve complicated user-made algorithms (Group A), which required a lot of visualization to implement them.

### reset

```
2 references
Public Sub reset(Cube(,,) As Cubie,
                 FaceW As Face, FaceY As Face,
                 FaceG As Face, FaceB As Face,
                 FaceR As Face, FaceO As Face)

    Dim i As Integer
    Dim j As Integer
    Dim k As Integer
    Dim CubiesColours(2, 2, 2) As String 'actually a constant
    CubiesColours = AssignCubiesArray(CubiesColours)

    For cubienumber = 0 To 26
        i = cubienumber Mod 3
        j = Math.Floor((i Mod 9) / 3)
        k = Math.Floor(cubienumber / 9)
        'this prevents three nested loops inside each other and uses logic instead

        Cube(i, j, k).SetPermutation(0)
        Cube(i, j, k).SetColours(CubiesColours(i, j, k)) 'assigns each cubie's colours
    Next

    For i = 0 To 2
        For j = 0 To 2
            FaceW.faceletsArray(i, j).BackColor = Color.White
            FaceY.faceletsArray(i, j).BackColor = Color.Yellow
            FaceG.faceletsArray(i, j).BackColor = Color.Green
            FaceB.faceletsArray(i, j).BackColor = Color.Blue
            FaceR.faceletsArray(i, j).BackColor = Color.Red
            FaceO.faceletsArray(i, j).BackColor = Color.Orange
        Next
    Next

End Sub
```

This calls a subroutine so that the array CubiesColours can be populated with colours by position.

### AssignCubiesArray

```
1 reference
Private Function AssignCubiesArray(Cube(,,) As String)
    Dim i As Integer
    Dim j As Integer
    Dim k As Integer
    Dim currentcubiecolours As String = ""

    For cubienumber = 0 To 26
        i = cubienumber Mod 3
        j = Math.Floor((i Mod 9) / 3)
        k = Math.Floor(cubienumber / 9)
        'this prevents three nested loops inside each other and uses logic instead

        If i = 0 Then
            currentcubiecolours = currentcubiecolours + "O"
        ElseIf i = 2 Then
            currentcubiecolours = currentcubiecolours + "R"
        End If
        If j = 0 Then 'front face
            currentcubiecolours = currentcubiecolours + "Y"
        ElseIf j = 2 Then
            currentcubiecolours = currentcubiecolours + "W"
        End If
        If k = 0 Then
            currentcubiecolours = currentcubiecolours + "B"
        ElseIf k = 2 Then
            currentcubiecolours = currentcubiecolours + "G"
        End If

        Cube(i, j, k) = currentcubiecolours
        currentcubiecolours = ""
    Next

    Return Cube
End Function
```

L

```
4 references
Public Sub L(Cube(,,) As Cubie, FaceL As Face,
             FaceF As Face, FaceB As Face,
             FaceU As Face, FaceD As Face)

    'FaceL corners
    Dim tempcolor As Color = FaceL.faceletsArray(2, 2).BackColor 'FLD facing left
    FaceL.faceletsArray(2, 2).BackColor = FaceL.faceletsArray(2, 0).BackColor
    FaceL.faceletsArray(2, 0).BackColor = FaceL.faceletsArray(0, 0).BackColor
    FaceL.faceletsArray(0, 0).BackColor = FaceL.faceletsArray(0, 2).BackColor
    FaceL.faceletsArray(0, 2).BackColor = tempcolor

    'FaceL edges
    tempcolor = FaceL.faceletsArray(1, 0).BackColor 'UL facing left
    FaceL.faceletsArray(1, 0).BackColor = FaceL.faceletsArray(0, 1).BackColor
    FaceL.faceletsArray(0, 1).BackColor = FaceL.faceletsArray(1, 2).BackColor
    FaceL.faceletsArray(1, 2).BackColor = FaceL.faceletsArray(2, 1).BackColor
    FaceL.faceletsArray(2, 1).BackColor = tempcolor

    'UL other faces
    tempcolor = FaceF.faceletsArray(0, 0).BackColor 'FLU facing front
    FaceF.faceletsArray(0, 0).BackColor = FaceU.faceletsArray(0, 0).BackColor
    FaceU.faceletsArray(0, 0).BackColor = FaceB.faceletsArray(2, 2).BackColor
    FaceB.faceletsArray(2, 2).BackColor = FaceD.faceletsArray(0, 0).BackColor
    FaceD.faceletsArray(0, 0).BackColor = tempcolor

    'DL other faces
    tempcolor = FaceF.faceletsArray(0, 2).BackColor 'BLU facing front
    FaceF.faceletsArray(0, 2).BackColor = FaceU.faceletsArray(0, 2).BackColor
    FaceU.faceletsArray(0, 2).BackColor = FaceB.faceletsArray(2, 0).BackColor
    FaceB.faceletsArray(2, 0).BackColor = FaceD.faceletsArray(0, 2).BackColor
    FaceD.faceletsArray(0, 2).BackColor = tempcolor

    'other faces edges
    tempcolor = FaceF.faceletsArray(0, 1).BackColor 'FL facing front
    FaceF.faceletsArray(0, 1).BackColor = FaceU.faceletsArray(0, 1).BackColor
    FaceU.faceletsArray(0, 1).BackColor = FaceB.faceletsArray(2, 1).BackColor
    FaceB.faceletsArray(2, 1).BackColor = FaceD.faceletsArray(0, 1).BackColor
    FaceD.faceletsArray(0, 1).BackColor = tempcolor

End Sub
```

R

```
4 references
Public Sub R(Cube(,,) As Cubie, FaceR As Face,
             FaceF As Face, FaceB As Face,
             FaceU As Face, FaceD As Face)

    'FaceR corners
    Dim tempcolor As Color = FaceR.faceletsArray(2, 2).BackColor 'BRD facing right
    FaceR.faceletsArray(2, 2).BackColor = FaceR.faceletsArray(2, 0).BackColor
    FaceR.faceletsArray(2, 0).BackColor = FaceR.faceletsArray(0, 0).BackColor
    FaceR.faceletsArray(0, 0).BackColor = FaceR.faceletsArray(0, 2).BackColor
    FaceR.faceletsArray(0, 2).BackColor = tempcolor

    'FaceR edges
    tempcolor = FaceR.faceletsArray(1, 0).BackColor 'RUE facing right
    FaceR.faceletsArray(1, 0).BackColor = FaceR.faceletsArray(0, 1).BackColor
    FaceR.faceletsArray(0, 1).BackColor = FaceR.faceletsArray(1, 2).BackColor
    FaceR.faceletsArray(1, 2).BackColor = FaceR.faceletsArray(2, 1).BackColor
    FaceR.faceletsArray(2, 1).BackColor = tempcolor

    'UR other faces
    tempcolor = FaceF.faceletsArray(2, 0).BackColor 'FRU facing front
    FaceF.faceletsArray(2, 0).BackColor = FaceD.faceletsArray(2, 0).BackColor
    FaceD.faceletsArray(2, 0).BackColor = FaceB.faceletsArray(0, 2).BackColor
    FaceB.faceletsArray(0, 2).BackColor = FaceU.faceletsArray(2, 0).BackColor
    FaceU.faceletsArray(2, 0).BackColor = tempcolor

    'DR other faces
    tempcolor = FaceF.faceletsArray(2, 2).BackColor
    FaceF.faceletsArray(2, 2).BackColor = FaceD.faceletsArray(2, 2).BackColor
    FaceD.faceletsArray(2, 2).BackColor = FaceB.faceletsArray(0, 0).BackColor
    FaceB.faceletsArray(0, 0).BackColor = FaceU.faceletsArray(2, 2).BackColor
    FaceU.faceletsArray(2, 2).BackColor = tempcolor

    'other faces edges
    tempcolor = FaceF.faceletsArray(2, 1).BackColor
    FaceF.faceletsArray(2, 1).BackColor = FaceD.faceletsArray(2, 1).BackColor
    FaceD.faceletsArray(2, 1).BackColor = FaceB.faceletsArray(0, 1).BackColor
    FaceB.faceletsArray(0, 1).BackColor = FaceU.faceletsArray(2, 1).BackColor
    FaceU.faceletsArray(2, 1).BackColor = tempcolor

End Sub
```

U

```
4 references
Public Sub U(Cube(,,) As Cubie, FaceU As Face,
             FaceF As Face, FaceB As Face,
             FaceR As Face, FaceL As Face)

    'FaceU corners
    Dim tempcolor As Color = FaceU.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceU.faceletsArray(0, 0).BackColor = FaceU.faceletsArray(0, 2).BackColor
    FaceU.faceletsArray(0, 2).BackColor = FaceU.faceletsArray(2, 2).BackColor
    FaceU.faceletsArray(2, 2).BackColor = FaceU.faceletsArray(2, 0).BackColor
    FaceU.faceletsArray(2, 0).BackColor = tempcolor

    'FaceU edges
    tempcolor = FaceU.faceletsArray(1, 0).BackColor 'UB facing up
    FaceU.faceletsArray(1, 0).BackColor = FaceU.faceletsArray(0, 1).BackColor
    FaceU.faceletsArray(0, 1).BackColor = FaceU.faceletsArray(1, 2).BackColor
    FaceU.faceletsArray(1, 2).BackColor = FaceU.faceletsArray(2, 1).BackColor
    FaceU.faceletsArray(2, 1).BackColor = tempcolor

    'UL other faces
    tempcolor = FaceL.faceletsArray(0, 0).BackColor 'BLU facing left
    FaceL.faceletsArray(0, 0).BackColor = FaceF.faceletsArray(0, 0).BackColor
    FaceF.faceletsArray(0, 0).BackColor = FaceR.faceletsArray(0, 0).BackColor
    FaceR.faceletsArray(0, 0).BackColor = FaceB.faceletsArray(0, 0).BackColor
    FaceB.faceletsArray(0, 0).BackColor = tempcolor

    'UR other faces
    tempcolor = FaceL.faceletsArray(2, 0).BackColor 'FLU facing left
    FaceL.faceletsArray(2, 0).BackColor = FaceF.faceletsArray(2, 0).BackColor
    FaceF.faceletsArray(2, 0).BackColor = FaceR.faceletsArray(2, 0).BackColor
    FaceR.faceletsArray(2, 0).BackColor = FaceB.faceletsArray(2, 0).BackColor
    FaceB.faceletsArray(2, 0).BackColor = tempcolor

    'other faces edges
    tempcolor = FaceL.faceletsArray(1, 0).BackColor 'UL facing left
    FaceL.faceletsArray(1, 0).BackColor = FaceF.faceletsArray(1, 0).BackColor
    FaceF.faceletsArray(1, 0).BackColor = FaceR.faceletsArray(1, 0).BackColor
    FaceR.faceletsArray(1, 0).BackColor = FaceB.faceletsArray(1, 0).BackColor
    FaceB.faceletsArray(1, 0).BackColor = tempcolor

End Sub
```

D

```
4 references
Public Sub D(Cube(,,) As Cubie, FaceD As Face,
             FaceF As Face, FaceB As Face,
             FaceR As Face, FaceL As Face)

    'FaceD corners
    Dim tempcolor As Color = FaceD.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceD.faceletsArray(0, 0).BackColor = FaceD.faceletsArray(0, 2).BackColor
    FaceD.faceletsArray(0, 2).BackColor = FaceD.faceletsArray(2, 2).BackColor
    FaceD.faceletsArray(2, 2).BackColor = FaceD.faceletsArray(2, 0).BackColor
    FaceD.faceletsArray(2, 0).BackColor = tempcolor

    'FaceD edges
    tempcolor = FaceD.faceletsArray(1, 0).BackColor 'UB facing up
    FaceD.faceletsArray(1, 0).BackColor = FaceD.faceletsArray(0, 1).BackColor
    FaceD.faceletsArray(0, 1).BackColor = FaceD.faceletsArray(1, 2).BackColor
    FaceD.faceletsArray(1, 2).BackColor = FaceD.faceletsArray(2, 1).BackColor
    FaceD.faceletsArray(2, 1).BackColor = tempcolor

    'DL other faces
    tempcolor = FaceL.faceletsArray(0, 2).BackColor 'BLU facing left
    FaceL.faceletsArray(0, 2).BackColor = FaceB.faceletsArray(0, 2).BackColor
    FaceB.faceletsArray(0, 2).BackColor = FaceR.faceletsArray(0, 2).BackColor
    FaceR.faceletsArray(0, 2).BackColor = FaceF.faceletsArray(0, 2).BackColor
    FaceF.faceletsArray(0, 2).BackColor = tempcolor

    'DR other faces
    tempcolor = FaceL.faceletsArray(2, 2).BackColor 'FLU facing left
    FaceL.faceletsArray(2, 2).BackColor = FaceB.faceletsArray(2, 2).BackColor
    FaceB.faceletsArray(2, 2).BackColor = FaceR.faceletsArray(2, 2).BackColor
    FaceR.faceletsArray(2, 2).BackColor = FaceF.faceletsArray(2, 2).BackColor
    FaceF.faceletsArray(2, 2).BackColor = tempcolor

    'other faces D edges
    tempcolor = FaceL.faceletsArray(1, 2).BackColor 'UL facing left
    FaceL.faceletsArray(1, 2).BackColor = FaceB.faceletsArray(1, 2).BackColor
    FaceB.faceletsArray(1, 2).BackColor = FaceR.faceletsArray(1, 2).BackColor
    FaceR.faceletsArray(1, 2).BackColor = FaceF.faceletsArray(1, 2).BackColor
    FaceF.faceletsArray(1, 2).BackColor = tempcolor

End Sub
```

F

```
4 references
Public Sub F(Cube(,,) As Cubie, FaceF As Face,
             FaceU As Face, FaceD As Face,
             FaceR As Face, FaceL As Face)

    'FaceF corners
    Dim tempcolor As Color = FaceF.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceF.faceletsArray(0, 0).BackColor = FaceF.faceletsArray(0, 2).BackColor
    FaceF.faceletsArray(0, 2).BackColor = FaceF.faceletsArray(2, 2).BackColor
    FaceF.faceletsArray(2, 2).BackColor = FaceF.faceletsArray(2, 0).BackColor
    FaceF.faceletsArray(2, 0).BackColor = tempcolor

    'FaceF edges
    tempcolor = FaceF.faceletsArray(1, 0).BackColor 'UB facing up
    FaceF.faceletsArray(1, 0).BackColor = FaceF.faceletsArray(0, 1).BackColor
    FaceF.faceletsArray(0, 1).BackColor = FaceF.faceletsArray(1, 2).BackColor
    FaceF.faceletsArray(1, 2).BackColor = FaceF.faceletsArray(2, 1).BackColor
    FaceF.faceletsArray(2, 1).BackColor = tempcolor

    'DR other faces from L
    tempcolor = FaceL.faceletsArray(2, 2).BackColor 'FLD facing front
    FaceL.faceletsArray(2, 2).BackColor = FaceD.faceletsArray(2, 0).BackColor
    FaceD.faceletsArray(2, 0).BackColor = FaceR.faceletsArray(0, 0).BackColor
    FaceR.faceletsArray(0, 0).BackColor = FaceU.faceletsArray(0, 2).BackColor
    FaceU.faceletsArray(0, 2).BackColor = tempcolor

    'UR other faces from L
    tempcolor = FaceL.faceletsArray(2, 0).BackColor 'FLU facing front
    FaceL.faceletsArray(2, 0).BackColor = FaceD.faceletsArray(0, 0).BackColor
    FaceD.faceletsArray(0, 0).BackColor = FaceR.faceletsArray(0, 2).BackColor
    FaceR.faceletsArray(0, 2).BackColor = FaceU.faceletsArray(2, 2).BackColor
    FaceU.faceletsArray(2, 2).BackColor = tempcolor

    'other faces edges from L
    tempcolor = FaceL.faceletsArray(2, 1).BackColor 'FL facing front
    FaceL.faceletsArray(2, 1).BackColor = FaceD.faceletsArray(1, 0).BackColor
    FaceD.faceletsArray(1, 0).BackColor = FaceR.faceletsArray(0, 1).BackColor
    FaceR.faceletsArray(0, 1).BackColor = FaceU.faceletsArray(1, 2).BackColor
    FaceU.faceletsArray(1, 2).BackColor = tempcolor

End Sub
```

B

```
4 references
Public Sub B(Cube(,,) As Cubie, FaceB As Face,
             FaceU As Face, FaceD As Face,
             FaceR As Face, FaceL As Face)

    'FaceB corners
    Dim tempcolor As Color = FaceB.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceB.faceletsArray(0, 0).BackColor = FaceB.faceletsArray(0, 2).BackColor
    FaceB.faceletsArray(0, 2).BackColor = FaceB.faceletsArray(2, 2).BackColor
    FaceB.faceletsArray(2, 2).BackColor = FaceB.faceletsArray(2, 0).BackColor
    FaceB.faceletsArray(2, 0).BackColor = tempcolor

    'FaceB edges
    tempcolor = FaceB.faceletsArray(1, 0).BackColor 'UB facing up
    FaceB.faceletsArray(1, 0).BackColor = FaceB.faceletsArray(0, 1).BackColor
    FaceB.faceletsArray(0, 1).BackColor = FaceB.faceletsArray(1, 2).BackColor
    FaceB.faceletsArray(1, 2).BackColor = FaceB.faceletsArray(2, 1).BackColor
    FaceB.faceletsArray(2, 1).BackColor = tempcolor

    'UL other faces from U
    tempcolor = FaceU.faceletsArray(0, 0).BackColor 'UBL facing up
    FaceU.faceletsArray(0, 0).BackColor = FaceR.faceletsArray(2, 0).BackColor
    FaceR.faceletsArray(2, 0).BackColor = FaceD.faceletsArray(2, 2).BackColor
    FaceD.faceletsArray(2, 2).BackColor = FaceL.faceletsArray(0, 2).BackColor
    FaceL.faceletsArray(0, 2).BackColor = tempcolor

    'UR other faces from U
    tempcolor = FaceU.faceletsArray(2, 0).BackColor 'FLU facing front
    FaceU.faceletsArray(2, 0).BackColor = FaceR.faceletsArray(2, 2).BackColor
    FaceR.faceletsArray(2, 2).BackColor = FaceD.faceletsArray(0, 2).BackColor
    FaceD.faceletsArray(0, 2).BackColor = FaceL.faceletsArray(0, 0).BackColor
    FaceL.faceletsArray(0, 0).BackColor = tempcolor

    'other faces edges from U
    tempcolor = FaceU.faceletsArray(1, 0).BackColor 'FL facing front
    FaceU.faceletsArray(1, 0).BackColor = FaceR.faceletsArray(2, 1).BackColor
    FaceR.faceletsArray(2, 1).BackColor = FaceD.faceletsArray(1, 2).BackColor
    FaceD.faceletsArray(1, 2).BackColor = FaceL.faceletsArray(0, 1).BackColor
    FaceL.faceletsArray(0, 1).BackColor = tempcolor

End Sub
```

E

```
4 references
Public Sub E(Cube(,,) As Cubie,
             FaceF As Face, FaceB As Face,
             FaceR As Face, FaceL As Face)

    'centres start from F
    Dim tempcolor As Color = FaceF.faceletsArray(1, 1).BackColor
    FaceF.faceletsArray(1, 1).BackColor = FaceR.faceletsArray(1, 1).BackColor
    FaceR.faceletsArray(1, 1).BackColor = FaceB.faceletsArray(1, 1).BackColor
    FaceB.faceletsArray(1, 1).BackColor = FaceL.faceletsArray(1, 1).BackColor
    FaceL.faceletsArray(1, 1).BackColor = tempcolor

    'L start from F
    tempcolor = FaceF.faceletsArray(0, 1).BackColor
    FaceF.faceletsArray(0, 1).BackColor = FaceR.faceletsArray(0, 1).BackColor
    FaceR.faceletsArray(0, 1).BackColor = FaceB.faceletsArray(0, 1).BackColor
    FaceB.faceletsArray(0, 1).BackColor = FaceL.faceletsArray(0, 1).BackColor
    FaceL.faceletsArray(0, 1).BackColor = tempcolor

    'R start from F
    tempcolor = FaceF.faceletsArray(2, 1).BackColor
    FaceF.faceletsArray(2, 1).BackColor = FaceR.faceletsArray(2, 1).BackColor
    FaceR.faceletsArray(2, 1).BackColor = FaceB.faceletsArray(2, 1).BackColor
    FaceB.faceletsArray(2, 1).BackColor = FaceL.faceletsArray(2, 1).BackColor
    FaceL.faceletsArray(2, 1).BackColor = tempcolor

End Sub
```

S

```
4 references
Public Sub S(Cube(,,) As Cubie,
             FaceU As Face, FaceD As Face,
             FaceR As Face, FaceL As Face)

    'centres start from U
    Dim tempcolor As Color = FaceU.faceletsArray(1, 1).BackColor
    FaceU.faceletsArray(1, 1).BackColor = FaceL.faceletsArray(1, 1).BackColor
    FaceL.faceletsArray(1, 1).BackColor = FaceD.faceletsArray(1, 1).BackColor
    FaceD.faceletsArray(1, 1).BackColor = FaceR.faceletsArray(1, 1).BackColor
    FaceR.faceletsArray(1, 1).BackColor = tempcolor

    'L start from U
    tempcolor = FaceU.faceletsArray(0, 1).BackColor
    FaceU.faceletsArray(0, 1).BackColor = FaceL.faceletsArray(1, 2).BackColor
    FaceL.faceletsArray(1, 2).BackColor = FaceD.faceletsArray(2, 1).BackColor
    FaceD.faceletsArray(2, 1).BackColor = FaceR.faceletsArray(1, 0).BackColor
    FaceR.faceletsArray(1, 0).BackColor = tempcolor

    'R start from U
    tempcolor = FaceU.faceletsArray(2, 1).BackColor
    FaceU.faceletsArray(2, 1).BackColor = FaceL.faceletsArray(1, 0).BackColor
    FaceL.faceletsArray(1, 0).BackColor = FaceD.faceletsArray(0, 1).BackColor
    FaceD.faceletsArray(0, 1).BackColor = FaceR.faceletsArray(1, 2).BackColor
    FaceR.faceletsArray(1, 2).BackColor = tempcolor

End Sub
```

M

```
4 references
Public Sub M(Cube(,,) As Cubie,
             FaceF As Face, FaceU As Face,
             FaceB As Face, FaceD As Face)

    'centres start from F
    Dim tempcolor As Color = FaceF.faceletsArray(1, 1).BackColor
    FaceF.faceletsArray(1, 1).BackColor = FaceD.faceletsArray(1, 1).BackColor
    FaceD.faceletsArray(1, 1).BackColor = FaceB.faceletsArray(1, 1).BackColor
    FaceB.faceletsArray(1, 1).BackColor = FaceU.faceletsArray(1, 1).BackColor
    FaceU.faceletsArray(1, 1).BackColor = tempcolor

    'UM start from F
    tempcolor = FaceF.faceletsArray(1, 0).BackColor
    FaceF.faceletsArray(1, 0).BackColor = FaceD.faceletsArray(1, 0).BackColor
    FaceD.faceletsArray(1, 0).BackColor = FaceB.faceletsArray(1, 2).BackColor
    FaceB.faceletsArray(1, 2).BackColor = FaceU.faceletsArray(1, 0).BackColor
    FaceU.faceletsArray(1, 0).BackColor = tempcolor

    'DM start from F
    tempcolor = FaceF.faceletsArray(1, 2).BackColor
    FaceF.faceletsArray(1, 2).BackColor = FaceD.faceletsArray(1, 2).BackColor
    FaceD.faceletsArray(1, 2).BackColor = FaceB.faceletsArray(1, 0).BackColor
    FaceB.faceletsArray(1, 0).BackColor = FaceU.faceletsArray(1, 2).BackColor
    FaceU.faceletsArray(1, 2).BackColor = tempcolor

End Sub
```

La

```
2 references
Public Sub La(Cube(,,) As Cubie, FaceL As Face,
              FaceF As Face, FaceB As Face,
              FaceU As Face, FaceD As Face)

    'FaceL corners
    Dim tempcolor As Color = FaceL.faceletsArray(2, 2).BackColor 'FLD facing left
    FaceL.faceletsArray(2, 2).BackColor = FaceL.faceletsArray(0, 2).BackColor
    FaceL.faceletsArray(0, 2).BackColor = FaceL.faceletsArray(0, 0).BackColor
    FaceL.faceletsArray(0, 0).BackColor = FaceL.faceletsArray(2, 0).BackColor
    FaceL.faceletsArray(2, 0).BackColor = tempcolor

    'FaceL edges
    tempcolor = FaceL.faceletsArray(1, 0).BackColor 'UL facing left
    FaceL.faceletsArray(1, 0).BackColor = FaceL.faceletsArray(2, 1).BackColor
    FaceL.faceletsArray(2, 1).BackColor = FaceL.faceletsArray(1, 2).BackColor
    FaceL.faceletsArray(1, 2).BackColor = FaceL.faceletsArray(0, 1).BackColor
    FaceL.faceletsArray(0, 1).BackColor = tempcolor

    'UL other faces
    tempcolor = FaceF.faceletsArray(0, 0).BackColor 'FLU facing front
    FaceF.faceletsArray(0, 0).BackColor = FaceD.faceletsArray(0, 0).BackColor
    FaceD.faceletsArray(0, 0).BackColor = FaceB.faceletsArray(2, 2).BackColor
    FaceB.faceletsArray(2, 2).BackColor = FaceU.faceletsArray(0, 0).BackColor
    FaceU.faceletsArray(0, 0).BackColor = tempcolor

    'DL other faces
    tempcolor = FaceF.faceletsArray(0, 2).BackColor 'BLU facing front
    FaceF.faceletsArray(0, 2).BackColor = FaceD.faceletsArray(0, 2).BackColor
    FaceD.faceletsArray(0, 2).BackColor = FaceB.faceletsArray(2, 0).BackColor
    FaceB.faceletsArray(2, 0).BackColor = FaceU.faceletsArray(0, 2).BackColor
    FaceU.faceletsArray(0, 2).BackColor = tempcolor

    'other faces edges
    tempcolor = FaceF.faceletsArray(0, 1).BackColor 'FL facing front
    FaceF.faceletsArray(0, 1).BackColor = FaceD.faceletsArray(0, 1).BackColor
    FaceD.faceletsArray(0, 1).BackColor = FaceB.faceletsArray(2, 1).BackColor
    FaceB.faceletsArray(2, 1).BackColor = FaceU.faceletsArray(0, 1).BackColor
    FaceU.faceletsArray(0, 1).BackColor = tempcolor

End Sub
```

## Ra

```
2 references
Public Sub Ra(Cube(,,) As Cubie, FaceR As Face,
              FaceF As Face, FaceB As Face,
              FaceU As Face, FaceD As Face)

    'FaceR corners
    Dim tempcolor As Color = FaceR.faceletsArray(2, 2).BackColor 'BRD facing right
    FaceR.faceletsArray(2, 2).BackColor = FaceR.faceletsArray(0, 2).BackColor
    FaceR.faceletsArray(0, 2).BackColor = FaceR.faceletsArray(0, 0).BackColor
    FaceR.faceletsArray(0, 0).BackColor = FaceR.faceletsArray(2, 0).BackColor
    FaceR.faceletsArray(2, 0).BackColor = tempcolor

    'FaceR edges
    tempcolor = FaceR.faceletsArray(1, 0).BackColor 'RUE facing right
    FaceR.faceletsArray(1, 0).BackColor = FaceR.faceletsArray(2, 1).BackColor
    FaceR.faceletsArray(2, 1).BackColor = FaceR.faceletsArray(1, 2).BackColor
    FaceR.faceletsArray(1, 2).BackColor = FaceR.faceletsArray(0, 1).BackColor
    FaceR.faceletsArray(0, 1).BackColor = tempcolor

    'UR other faces
    tempcolor = FaceF.faceletsArray(2, 0).BackColor 'FRU facing front
    FaceF.faceletsArray(2, 0).BackColor = FaceU.faceletsArray(2, 0).BackColor
    FaceU.faceletsArray(2, 0).BackColor = FaceB.faceletsArray(0, 2).BackColor
    FaceB.faceletsArray(0, 2).BackColor = FaceD.faceletsArray(2, 0).BackColor
    FaceD.faceletsArray(2, 0).BackColor = tempcolor

    'DR other faces
    tempcolor = FaceF.faceletsArray(2, 2).BackColor
    FaceF.faceletsArray(2, 2).BackColor = FaceU.faceletsArray(2, 2).BackColor
    FaceU.faceletsArray(2, 2).BackColor = FaceB.faceletsArray(0, 0).BackColor
    FaceB.faceletsArray(0, 0).BackColor = FaceD.faceletsArray(2, 2).BackColor
    FaceD.faceletsArray(2, 2).BackColor = tempcolor

    'other faces edges
    tempcolor = FaceF.faceletsArray(2, 1).BackColor
    FaceF.faceletsArray(2, 1).BackColor = FaceU.faceletsArray(2, 1).BackColor
    FaceU.faceletsArray(2, 1).BackColor = FaceB.faceletsArray(0, 1).BackColor
    FaceB.faceletsArray(0, 1).BackColor = FaceD.faceletsArray(2, 1).BackColor
    FaceD.faceletsArray(2, 1).BackColor = tempcolor

End Sub
```

## Ua

```
2 references
Public Sub Ua(Cube(,,) As Cubie, FaceU As Face,
              FaceF As Face, FaceB As Face,
              FaceR As Face, FaceL As Face)

    'FaceU corners
    Dim tempcolor As Color = FaceU.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceU.faceletsArray(0, 0).BackColor = FaceU.faceletsArray(2, 0).BackColor
    FaceU.faceletsArray(2, 0).BackColor = FaceU.faceletsArray(2, 2).BackColor
    FaceU.faceletsArray(2, 2).BackColor = FaceU.faceletsArray(0, 2).BackColor
    FaceU.faceletsArray(0, 2).BackColor = tempcolor

    'FaceU edges
    tempcolor = FaceU.faceletsArray(1, 0).BackColor 'UB facing up
    FaceU.faceletsArray(1, 0).BackColor = FaceU.faceletsArray(2, 1).BackColor
    FaceU.faceletsArray(2, 1).BackColor = FaceU.faceletsArray(1, 2).BackColor
    FaceU.faceletsArray(1, 2).BackColor = FaceU.faceletsArray(0, 1).BackColor
    FaceU.faceletsArray(0, 1).BackColor = tempcolor

    'UL other faces
    tempcolor = FaceL.faceletsArray(0, 0).BackColor 'BLU facing left
    FaceL.faceletsArray(0, 0).BackColor = FaceB.faceletsArray(0, 0).BackColor
    FaceB.faceletsArray(0, 0).BackColor = FaceR.faceletsArray(0, 0).BackColor
    FaceR.faceletsArray(0, 0).BackColor = FaceF.faceletsArray(0, 0).BackColor
    FaceF.faceletsArray(0, 0).BackColor = tempcolor

    'UR other faces
    tempcolor = FaceL.faceletsArray(2, 0).BackColor 'FLU facing left
    FaceL.faceletsArray(2, 0).BackColor = FaceB.faceletsArray(2, 0).BackColor
    FaceB.faceletsArray(2, 0).BackColor = FaceR.faceletsArray(2, 0).BackColor
    FaceR.faceletsArray(2, 0).BackColor = FaceF.faceletsArray(2, 0).BackColor
    FaceF.faceletsArray(2, 0).BackColor = tempcolor

    'other faces edges
    tempcolor = FaceL.faceletsArray(1, 0).BackColor 'UL facing left
    FaceL.faceletsArray(1, 0).BackColor = FaceB.faceletsArray(1, 0).BackColor
    FaceB.faceletsArray(1, 0).BackColor = FaceR.faceletsArray(1, 0).BackColor
    FaceR.faceletsArray(1, 0).BackColor = FaceF.faceletsArray(1, 0).BackColor
    FaceF.faceletsArray(1, 0).BackColor = tempcolor

End Sub
```

## Da

```
2 references
Public Sub Da(Cube(,,) As Cubie, FaceD As Face,
              FaceF As Face, FaceB As Face,
              FaceR As Face, FaceL As Face)

    'FaceD corners
    Dim tempcolor As Color = FaceD.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceD.faceletsArray(0, 0).BackColor = FaceD.faceletsArray(2, 0).BackColor
    FaceD.faceletsArray(2, 0).BackColor = FaceD.faceletsArray(2, 2).BackColor
    FaceD.faceletsArray(2, 2).BackColor = FaceD.faceletsArray(0, 2).BackColor
    FaceD.faceletsArray(0, 2).BackColor = tempcolor

    'FaceD edges
    tempcolor = FaceD.faceletsArray(1, 0).BackColor 'UB facing up
    FaceD.faceletsArray(1, 0).BackColor = FaceD.faceletsArray(2, 1).BackColor
    FaceD.faceletsArray(2, 1).BackColor = FaceD.faceletsArray(1, 2).BackColor
    FaceD.faceletsArray(1, 2).BackColor = FaceD.faceletsArray(0, 1).BackColor
    FaceD.faceletsArray(0, 1).BackColor = tempcolor

    'DL other faces
    tempcolor = FaceL.faceletsArray(0, 2).BackColor 'BLU facing left
    FaceL.faceletsArray(0, 2).BackColor = FaceF.faceletsArray(0, 2).BackColor
    FaceF.faceletsArray(0, 2).BackColor = FaceR.faceletsArray(0, 2).BackColor
    FaceR.faceletsArray(0, 2).BackColor = FaceB.faceletsArray(0, 2).BackColor
    FaceB.faceletsArray(0, 2).BackColor = tempcolor

    'DR other faces
    tempcolor = FaceL.faceletsArray(2, 2).BackColor 'FLU facing left
    FaceL.faceletsArray(2, 2).BackColor = FaceF.faceletsArray(2, 2).BackColor
    FaceF.faceletsArray(2, 2).BackColor = FaceR.faceletsArray(2, 2).BackColor
    FaceR.faceletsArray(2, 2).BackColor = FaceB.faceletsArray(2, 2).BackColor
    FaceB.faceletsArray(2, 2).BackColor = tempcolor

    'other faces D edges
    tempcolor = FaceL.faceletsArray(1, 2).BackColor 'UL facing left
    FaceL.faceletsArray(1, 2).BackColor = FaceF.faceletsArray(1, 2).BackColor
    FaceF.faceletsArray(1, 2).BackColor = FaceR.faceletsArray(1, 2).BackColor
    FaceR.faceletsArray(1, 2).BackColor = FaceB.faceletsArray(1, 2).BackColor
    FaceB.faceletsArray(1, 2).BackColor = tempcolor

End Sub
```

## Fa

```
2 references
Public Sub Fa(Cube(,,) As Cubie, FaceF As Face,
              FaceU As Face, FaceD As Face,
              FaceR As Face, FaceL As Face)

    'FaceF corners
    Dim tempcolor As Color = FaceF.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceF.faceletsArray(0, 0).BackColor = FaceF.faceletsArray(2, 0).BackColor
    FaceF.faceletsArray(2, 0).BackColor = FaceF.faceletsArray(2, 2).BackColor
    FaceF.faceletsArray(2, 2).BackColor = FaceF.faceletsArray(0, 2).BackColor
    FaceF.faceletsArray(0, 2).BackColor = tempcolor

    'FaceF edges
    tempcolor = FaceF.faceletsArray(1, 0).BackColor 'UB facing up
    FaceF.faceletsArray(1, 0).BackColor = FaceF.faceletsArray(2, 1).BackColor
    FaceF.faceletsArray(2, 1).BackColor = FaceF.faceletsArray(1, 2).BackColor
    FaceF.faceletsArray(1, 2).BackColor = FaceF.faceletsArray(0, 1).BackColor
    FaceF.faceletsArray(0, 1).BackColor = tempcolor

    'DR other faces from L
    tempcolor = FaceL.faceletsArray(2, 2).BackColor 'FLD facing front
    FaceL.faceletsArray(2, 2).BackColor = FaceU.faceletsArray(0, 2).BackColor
    FaceU.faceletsArray(0, 2).BackColor = FaceR.faceletsArray(0, 0).BackColor
    FaceR.faceletsArray(0, 0).BackColor = FaceD.faceletsArray(2, 0).BackColor
    FaceD.faceletsArray(2, 0).BackColor = tempcolor

    'UR other faces from L
    tempcolor = FaceL.faceletsArray(2, 0).BackColor 'FLU facing front
    FaceL.faceletsArray(2, 0).BackColor = FaceU.faceletsArray(2, 2).BackColor
    FaceU.faceletsArray(2, 2).BackColor = FaceR.faceletsArray(0, 2).BackColor
    FaceR.faceletsArray(0, 2).BackColor = FaceD.faceletsArray(0, 0).BackColor
    FaceD.faceletsArray(0, 0).BackColor = tempcolor

    'other faces edges from L
    tempcolor = FaceL.faceletsArray(2, 1).BackColor 'FL facing front
    FaceL.faceletsArray(2, 1).BackColor = FaceU.faceletsArray(1, 2).BackColor
    FaceU.faceletsArray(1, 2).BackColor = FaceR.faceletsArray(0, 1).BackColor
    FaceR.faceletsArray(0, 1).BackColor = FaceD.faceletsArray(1, 0).BackColor
    FaceD.faceletsArray(1, 0).BackColor = tempcolor

End Sub
```

## Ba

```
2 references
Public Sub Ba(Cube(,,) As Cubie, FaceB As Face,
              FaceU As Face, FaceD As Face,
              FaceR As Face, FaceL As Face)

    'FaceB corners
    Dim tempcolor As Color = FaceB.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceB.faceletsArray(0, 0).BackColor = FaceB.faceletsArray(2, 0).BackColor
    FaceB.faceletsArray(2, 0).BackColor = FaceB.faceletsArray(2, 2).BackColor
    FaceB.faceletsArray(2, 2).BackColor = FaceB.faceletsArray(0, 2).BackColor
    FaceB.faceletsArray(0, 2).BackColor = tempcolor

    'FaceB edges
    tempcolor = FaceB.faceletsArray(1, 0).BackColor 'UB facing up
    FaceB.faceletsArray(1, 0).BackColor = FaceB.faceletsArray(2, 1).BackColor
    FaceB.faceletsArray(2, 1).BackColor = FaceB.faceletsArray(1, 2).BackColor
    FaceB.faceletsArray(1, 2).BackColor = FaceB.faceletsArray(0, 1).BackColor
    FaceB.faceletsArray(0, 1).BackColor = tempcolor

    'UL other faces from U
    tempcolor = FaceU.faceletsArray(0, 0).BackColor 'UBL facing up
    FaceU.faceletsArray(0, 0).BackColor = FaceL.faceletsArray(0, 2).BackColor
    FaceL.faceletsArray(0, 2).BackColor = FaceD.faceletsArray(2, 2).BackColor
    FaceD.faceletsArray(2, 2).BackColor = FaceR.faceletsArray(2, 0).BackColor
    FaceR.faceletsArray(2, 0).BackColor = tempcolor

    'UR other faces from U
    tempcolor = FaceU.faceletsArray(2, 0).BackColor 'FLU facing front
    FaceU.faceletsArray(2, 0).BackColor = FaceL.faceletsArray(0, 0).BackColor
    FaceL.faceletsArray(0, 0).BackColor = FaceD.faceletsArray(0, 2).BackColor
    FaceD.faceletsArray(0, 2).BackColor = FaceR.faceletsArray(2, 2).BackColor
    FaceR.faceletsArray(2, 2).BackColor = tempcolor

    'other faces edges from U
    tempcolor = FaceU.faceletsArray(1, 0).BackColor 'FL facing front
    FaceU.faceletsArray(1, 0).BackColor = FaceL.faceletsArray(0, 1).BackColor
    FaceL.faceletsArray(0, 1).BackColor = FaceD.faceletsArray(1, 2).BackColor
    FaceD.faceletsArray(1, 2).BackColor = FaceR.faceletsArray(2, 1).BackColor
    FaceR.faceletsArray(2, 1).BackColor = tempcolor

End Sub
```

## Ma

```
2 references
Public Sub Ma(Cube(,,) As Cubie,
              FaceF As Face, FaceU As Face,
              FaceB As Face, FaceD As Face)

    'centres start from F
    Dim tempcolor As Color = FaceF.faceletsArray(1, 1).BackColor
    FaceF.faceletsArray(1, 1).BackColor = FaceU.faceletsArray(1, 1).BackColor
    FaceU.faceletsArray(1, 1).BackColor = FaceB.faceletsArray(1, 1).BackColor
    FaceB.faceletsArray(1, 1).BackColor = FaceD.faceletsArray(1, 1).BackColor
    FaceD.faceletsArray(1, 1).BackColor = tempcolor

    'UM start from F
    tempcolor = FaceF.faceletsArray(1, 0).BackColor
    FaceF.faceletsArray(1, 0).BackColor = FaceU.faceletsArray(1, 0).BackColor
    FaceU.faceletsArray(1, 0).BackColor = FaceB.faceletsArray(1, 2).BackColor
    FaceB.faceletsArray(1, 2).BackColor = FaceD.faceletsArray(1, 0).BackColor
    FaceD.faceletsArray(1, 0).BackColor = tempcolor

    'DM start from F
    tempcolor = FaceF.faceletsArray(1, 2).BackColor
    FaceF.faceletsArray(1, 2).BackColor = FaceU.faceletsArray(1, 2).BackColor
    FaceU.faceletsArray(1, 2).BackColor = FaceB.faceletsArray(1, 0).BackColor
    FaceB.faceletsArray(1, 0).BackColor = FaceD.faceletsArray(1, 2).BackColor
    FaceD.faceletsArray(1, 2).BackColor = tempcolor

End Sub
```

Ea

```
2 references
Public Sub Ea(Cube(,,) As Cubie,
              FaceF As Face, FaceB As Face,
              FaceR As Face, FaceL As Face)

    'centres start from F
    Dim tempcolor As Color = FaceF.faceletsArray(1, 1).BackColor
    FaceF.faceletsArray(1, 1).BackColor = FaceL.faceletsArray(1, 1).BackColor
    FaceL.faceletsArray(1, 1).BackColor = FaceB.faceletsArray(1, 1).BackColor
    FaceB.faceletsArray(1, 1).BackColor = FaceR.faceletsArray(1, 1).BackColor
    FaceR.faceletsArray(1, 1).BackColor = tempcolor

    'L start from F
    tempcolor = FaceF.faceletsArray(0, 1).BackColor
    FaceF.faceletsArray(0, 1).BackColor = FaceL.faceletsArray(0, 1).BackColor
    FaceL.faceletsArray(0, 1).BackColor = FaceB.faceletsArray(0, 1).BackColor
    FaceB.faceletsArray(0, 1).BackColor = FaceR.faceletsArray(0, 1).BackColor
    FaceR.faceletsArray(0, 1).BackColor = tempcolor

    'R start from F
    tempcolor = FaceF.faceletsArray(2, 1).BackColor
    FaceF.faceletsArray(2, 1).BackColor = FaceL.faceletsArray(2, 1).BackColor
    FaceL.faceletsArray(2, 1).BackColor = FaceB.faceletsArray(2, 1).BackColor
    FaceB.faceletsArray(2, 1).BackColor = FaceR.faceletsArray(2, 1).BackColor
    FaceR.faceletsArray(2, 1).BackColor = tempcolor

End Sub
```

Sa

```
2 references
Public Sub Sa(Cube(,,) As Cubie,
              FaceU As Face, FaceD As Face,
              FaceR As Face, FaceL As Face)

    'centres start from U
    Dim tempcolor As Color = FaceU.faceletsArray(1, 1).BackColor
    FaceU.faceletsArray(1, 1).BackColor = FaceR.faceletsArray(1, 1).BackColor
    FaceR.faceletsArray(1, 1).BackColor = FaceD.faceletsArray(1, 1).BackColor
    FaceD.faceletsArray(1, 1).BackColor = FaceL.faceletsArray(1, 1).BackColor
    FaceL.faceletsArray(1, 1).BackColor = tempcolor

    'L start from U
    tempcolor = FaceU.faceletsArray(0, 1).BackColor
    FaceU.faceletsArray(0, 1).BackColor = FaceR.faceletsArray(1, 0).BackColor
    FaceR.faceletsArray(1, 0).BackColor = FaceD.faceletsArray(2, 1).BackColor
    FaceD.faceletsArray(2, 1).BackColor = FaceL.faceletsArray(1, 2).BackColor
    FaceL.faceletsArray(1, 2).BackColor = tempcolor

    'R start from U
    tempcolor = FaceU.faceletsArray(2, 1).BackColor
    FaceU.faceletsArray(2, 1).BackColor = FaceR.faceletsArray(1, 2).BackColor
    FaceR.faceletsArray(1, 2).BackColor = FaceD.faceletsArray(0, 1).BackColor
    FaceD.faceletsArray(0, 1).BackColor = FaceL.faceletsArray(1, 0).BackColor
    FaceL.faceletsArray(1, 0).BackColor = tempcolor

End Sub
```

X

```
2 references
Public Sub x(Cube(,,) As Cubie,
             FaceR As Face, FaceL As Face,
             FaceF As Face, FaceB As Face,
             FaceU As Face, FaceD As Face)

    Dim tempcolor As Color
    Dim row As Integer
    Dim column As Integer

    For i = 0 To 8
        row = i Mod 3
        column = Math.Floor(i / 3)

        tempcolor = FaceF.faceletsArray(column, row).BackColor
        FaceF.faceletsArray(column, row).BackColor = FaceD.faceletsArray(column, row).BackColor
        FaceD.faceletsArray(column, row).BackColor = FaceB.faceletsArray(2 - column, 2 - row).BackColor
        FaceB.faceletsArray(2 - column, 2 - row).BackColor = FaceU.faceletsArray(column, row).BackColor
        FaceU.faceletsArray(column, row).BackColor = tempcolor
    Next

    'La and R

    'FaceL corners
    tempcolor = FaceL.faceletsArray(2, 2).BackColor 'FLD facing left
    FaceL.faceletsArray(2, 2).BackColor = FaceL.faceletsArray(0, 2).BackColor
    FaceL.faceletsArray(0, 2).BackColor = FaceL.faceletsArray(0, 0).BackColor
    FaceL.faceletsArray(0, 0).BackColor = FaceL.faceletsArray(2, 0).BackColor
    FaceL.faceletsArray(2, 0).BackColor = tempcolor
    'FaceL edges
    tempcolor = FaceL.faceletsArray(1, 0).BackColor 'UL facing left
    FaceL.faceletsArray(1, 0).BackColor = FaceL.faceletsArray(2, 1).BackColor
    FaceL.faceletsArray(2, 1).BackColor = FaceL.faceletsArray(1, 2).BackColor
    FaceL.faceletsArray(1, 2).BackColor = FaceL.faceletsArray(0, 1).BackColor
    FaceL.faceletsArray(0, 1).BackColor = tempcolor


    'FaceR corners
    tempcolor = FaceR.faceletsArray(2, 2).BackColor 'BRD facing right
    FaceR.faceletsArray(2, 2).BackColor = FaceR.faceletsArray(2, 0).BackColor
    FaceR.faceletsArray(2, 0).BackColor = FaceR.faceletsArray(0, 0).BackColor
    FaceR.faceletsArray(0, 0).BackColor = FaceR.faceletsArray(0, 2).BackColor
    FaceR.faceletsArray(0, 2).BackColor = tempcolor
    'FaceR edges
    tempcolor = FaceR.faceletsArray(1, 0).BackColor 'RUE facing right
    FaceR.faceletsArray(1, 0).BackColor = FaceR.faceletsArray(0, 1).BackColor
    FaceR.faceletsArray(0, 1).BackColor = FaceR.faceletsArray(1, 2).BackColor
    FaceR.faceletsArray(1, 2).BackColor = FaceR.faceletsArray(2, 1).BackColor
    FaceR.faceletsArray(2, 1).BackColor = tempcolor

End Sub
```

y

```
2 references
Public Sub y(Cube(,,) As Cubie,
             FaceR As Face, FaceL As Face,
             FaceF As Face, FaceB As Face,
             FaceU As Face, FaceD As Face)

    Dim tempcolor As Color
    Dim row As Integer
    Dim column As Integer

    For i = 0 To 8
        row = i Mod 3
        column = Math.Floor(i / 3)

        tempcolor = FaceF.faceletsArray(column, row).BackColor
        FaceF.faceletsArray(column, row).BackColor = FaceR.faceletsArray(column, row).BackColor
        FaceR.faceletsArray(column, row).BackColor = FaceB.faceletsArray(column, row).BackColor
        FaceB.faceletsArray(column, row).BackColor = FaceL.faceletsArray(column, row).BackColor
        FaceL.faceletsArray(column, row).BackColor = tempcolor
    Next

    'U and Da

    'FaceU corners
    tempcolor = FaceU.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceU.faceletsArray(0, 0).BackColor = FaceU.faceletsArray(0, 2).BackColor
    FaceU.faceletsArray(0, 2).BackColor = FaceU.faceletsArray(2, 2).BackColor
    FaceU.faceletsArray(2, 2).BackColor = FaceU.faceletsArray(2, 0).BackColor
    FaceU.faceletsArray(2, 0).BackColor = tempcolor
    'FaceU edges
    tempcolor = FaceU.faceletsArray(1, 0).BackColor 'UB facing up
    FaceU.faceletsArray(1, 0).BackColor = FaceU.faceletsArray(0, 1).BackColor
    FaceU.faceletsArray(0, 1).BackColor = FaceU.faceletsArray(1, 2).BackColor
    FaceU.faceletsArray(1, 2).BackColor = FaceU.faceletsArray(2, 1).BackColor
    FaceU.faceletsArray(2, 1).BackColor = tempcolor

    'FaceD corners
    tempcolor = FaceD.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceD.faceletsArray(0, 0).BackColor = FaceD.faceletsArray(2, 0).BackColor
    FaceD.faceletsArray(2, 0).BackColor = FaceD.faceletsArray(2, 2).BackColor
    FaceD.faceletsArray(2, 2).BackColor = FaceD.faceletsArray(0, 2).BackColor
    FaceD.faceletsArray(0, 2).BackColor = tempcolor
    'FaceD edges
    tempcolor = FaceD.faceletsArray(1, 0).BackColor 'UB facing up
    FaceD.faceletsArray(1, 0).BackColor = FaceD.faceletsArray(2, 1).BackColor
    FaceD.faceletsArray(2, 1).BackColor = FaceD.faceletsArray(1, 2).BackColor
    FaceD.faceletsArray(1, 2).BackColor = FaceD.faceletsArray(0, 1).BackColor
    FaceD.faceletsArray(0, 1).BackColor = tempcolor

End Sub
```

Z

```
2 references
Public Sub z(Cube(,,) As Cubie,
             FaceR As Face, FaceL As Face,
             FaceF As Face, FaceB As Face,
             FaceU As Face, FaceD As Face)

    Dim tempcolor As Color
    Dim row As Integer
    Dim column As Integer

    For i = 0 To 8
        row = i Mod 3
        column = Math.Floor(i / 3)

        'from L
        tempcolor = FaceL.faceletsArray(column, row).BackColor
        FaceL.faceletsArray(column, row).BackColor = FaceD.faceletsArray(row, 2 - column).BackColor
        FaceD.faceletsArray(row, 2 - column).BackColor = FaceR.faceletsArray(2 - column, 2 - row).BackColor
        FaceR.faceletsArray(2 - column, 2 - row).BackColor = FaceU.faceletsArray(row, 2 - column).BackColor
        FaceU.faceletsArray(row, 2 - column).BackColor = tempcolor
    Next

    'F and Ba

    'FaceF corners
    tempcolor = FaceF.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceF.faceletsArray(0, 0).BackColor = FaceF.faceletsArray(0, 2).BackColor
    FaceF.faceletsArray(0, 2).BackColor = FaceF.faceletsArray(2, 2).BackColor
    FaceF.faceletsArray(2, 2).BackColor = FaceF.faceletsArray(2, 0).BackColor
    FaceF.faceletsArray(2, 0).BackColor = tempcolor
    'FaceF edges
    tempcolor = FaceF.faceletsArray(1, 0).BackColor 'UB facing up
    FaceF.faceletsArray(1, 0).BackColor = FaceF.faceletsArray(0, 1).BackColor
    FaceF.faceletsArray(0, 1).BackColor = FaceF.faceletsArray(1, 2).BackColor
    FaceF.faceletsArray(1, 2).BackColor = FaceF.faceletsArray(2, 1).BackColor
    FaceF.faceletsArray(2, 1).BackColor = tempcolor

    'FaceB corners
    tempcolor = FaceB.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceB.faceletsArray(0, 0).BackColor = FaceB.faceletsArray(2, 0).BackColor
    FaceB.faceletsArray(2, 0).BackColor = FaceB.faceletsArray(2, 2).BackColor
    FaceB.faceletsArray(2, 2).BackColor = FaceB.faceletsArray(0, 2).BackColor
    FaceB.faceletsArray(0, 2).BackColor = tempcolor
    'FaceB edges
    tempcolor = FaceB.faceletsArray(1, 0).BackColor 'UB facing up
    FaceB.faceletsArray(1, 0).BackColor = FaceB.faceletsArray(2, 1).BackColor
    FaceB.faceletsArray(2, 1).BackColor = FaceB.faceletsArray(1, 2).BackColor
    FaceB.faceletsArray(1, 2).BackColor = FaceB.faceletsArray(0, 1).BackColor
    FaceB.faceletsArray(0, 1).BackColor = tempcolor

End Sub
```

xa

```
2 references
Public Sub xa(Cube(,,) As Cubie,
              FaceR As Face, FaceL As Face,
              FaceF As Face, FaceB As Face,
              FaceU As Face, FaceD As Face)

    Dim tempcolor As Color
    Dim row As Integer
    Dim column As Integer

    For i = 0 To 8
        row = i Mod 3
        column = Math.Floor(i / 3)

        tempcolor = FaceF.faceletsArray(column, row).BackColor
        FaceF.faceletsArray(column, row).BackColor = FaceU.faceletsArray(column, row).BackColor
        FaceU.faceletsArray(column, row).BackColor = FaceB.faceletsArray(2 - column, 2 - row).BackColor
        FaceB.faceletsArray(2 - column, 2 - row).BackColor = FaceD.faceletsArray(column, row).BackColor
        FaceD.faceletsArray(column, row).BackColor = tempcolor
    Next

    'L and Ra

    'FaceL corners
    tempcolor = FaceL.faceletsArray(2, 2).BackColor 'FLD facing left
    FaceL.faceletsArray(2, 2).BackColor = FaceL.faceletsArray(2, 0).BackColor
    FaceL.faceletsArray(2, 0).BackColor = FaceL.faceletsArray(0, 0).BackColor
    FaceL.faceletsArray(0, 0).BackColor = FaceL.faceletsArray(0, 2).BackColor
    FaceL.faceletsArray(0, 2).BackColor = tempcolor
    'FaceL edges
    tempcolor = FaceL.faceletsArray(1, 0).BackColor 'UL facing left
    FaceL.faceletsArray(1, 0).BackColor = FaceL.faceletsArray(0, 1).BackColor
    FaceL.faceletsArray(0, 1).BackColor = FaceL.faceletsArray(1, 2).BackColor
    FaceL.faceletsArray(1, 2).BackColor = FaceL.faceletsArray(2, 1).BackColor
    FaceL.faceletsArray(2, 1).BackColor = tempcolor

    'FaceR corners
    tempcolor = FaceR.faceletsArray(2, 2).BackColor 'BRD facing right
    FaceR.faceletsArray(2, 2).BackColor = FaceR.faceletsArray(0, 2).BackColor
    FaceR.faceletsArray(0, 2).BackColor = FaceR.faceletsArray(0, 0).BackColor
    FaceR.faceletsArray(0, 0).BackColor = FaceR.faceletsArray(2, 0).BackColor
    FaceR.faceletsArray(2, 0).BackColor = tempcolor
    'FaceR edges
    tempcolor = FaceR.faceletsArray(1, 0).BackColor 'RUE facing right
    FaceR.faceletsArray(1, 0).BackColor = FaceR.faceletsArray(2, 1).BackColor
    FaceR.faceletsArray(2, 1).BackColor = FaceR.faceletsArray(1, 2).BackColor
    FaceR.faceletsArray(1, 2).BackColor = FaceR.faceletsArray(0, 1).BackColor
    FaceR.faceletsArray(0, 1).BackColor = tempcolor

End Sub
```

ya

```
2 references
Public Sub ya(Cube(,,) As Cubie,
              FaceR As Face, FaceL As Face,
              FaceF As Face, FaceB As Face,
              FaceU As Face, FaceD As Face)

    Dim tempcolor As Color
    Dim row As Integer
    Dim column As Integer

    For i = 0 To 8
        row = i Mod 3
        column = Math.Floor(i / 3)

        tempcolor = FaceF.faceletsArray(column, row).BackColor
        FaceF.faceletsArray(column, row).BackColor = FaceL.faceletsArray(column, row).BackColor
        FaceL.faceletsArray(column, row).BackColor = FaceB.faceletsArray(column, row).BackColor
        FaceB.faceletsArray(column, row).BackColor = FaceR.faceletsArray(column, row).BackColor
        FaceR.faceletsArray(column, row).BackColor = tempcolor
    Next

    'Ua and D

    'FaceU corners
    tempcolor = FaceU.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceU.faceletsArray(0, 0).BackColor = FaceU.faceletsArray(2, 0).BackColor
    FaceU.faceletsArray(2, 0).BackColor = FaceU.faceletsArray(2, 2).BackColor
    FaceU.faceletsArray(2, 2).BackColor = FaceU.faceletsArray(0, 2).BackColor
    FaceU.faceletsArray(0, 2).BackColor = tempcolor
    'FaceU edges
    tempcolor = FaceU.faceletsArray(1, 0).BackColor 'UB facing up
    FaceU.faceletsArray(1, 0).BackColor = FaceU.faceletsArray(2, 1).BackColor
    FaceU.faceletsArray(2, 1).BackColor = FaceU.faceletsArray(1, 2).BackColor
    FaceU.faceletsArray(1, 2).BackColor = FaceU.faceletsArray(0, 1).BackColor
    FaceU.faceletsArray(0, 1).BackColor = tempcolor

    'FaceD corners
    tempcolor = FaceD.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceD.faceletsArray(0, 0).BackColor = FaceD.faceletsArray(0, 2).BackColor
    FaceD.faceletsArray(0, 2).BackColor = FaceD.faceletsArray(2, 2).BackColor
    FaceD.faceletsArray(2, 2).BackColor = FaceD.faceletsArray(2, 0).BackColor
    FaceD.faceletsArray(2, 0).BackColor = tempcolor
    'FaceD edges
    tempcolor = FaceD.faceletsArray(1, 0).BackColor 'UB facing up
    FaceD.faceletsArray(1, 0).BackColor = FaceD.faceletsArray(0, 1).BackColor
    FaceD.faceletsArray(0, 1).BackColor = FaceD.faceletsArray(1, 2).BackColor
    FaceD.faceletsArray(1, 2).BackColor = FaceD.faceletsArray(2, 1).BackColor
    FaceD.faceletsArray(2, 1).BackColor = tempcolor

End Sub
```

za

```
2 references
Public Sub za(Cube(,,) As Cubie,
              FaceR As Face, FaceL As Face,
              FaceF As Face, FaceB As Face,
              FaceU As Face, FaceD As Face)

    Dim tempcolor As Color
    Dim row As Integer
    Dim column As Integer

    For i = 0 To 8
        row = i Mod 3
        column = Math.Floor(i / 3)

        'from L
        tempcolor = FaceL.faceletsArray(column, row).BackColor
        FaceL.faceletsArray(column, row).BackColor = FaceU.faceletsArray(row, 2 - column).BackColor
        FaceU.faceletsArray(row, 2 - column).BackColor = FaceR.faceletsArray(2 - column, 2 - row).BackColor
        FaceR.faceletsArray(2 - column, 2 - row).BackColor = FaceD.faceletsArray(row, 2 - column).BackColor
        FaceD.faceletsArray(row, 2 - column).BackColor = tempcolor
    Next

    'Fa and B

    'FaceF corners
    tempcolor = FaceF.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceF.faceletsArray(0, 0).BackColor = FaceF.faceletsArray(2, 0).BackColor
    FaceF.faceletsArray(2, 0).BackColor = FaceF.faceletsArray(2, 2).BackColor
    FaceF.faceletsArray(2, 2).BackColor = FaceF.faceletsArray(0, 2).BackColor
    FaceF.faceletsArray(0, 2).BackColor = tempcolor
    'FaceF edges
    tempcolor = FaceF.faceletsArray(1, 0).BackColor 'UB facing up
    FaceF.faceletsArray(1, 0).BackColor = FaceF.faceletsArray(2, 1).BackColor
    FaceF.faceletsArray(2, 1).BackColor = FaceF.faceletsArray(1, 2).BackColor
    FaceF.faceletsArray(1, 2).BackColor = FaceF.faceletsArray(0, 1).BackColor
    FaceF.faceletsArray(0, 1).BackColor = tempcolor

    'FaceB corners
    tempcolor = FaceB.faceletsArray(0, 0).BackColor 'ULB facing up
    FaceB.faceletsArray(0, 0).BackColor = FaceB.faceletsArray(0, 2).BackColor
    FaceB.faceletsArray(0, 2).BackColor = FaceB.faceletsArray(2, 2).BackColor
    FaceB.faceletsArray(2, 2).BackColor = FaceB.faceletsArray(2, 0).BackColor
    FaceB.faceletsArray(2, 0).BackColor = tempcolor
    'FaceB edges
    tempcolor = FaceB.faceletsArray(1, 0).BackColor 'UB facing up
    FaceB.faceletsArray(1, 0).BackColor = FaceB.faceletsArray(0, 1).BackColor
    FaceB.faceletsArray(0, 1).BackColor = FaceB.faceletsArray(1, 2).BackColor
    FaceB.faceletsArray(1, 2).BackColor = FaceB.faceletsArray(2, 1).BackColor
    FaceB.faceletsArray(2, 1).BackColor = tempcolor

End Sub
```

## CheckSolvable

```
1 reference
1032 Public Function CheckSolvable(Cube(,,) As Cubie,
1033                 FaceR As Face, FaceL As Face,
1034                 FaceF As Face, FaceB As Face,
1035                 FaceU As Face, FaceD As Face) As String
1036     Dim myarray() As Integer = {0, 0, 0, 0, 0, 0}
1037     Dim returnstring As String = ""
1038     Dim booleanarray() As Boolean = {False, False, False, False, False, False}
1039     Dim i As Integer
1040
1041     Dim solved As Boolean = Form1.CheckSolved("")
1042     If solved = True Then
1043         Return "The cube is already solved."
1044     End If
1045
1046     If FaceU.faceletsArray(1, 1).BackColor <> Color.Yellow Then
1047         booleanarray(0) = True
1048     ElseIf FaceD.faceletsArray(1, 1).BackColor <> Color.white Then
1049         booleanarray(1) = True
1050     ElseIf Facef.faceletsArray(1, 1).BackColor <> Color.blue Then
1051         booleanarray(2) = True
1052     ElseIf Faceb.faceletsArray(1, 1).BackColor <> Color.green Then
1053         booleanarray(3) = True
1054     ElseIf Facer.faceletsArray(1, 1).BackColor <> Color.red Then
1055         booleanarray(4) = True
1056     ElseIf Facel.faceletsArray(1, 1).BackColor <> Color.Orange Then
1057         booleanarray(5) = True
1058     End If
1059     Dim iWrongCentres As Integer = 0
1060     For i = 0 To 5
1061         If booleanarray(i) = True Then
1062             iWrongCentres = iWrongCentres + 1
1063         End If
1064     Next
1065     i = 0
1066     If iWrongCentres <> 0 Then
1067         returnstring = "The "
1068         While iWrongCentres > 0
1069
1070             If booleanarray(i) = True Then
1071                 Select Case i
1072                     Case 0
1073                         returnstring = returnstring + "yellow"
1074                     Case 1
1075                         returnstring = returnstring + "white"
1076                     Case 2
1077                         returnstring = returnstring + "blue"
1078                     Case 3
1079                         returnstring = returnstring + "green"
1080                     Case 4
1081                         returnstring = returnstring + "red"
1082                     Case 5
1083                         returnstring = returnstring + "orange"
1084                 End Select
1085                 If iWrongCentres > 1 Then
1086                     returnstring = returnstring + ", "
1087                 End If
1088             End If
1089
1090             iWrongCentres = iWrongCentres - 1
1091             i = i + 1
1092         End While
1093         returnstring = returnstring + " centres are in the wrong place."
1094         Return returnstring
1095     End If
1096
1097     myarray = checkface(FaceF, myarray)
1098     myarray = checkface(FaceB, myarray)
1099     myarray = checkface(FaceR, myarray)
1100     myarray = checkface(FaceL, myarray)
1101     myarray = checkface(FaceU, myarray)
1102     myarray = checkface(FaceD, myarray)
1103     For i = 0 To 5
1104         If myarray(i) <> 9 Then
1105             If returnstring <> "" Then
1106                 returnstring = "The number of facelets of each colour does not match.
1107 "
1108             End If
1109             Select Case i
1110                 Case 0
1111                     returnstring = returnstring + "Yellow:" + Str(myarray(0)) + " "
1112                 Case 1
1113                     returnstring = returnstring + "White:" + Str(myarray(1)) + " "
1114                 Case 2
1115                     returnstring = returnstring + "Blue:" + Str(myarray(2)) + " "
1116                 Case 3
1117                     returnstring = returnstring + "Green:" + Str(myarray(3)) + " "
1118                 Case 4
1119                     returnstring = returnstring + "Red:" + Str(myarray(4)) + " "
1120                 Case 5
1121                     returnstring = returnstring + "Orange:" + Str(myarray(5)) + " "
1122             End Select
1123
1124             Return returnstring
1125         End If
1126     Next
1127
1128     Return ""
1129 End Function
```

The returns on lines 1043 and 1094 go back to the previous subroutine so that time is not wasted in the next steps of checking if the cube is solvable.

## checkface

```
6 references
Private Function checkface(ByVal myface As Face, ByRef myarray() As Integer)
    Dim i As Integer
    Dim j As Integer

    For faceletnumber = 0 To 8
        i = Math.Floor(faceletnumber / 3)
        j = faceletnumber Mod 3

        Select Case myface.faceletsArray(i, j).BackColor
            Case Color.Yellow
                myarray(0) = myarray(0) + 1
            Case Color.White
                myarray(1) = myarray(1) + 1
            Case Color.Blue
                myarray(2) = myarray(2) + 1
            Case Color.Green
                myarray(3) = myarray(3) + 1
            Case Color.Red
                myarray(4) = myarray(4) + 1
            Case Color.Orange
                myarray(5) = myarray(5) + 1
        End Select
    Next
    Return myarray
End Function
```

## SOLVE

```
1 reference
Public Sub SOLVE(Cube(,,) As Cubie,
                 FaceR As Face, FaceL As Face,
                 FaceF As Face, FaceB As Face,
                 FaceU As Face, FaceD As Face)
    MsgBox("Solve algorithm")
End Sub

End Module
```

## PaletteChanges

```
Module PaletteChanges
    Public currentpalettecolour As Color
```

## PaletteY_click

```
    1 reference
    Public Sub PaletteY_click(ByRef faceletface As String, ByRef paletteselected As Boolean,
                              ByRef FaceY As Face, ByRef FaceW As Face,
                              ByRef FaceB As Face, ByRef FaceG As Face,
                              ByRef FaceR As Face, ByRef FaceO As Face)
        Dim i As Integer
        Dim j As Integer
        Dim faceletcolour As Char
        currentpalettecolour = Color.Yellow
        If faceletface = "" Then
            paletteselected = True
        Else
            faceletcolour = Mid(faceletface, 1, 1)
            i = Int(Mid(faceletface, 2, 1))
            j = Int(Mid(faceletface, 3, 1))
            Select Case faceletcolour
                Case "Y"
                    FaceY.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "W"
                    FaceW.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "B"
                    FaceB.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "G"
                    FaceG.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "R"
                    FaceR.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "O"
                    FaceO.faceletsArray(i, j).BackColor = currentpalettecolour
            End Select
        End If
    End Sub
```

## PaletteW_click

```
    1 reference
    Public Sub PaletteW_click(ByVal faceletface As String, ByRef paletteselected As Boolean,
                              ByRef FaceY As Face, ByRef FaceW As Face,
                              ByRef FaceB As Face, ByRef FaceG As Face,
                              ByRef FaceR As Face, ByRef FaceO As Face)
        Dim i As Integer
        Dim j As Integer
        Dim faceletcolour As Char
        currentpalettecolour = Color.White
        If faceletface = "" Then
            paletteselected = True
        Else
            faceletcolour = Mid(faceletface, 1, 1)
            i = Int(Mid(faceletface, 2, 1))
            j = Int(Mid(faceletface, 3, 1))
            Select Case faceletface
                Case "Y"
                    FaceY.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "W"
                    FaceW.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "B"
                    FaceB.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "G"
                    FaceG.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "R"
                    FaceR.faceletsArray(i, j).BackColor = currentpalettecolour
                Case "O"
                    FaceO.faceletsArray(i, j).BackColor = currentpalettecolour
            End Select
        End If
    End Sub
```

PaletteB_click

```
1 reference
Public Sub PaletteB_click(ByVal faceletface As String, ByRef paletteselected As Boolean,
                          ByRef FaceY As Face, ByRef FaceW As Face,
                          ByRef FaceB As Face, ByRef FaceG As Face,
                          ByRef FaceR As Face, ByRef FaceO As Face)
    Dim i As Integer
    Dim j As Integer
    Dim faceletcolour As Char
    currentpalettecolour = Color.Blue
    If faceletface = "" Then
        paletteselected = True
    Else
        faceletcolour = Mid(faceletface, 1, 1)
        i = Int(Mid(faceletface, 2, 1))
        j = Int(Mid(faceletface, 3, 1))
        Select Case faceletface
            Case "Y"
                FaceY.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "W"
                FaceW.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "B"
                FaceB.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "G"
                FaceG.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "R"
                FaceR.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "O"
                FaceO.faceletsArray(i, j).BackColor = currentpalettecolour
        End Select
    End If
End Sub
```

PaletteG_click

```
1 reference
Public Sub PaletteG_click(ByVal faceletface As String, ByRef paletteselected As Boolean,
                          ByRef FaceY As Face, ByRef FaceW As Face,
                          ByRef FaceB As Face, ByRef FaceG As Face,
                          ByRef FaceR As Face, ByRef FaceO As Face)
    Dim i As Integer
    Dim j As Integer
    Dim faceletcolour As Char
    currentpalettecolour = Color.Green
    If faceletface = "" Then
        paletteselected = True
    Else
        faceletcolour = Mid(faceletface, 1, 1)
        i = Int(Mid(faceletface, 2, 1))
        j = Int(Mid(faceletface, 3, 1))
        Select Case faceletface
            Case "Y"
                FaceY.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "W"
                FaceW.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "B"
                FaceB.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "G"
                FaceG.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "R"
                FaceR.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "O"
                FaceO.faceletsArray(i, j).BackColor = currentpalettecolour
        End Select
    End If
End Sub
```

## PaletteR_click

```
1 reference
Public Sub PaletteR_click(ByVal faceletface As String, ByRef paletteselected As Boolean,
                          ByRef FaceY As Face, ByRef FaceW As Face,
                          ByRef FaceB As Face, ByRef FaceG As Face,
                          ByRef FaceR As Face, ByRef FaceO As Face)
    Dim i As Integer
    Dim j As Integer
    Dim faceletcolour As Char
    currentpalettecolour = Color.Red
    If faceletface = "" Then
        paletteselected = True
    Else
        faceletcolour = Mid(faceletface, 1, 1)
        i = Int(Mid(faceletface, 2, 1))
        j = Int(Mid(faceletface, 3, 1))
        Select Case faceletface
            Case "Y"
                FaceY.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "W"
                FaceW.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "B"
                FaceB.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "G"
                FaceG.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "R"
                FaceR.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "O"
                FaceO.faceletsArray(i, j).BackColor = currentpalettecolour
        End Select
    End If
End Sub
```

## PaletteO_click

```
1 reference
Public Sub PaletteO_click(ByVal faceletface As String, ByRef paletteselected As Boolean,
                          ByRef FaceY As Face, ByRef FaceW As Face,
                          ByRef FaceB As Face, ByRef FaceG As Face,
                          ByRef FaceR As Face, ByRef FaceO As Face)
    Dim i As Integer
    Dim j As Integer
    Dim faceletcolour As Char
    currentpalettecolour = Color.Orange
    If faceletface = "" Then
        paletteselected = True
    Else
        faceletcolour = Mid(faceletface, 1, 1)
        i = Int(Mid(faceletface, 2, 1))
        j = Int(Mid(faceletface, 3, 1))
        Select Case faceletface
            Case "Y"
                FaceY.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "W"
                FaceW.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "B"
                FaceB.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "G"
                FaceG.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "R"
                FaceR.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "O"
                FaceO.faceletsArray(i, j).BackColor = currentpalettecolour
        End Select
    End If
End Sub
```

## faceletclicked

```
0 references
Public Sub faceletclicked(faceinitial As Char, i As Integer, j As Integer)
    If Form1.paletteselected = True Then
        Select Case faceinitial
            Case "W"
                Form1.FaceW.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "Y"
                Form1.FaceY.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "B"
                Form1.FaceB.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "G"
                Form1.FaceG.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "R"
                Form1.FaceR.faceletsArray(i, j).BackColor = currentpalettecolour
            Case "O"
                Form1.FaceO.faceletsArray(i, j).BackColor = currentpalettecolour
        End Select
    Else
        Form1.faceletface = faceinitial + Str(i) + Str(j)
    End If
End Sub
End Module
```

## Form1

Throughout the subroutines in Form1, they contribute to sLogString, which is displayed when the user clicks on the "Log2 button.

```
12 references
1  Public Class Form1
2      Public Cube(2, 2, 2) As Cubie
3      Public FaceW As New Face
4      Public FaceY As New Face
5      Public FaceB As New Face
6      Public FaceG As New Face
7      Public FaceR As New Face
8      Public FaceO As New Face
9      Dim sLogString As String = "Log:
10
11 "
12     Public paletteselected As Boolean = False
13     Public faceletface As String = ""
14     Dim movessequence As String
```

On line 2, the Cube is instantiated at run-time and makes use of OOP composition (Group A).

## Scramble_Click

```
0 references
16  Private Sub Scramble_Click(sender As Object, e As EventArgs) Handles btnScramble.Click
17      Dim ScrambleMovesNumber As Integer = udScramble.Value
18
19      Dim value As Integer
20      Dim cubesolved As Boolean = True
21
22      Do While cubesolved = True
23          For i = 1 To ScrambleMovesNumber
24              Randomize()
25              value = CInt(Int((33 * Rnd()) + 1))
26              Select Case value
27                  Case 1
28                      Moves.x(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
29                  Case 2
30                      Moves.y(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
31                  Case 3
32                      Moves.z(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
33                  Case 4
34                      Moves.xa(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
35                  Case 5
36                      Moves.ya(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
37                  Case 6
38                      Moves.za(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
39                  Case 7
40                      Moves.R(Cube, FaceR, FaceB, FaceG, FaceY, FaceW)
41                  Case 8
42                      Moves.L(Cube, FaceO, FaceB, FaceG, FaceY, FaceW)
43                  Case 9
44                      Moves.U(Cube, FaceY, FaceB, FaceG, FaceR, FaceO)
45                  Case 10
46                      Moves.D(Cube, FaceW, FaceB, FaceG, FaceR, FaceO)
47                  Case 11
48                      Moves.F(Cube, FaceB, FaceY, FaceW, FaceR, FaceO)
49                  Case 12
50                      Moves.B(Cube, FaceG, FaceY, FaceW, FaceR, FaceO)
51                  Case 13
52                      Moves.M(Cube, FaceB, FaceY, FaceG, FaceW)
53                  Case 14
54                      Moves.E(Cube, FaceB, FaceG, FaceR, FaceO)
55                  Case 15
56                      Moves.S(Cube, FaceY, FaceW, FaceR, FaceO)
57                  Case 16
58                      Moves.Ra(Cube, FaceR, FaceB, FaceG, FaceY, FaceW)
59                  Case 17
60                      Moves.La(Cube, FaceO, FaceB, FaceG, FaceY, FaceW)
61                  Case 18
62                      Moves.Ua(Cube, FaceY, FaceB, FaceG, FaceR, FaceO)
63                  Case 19
```

Line 17 uses OOP to get the integer value in the up-down control.

```
64                          Moves.Da(Cube, FaceW, FaceB, FaceG, FaceR, FaceO)
65                      Case 20
66                          Moves.Fa(Cube, FaceB, FaceY, FaceW, FaceR, FaceO)
67                      Case 21
68                          Moves.Ba(Cube, FaceG, FaceY, FaceW, FaceR, FaceO)
69                      Case 22
70                          Moves.Ma(Cube, FaceB, FaceY, FaceG, FaceW)
71                      Case 23
72                          Moves.Ea(Cube, FaceB, FaceG, FaceR, FaceO)
73                      Case 24
74                          Moves.Sa(Cube, FaceY, FaceW, FaceR, FaceO)
75                      Case 25
76                          Moves.R(Cube, FaceR, FaceB, FaceG, FaceY, FaceW)
77                          Moves.R(Cube, FaceR, FaceB, FaceG, FaceY, FaceW)
78                      Case 26
79                          Moves.L(Cube, FaceO, FaceB, FaceG, FaceY, FaceW)
80                          Moves.L(Cube, FaceO, FaceB, FaceG, FaceY, FaceW)
81                      Case 27
82                          Moves.U(Cube, FaceY, FaceB, FaceG, FaceR, FaceO)
83                          Moves.U(Cube, FaceY, FaceB, FaceG, FaceR, FaceO)
84                      Case 28
85                          Moves.D(Cube, FaceW, FaceB, FaceG, FaceR, FaceO)
86                          Moves.D(Cube, FaceW, FaceB, FaceG, FaceR, FaceO)
87                      Case 29
88                          Moves.F(Cube, FaceB, FaceY, FaceW, FaceR, FaceO)
89                          Moves.F(Cube, FaceB, FaceY, FaceW, FaceR, FaceO)
90                      Case 30
91                          Moves.B(Cube, FaceG, FaceY, FaceW, FaceR, FaceO)
92                          Moves.B(Cube, FaceG, FaceY, FaceW, FaceR, FaceO)
93                      Case 31
94                          Moves.M(Cube, FaceB, FaceY, FaceG, FaceW)
95                          Moves.M(Cube, FaceB, FaceY, FaceG, FaceW)
96                      Case 32
97                          Moves.E(Cube, FaceB, FaceG, FaceR, FaceO)
98                          Moves.E(Cube, FaceB, FaceG, FaceR, FaceO)
99                      Case 33
100                         Moves.S(Cube, FaceY, FaceW, FaceR, FaceO)
101                         Moves.S(Cube, FaceY, FaceW, FaceR, FaceO)
102                 End Select
103                 cubesolved = CheckSolved("")
104             Next
105         Loop
106
107         If movessequence <> "" Then
108             sLogString = sLogString + "
109 "
110         End If
111         movessequence = ""
112         sLogString = sLogString + "Cube scrambled using " + Trim(Str(ScrambleMovesNumber)) + " moves.
113 "
114     End Sub
```

## Form1_Load

```
0 references
Private Sub Form1_Load(sender As Object, e As EventArgs) Handles MyBase.Load

    Me.Width = 800
    Me.Height = 500

    ArrangeButtons()

    udScramble.Maximum = 30
    udScramble.Left = 630
    udScramble.Top = 290
    btnScramble.Left = 670
    btnScramble.Top = 290
    btnLog.Left = 170
    btnLog.Top = 30
    btnLog.Text = "Log"
    btnHelp.Top = 12
    btnHelp.Left = 697
    txtPalette.ReadOnly = True
    movessequence = ""

    Me.Controls.Add(PaletteY)
    Me.Controls.Add(PaletteW)
    Me.Controls.Add(PaletteB)
    Me.Controls.Add(PaletteG)
    Me.Controls.Add(PaletteR)
    Me.Controls.Add(PaletteO)
    PaletteY.BackColor = Color.Yellow
    PaletteW.BackColor = Color.White
    PaletteB.BackColor = Color.Blue
    PaletteG.BackColor = Color.Green
    PaletteR.BackColor = Color.Red
    PaletteO.BackColor = Color.Orange
    PaletteY.Top = 80
    PaletteY.Left = 15
    PaletteW.Top = 80
    PaletteW.Left = 75
    PaletteB.Top = 140
    PaletteB.Left = 15
    PaletteG.Top = 140
    PaletteG.Left = 75
    PaletteR.Top = 200
    PaletteR.Left = 15
    PaletteO.Top = 200
    PaletteO.Left = 75
    PaletteY.Text = ""
    PaletteW.Text = ""
    PaletteB.Text = ""
    PaletteG.Text = ""

    PaletteR.Text = ""
    PaletteO.Text = ""
```

Although the Form1_Load subroutine is already pretty crowded with code, I think making a separate subroutine for the moves buttons was worth it, and prevents more code being in the Form1_Load subroutine..

```
166
167         Me.Controls.Add(FaceW)
168         FaceW.Left = 310
169         FaceW.Top = 210
170
171         Me.Controls.Add(FaceY)
172         FaceY.Left = 310
173         FaceY.Top = 30
174
175         Me.Controls.Add(FaceB)
176         FaceB.Left = 310
177         FaceB.Top = 120
178
179         Me.Controls.Add(FaceG)
180         FaceG.Left = 490
181         FaceG.Top = 120
182
183         Me.Controls.Add(FaceR)
184         FaceR.Left = 400
185         FaceR.Top = 120
186
187         Me.Controls.Add(FaceO)
188         FaceO.Left = 220
189         FaceO.Top = 120
190
191         For i = 0 To 2
192             For j = 0 To 2
193                 For k = 0 To 2
194                     Cube(i, j, k) = New Cubie
195                     If i <> 1 And j <> 1 And k <> 1 Then
196                         Cube(i, j, k).SetCubieType(3) 'corner
197                     ElseIf (i = 1 And j <> 1 And k <> 1) Or (i <> 1 And j <> 1 And k = 1) Or (i <> 1 And j = 1 And k <> 1) Then
198                         Cube(i, j, k).SetCubieType(2) 'edge
199                     ElseIf (i = 1 And j = 1 And k <> 1) Or (i <> 1 And j = 1 And k = 1) Or (i = 1 And j <> 1 And k = 1) Then
200                         Cube(i, j, k).SetCubieType(1) 'centre
201                     End If
202                 Next
203             Next
204         Next
205         Cube(1, 1, 1).SetCubieType(0) 'middle middle middle
206
207         Moves.reset(Cube, FaceW, FaceY, FaceG, FaceB, FaceR, FaceO)
208
209         sLogString = sLogString + "Session started.
210 "
211     End Sub
```

btnReset_Click

```
        0 references
213     Private Sub btnReset_Click(sender As Object, e As EventArgs) Handles btnReset.Click
214         Moves.reset(Cube,
215                     FaceW, FaceY,
216                     FaceG, FaceB,
217                     FaceR, FaceO)
218         If movessequence <> "" Then
219             sLogString = sLogString + "
220 "
221         End If
222         movessequence = ""
223         sLogString = sLogString + "Cube resetted.
224 "
225     End Sub
```

L_Click

```
        0 references
227     Private Sub L_Click(sender As Object, e As EventArgs) Handles btnL.Click
228         Moves.L(Cube, FaceO, FaceB, FaceG, FaceY, FaceW)
229         Dim cubesolved As Boolean = CheckSolved("L")
230     End Sub
```

R_Click

```
        0 references
232     Private Sub R_Click(sender As Object, e As EventArgs) Handles btnR.Click
233         Moves.R(Cube, FaceR, FaceB, FaceG, FaceY, FaceW)
234         Dim cubesolved As Boolean = CheckSolved("R")
235     End Sub
```

U_Click

```
0 references
Private Sub U_Click(sender As Object, e As EventArgs) Handles btnU.Click
    Moves.U(Cube, FaceY, FaceB, FaceG, FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("U")
End Sub
```

D_Click

```
0 references
Private Sub D_Click(sender As Object, e As EventArgs) Handles btnD.Click
    Moves.D(Cube, FaceW, FaceB, FaceG, FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("D")
End Sub
```

F_Click

```
0 references
Private Sub F_Click(sender As Object, e As EventArgs) Handles btnF.Click
    Moves.F(Cube, FaceB, FaceY, FaceW, FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("F")
End Sub
```

B_Click

```
0 references
Private Sub B_Click(sender As Object, e As EventArgs) Handles btnB.Click
    Moves.B(Cube, FaceG, FaceY, FaceW, FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("B")
End Sub
```

M_Click

```
0 references
Private Sub M_Click(sender As Object, e As EventArgs) Handles btnM.Click
    Moves.M(Cube, FaceB, FaceY, FaceG, FaceW)
    Dim cubesolved As Boolean = CheckSolved("M")
End Sub
```

E_Click

```
0 references
Private Sub E_Click(sender As Object, e As EventArgs) Handles btnE.Click
    Moves.E(Cube,
            FaceB, FaceG,
            FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("E")
End Sub
```

S_Click

```
0 references
Private Sub S_Click(sender As Object, e As EventArgs) Handles btnS.Click
    Moves.S(Cube,
            FaceY, FaceW,
            FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("S")
End Sub
```

btnLa_Click

```
0 references
Private Sub btnLa_Click(sender As Object, e As EventArgs) Handles btnLa.Click
    Moves.La(Cube, FaceO, FaceB, FaceG, FaceY, FaceW)
    Dim cubesolved As Boolean = CheckSolved("L'")
End Sub
```

btnRa_Click

```
0 references
Private Sub btnRa_Click(sender As Object, e As EventArgs) Handles btnRa.Click
    Moves.Ra(Cube, FaceR, FaceB, FaceG, FaceY, FaceW)
    Dim cubesolved As Boolean = CheckSolved("R'")
End Sub
```

btnUa_Click

```
0 references
Private Sub btnUa_Click(sender As Object, e As EventArgs) Handles btnUa.Click
    Moves.Ua(Cube, FaceY, FaceB, FaceG, FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("U'")
End Sub
```

btnDa_Click

```
0 references
Private Sub btnDa_Click(sender As Object, e As EventArgs) Handles btnDa.Click
    Moves.Da(Cube, FaceW, FaceB, FaceG, FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("D'")
End Sub
```

btnFa_Click

```
0 references
Private Sub btnFa_Click(sender As Object, e As EventArgs) Handles btnFa.Click
    Moves.Fa(Cube, FaceB, FaceY, FaceW, FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("F'")
End Sub
```

btnBa_Click

```
0 references
Private Sub btnBa_Click(sender As Object, e As EventArgs) Handles btnBa.Click
    Moves.Ba(Cube, FaceG, FaceY, FaceW, FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("B'")
End Sub
```

btnMa_Click

```
0 references
Private Sub btnMa_Click(sender As Object, e As EventArgs) Handles btnMa.Click
    Moves.Ma(Cube, FaceB, FaceY, FaceG, FaceW)
    Dim cubesolved As Boolean = CheckSolved("M'")
End Sub
```

btnEa_Click

```
0 references
Private Sub btnEa_Click(sender As Object, e As EventArgs) Handles btnEa.Click
    Moves.Ea(Cube,
             FaceB, FaceG,
             FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("E'")
End Sub
```

btnSa_Click

```
0 references
Private Sub btnSa_Click(sender As Object, e As EventArgs) Handles btnSa.Click
    Moves.Sa(Cube, FaceY, FaceW, FaceR, FaceO)
    Dim cubesolved As Boolean = CheckSolved("S'")
End Sub
```

btnx_Click

```
0 references
Private Sub btnx_Click(sender As Object, e As EventArgs) Handles btnx.Click
    Moves.x(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
    Dim cubesolved As Boolean = CheckSolved("x")
End Sub
```

btny_Click

```
0 references
Private Sub btny_Click(sender As Object, e As EventArgs) Handles btny.Click
    Moves.y(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
    Dim cubesolved As Boolean = CheckSolved("y")
End Sub
```

btnz_Click

```
0 references
Private Sub btnz_Click(sender As Object, e As EventArgs) Handles btnz.Click
    Moves.z(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
    Dim cubesolved As Boolean = CheckSolved("z")
End Sub
```

btnxa_Click

```
0 references
Private Sub btnxa_Click(sender As Object, e As EventArgs) Handles btnxa.Click
    Moves.xa(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
    Dim cubesolved As Boolean = CheckSolved("x'")
End Sub
```

btnya_Click

```
0 references
Private Sub btnya_Click(sender As Object, e As EventArgs) Handles btnya.Click
    Moves.ya(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
    Dim cubesolved As Boolean = CheckSolved("y'")
End Sub
```

btnza_Click

```
0 references
Private Sub btnza_Click(sender As Object, e As EventArgs) Handles btnza.Click
    Moves.za(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
    Dim cubesolved As Boolean = CheckSolved("z'")
End Sub
```

btnHelp_Click

```
0 references
Private Sub btnHelp_Click(sender As Object, e As EventArgs) Handles btnHelp.Click
    Dim helpstring As String = "Help box.

Moves:
R - Turns the right face clockwise by 90
L - Turns the left face clockwise
U - Turns the up face clockwise
D - Turns the down face clockwise
F - Turns the front face clockwise
B - Turns the back face clockwise
Rotations:
x - Rotates the cube from down to up
y - Rotates the cube from right to left
z - Rotates the cube clockwise around the front
If the button has a ', it turns the corresponding face or rotation anticlockwise.

To use the palette, either:
1. Select the colour from the palette that you wish to put on the cube
2. Select the face;et you wish to change colour
Or:
1. Select the facelet you wish to edit the colour of.
2. Select the colour from the palette that you wish to put on the selected facelet.

The scramble has a maximum limit of 30 and includes 180 moves and rotations.
The reset button resets the cube.

Have a nice day!"
        MsgBox(helpstring)
End Sub
```

## ArrangeButtons

```
1 reference
Private Sub ArrangeButtons()

    'top row buttons
    btnL.Top = 315
    btnL.Left = 358 - 3 * 81
    btnR.Top = 315
    btnR.Left = 358 - 2 * 81
    btnU.Top = 315
    btnU.Left = 358 - 1 * 81
    btnD.Top = 315
    btnD.Left = 358 + 0 * 81
    btnF.Top = 315
    btnF.Left = 358 + 1 * 81
    btnB.Top = 315
    btnB.Left = 358 + 2 * 81

    '2nd row buttons
    btnLa.Top = 315 + 29
    btnLa.Left = 358 - 3 * 81
    btnRa.Top = 315 + 29
    btnRa.Left = 358 - 2 * 81
    btnUa.Top = 315 + 29
    btnUa.Left = 358 - 1 * 81
    btnDa.Top = 315 + 29
    btnDa.Left = 358 + 0 * 81
    btnFa.Top = 315 + 29
    btnFa.Left = 358 + 1 * 81
    btnBa.Top = 315 + 29
    btnBa.Left = 358 + 2 * 81

    '3rd row buttons
    btnM.Top = 315 + 29 * 2
    btnM.Left = 358 - 3 * 81
    btnE.Top = 315 + 29 * 2
    btnE.Left = 358 - 2 * 81
    btnS.Top = 315 + 29 * 2
    btnS.Left = 358 - 1 * 81
    btnx.Top = 315 + 29 * 2
    btnx.Left = 358 + 0 * 81
    btny.Top = 315 + 29 * 2
    btny.Left = 358 + 1 * 81
    btnz.Top = 315 + 29 * 2
    btnz.Left = 358 + 2 * 81
```

```
426
427         '4th row buttons
428         btnMa.Top = 315 + 29 * 3
429         btnMa.Left = 358 - 3 * 81
430         btnEa.Top = 315 + 29 * 3
431         btnEa.Left = 358 - 2 * 81
432         btnSa.Top = 315 + 29 * 3
433         btnSa.Left = 358 - 1 * 81
434         btnxa.Top = 315 + 29 * 3
435         btnxa.Left = 358 + 0 * 81
436         btnya.Top = 315 + 29 * 3
437         btnya.Left = 358 + 1 * 81
438         btnza.Top = 315 + 29 * 3
439         btnza.Left = 358 + 2 * 81
440
441         btnL.Text = "L"
442         btnR.Text = "R"
443         btnU.Text = "U"
444         btnD.Text = "D"
445         btnF.Text = "F"
446         btnB.Text = "B"
447         btnLa.Text = "L'"
448         btnRa.Text = "R'"
449         btnUa.Text = "U'"
450         btnDa.Text = "D'"
451         btnFa.Text = "F'"
452         btnBa.Text = "B'"
453         btnM.Text = "M"
454         btnE.Text = "E"
455         btnS.Text = "S"
456         btnx.Text = "x"
457         btny.Text = "y"
458         btnz.Text = "z"
459         btnMa.Text = "M'"
460         btnEa.Text = "E'"
461         btnSa.Text = "S'"
462         btnxa.Text = "x'"
463         btnya.Text = "y'"
464         btnza.Text = "z'"
465
466     End Sub
```

## CheckSolved

```
26 references
468 Public Function CheckSolved(moveinitial As String)
469     Dim cubesolved As Boolean = True
470     Dim i As Integer
471     Dim j As Integer
472     Dim faceletnumber As Integer = 0
473
474     Do While cubesolved = True And faceletnumber < 9
475         i = Math.Floor(faceletnumber / 3)
476         j = faceletnumber Mod 3
477         If FaceW.faceletsArray(i, j).BackColor <> Color.White Then
478             cubesolved = False
479         ElseIf FaceY.faceletsArray(i, j).BackColor <> Color.Yellow Then
480             cubesolved = False
481         ElseIf FaceB.faceletsArray(i, j).BackColor <> Color.Blue Then
482             cubesolved = False
483         ElseIf FaceG.faceletsArray(i, j).BackColor <> Color.Green Then
484             cubesolved = False
485         ElseIf FaceR.faceletsArray(i, j).BackColor <> Color.Red Then
486             cubesolved = False
487         ElseIf FaceO.faceletsArray(i, j).BackColor <> Color.Orange Then
488             cubesolved = False
489         End If
490         faceletnumber = faceletnumber + 1
491     Loop
492
493     If moveinitial <> "" Then
494         If movessequence = "" Then
495             sLogString = sLogString + "Moves: " + moveinitial
496         Else
497             sLogString = sLogString + ", " + moveinitial
498         End If
499     End If
500
501     If cubesolved = True And moveinitial <> "" Then
502         MsgBox("Congratulations! You have solved the cube.")
503         sLogString = sLogString + "
504 Cube solved.
505 "
506     End If
507
508     movessequence = movessequence + moveinitial
509
510     Return cubesolved
511 End Function
```

Lines 493 to 508 deals with putting moves onto the log in a suitable manner.

## ShowLog

```
0 references
513 Private Sub ShowLog() Handles btnLog.Click
514     MsgBox(sLogString)
515 End Sub
```

## PaletteY_Click

```
0 references
517 Private Sub PaletteY_Click(sender As Object, e As EventArgs) Handles PaletteY.Click
518     If chkEditingMode.Checked = True Then
519         PaletteChanges.PaletteY_click(faceletface, paletteselected,
520                     FaceY, FaceW,
521                     FaceB, FaceG,
522                     FaceR, FaceO)
523     Else
524         MsgBox("You are not in editing mode!")
525     End If
526 End Sub
```

Line 518 uses OOP again (Group A).

PaletteW_Click

```
0 references
Private Sub PaletteW_Click(sender As Object, e As EventArgs) Handles PaletteW.Click
    If chkEditingMode.Checked = True Then
        PaletteChanges.PaletteW_click(faceletface, paletteselected,
                    FaceY, FaceW,
                    FaceB, FaceG,
                    FaceR, FaceO)
    Else
        msgbox("You are not in editing mode!")
    End If
End Sub
```

PaletteB_Click

```
0 references
Private Sub PaletteB_Click(sender As Object, e As EventArgs) Handles PaletteB.Click
    If chkEditingMode.Checked = True Then
        PaletteChanges.PaletteB_click(faceletface, paletteselected,
                    FaceY, FaceW,
                    FaceB, FaceG,
                    FaceR, FaceO)
    Else
        MsgBox("You are not in editing mode!")
    End If
End Sub
```

PaletteG_Click

```
0 references
Private Sub PaletteG_Click(sender As Object, e As EventArgs) Handles PaletteG.Click
    If chkEditingMode.Checked = True Then
        PaletteChanges.PaletteG_click(faceletface, paletteselected,
                    FaceY, FaceW,
                    FaceB, FaceG,
                    FaceR, FaceO)
    Else
        MsgBox("You are not in editing mode!")
    End If
End Sub
```

PaletteR_Click

```
0 references
Private Sub PaletteR_Click(sender As Object, e As EventArgs) Handles PaletteR.Click
    If chkEditingMode.Checked = True Then
        PaletteChanges.PaletteR_click(faceletface, paletteselected,
                    FaceY, FaceW,
                    FaceB, FaceG,
                    FaceR, FaceO)
    Else
        MsgBox("You are not in editing mode!")
    End If
End Sub
```

PaletteO_Click

```
0 references
Private Sub PaletteO_Click(sender As Object, e As EventArgs) Handles PaletteO.Click
    If chkEditingMode.Checked = True Then
        PaletteChanges.PaletteO_click(faceletface, paletteselected,
                    FaceY, FaceW,
                    FaceB, FaceG,
                    FaceR, FaceO)
    Else
        MsgBox("You are not in editing mode!")
    End If
End Sub
```

## btnSolve_Click

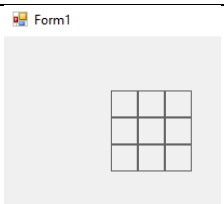
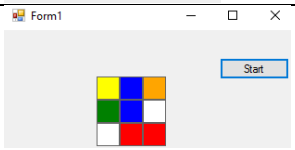
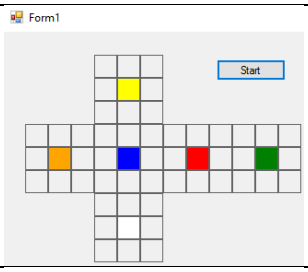
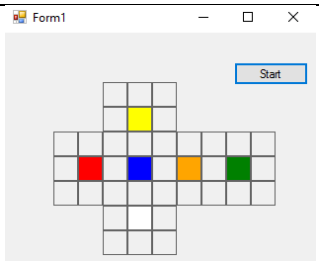
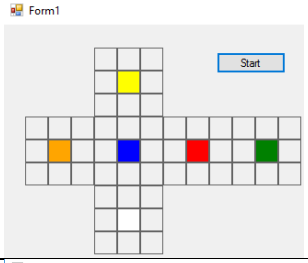
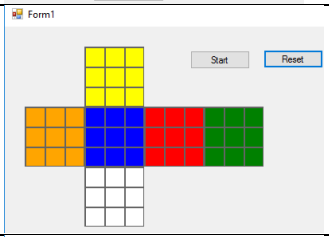
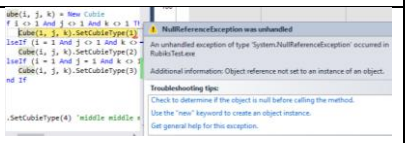
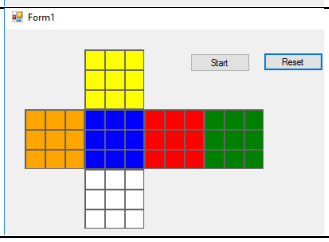
```
0 references
Private Sub btnSolve_Click(sender As Object, e As EventArgs) Handles btnSolve.Click
    Dim returnstring As String = Moves.CheckSolvable(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
    If returnstring = "" Then
        Moves.SOLVE(Cube, FaceR, FaceO, FaceB, FaceG, FaceY, FaceW)
    Else
        MsgBox(returnstring)
    End If
    If movessequence <> "" Then
        sLogString = sLogString + "
"
    End If
    movessequence = ""
    sLogString = sLogString + "Solve asked for.
"
End Sub
```

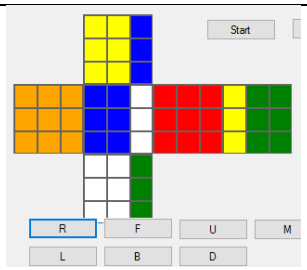
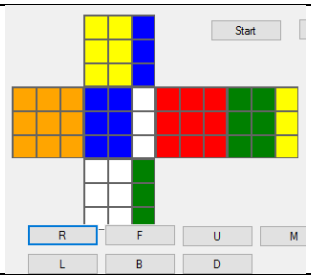
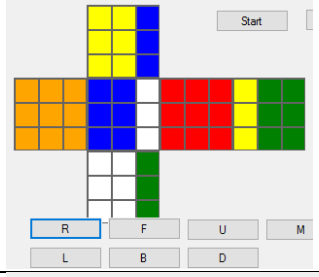
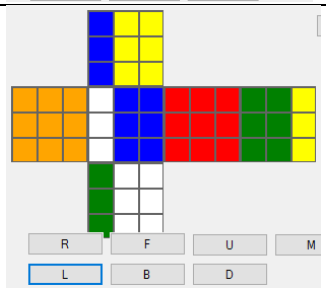
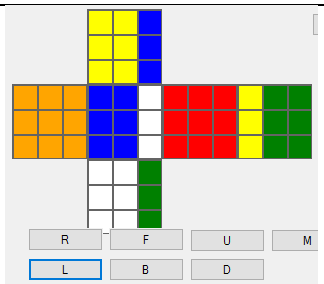
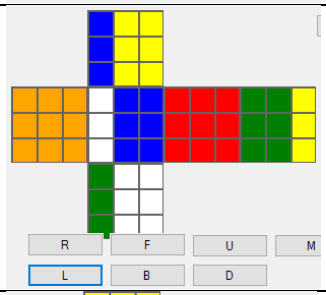
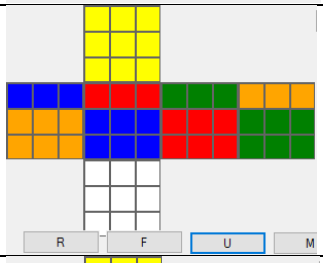
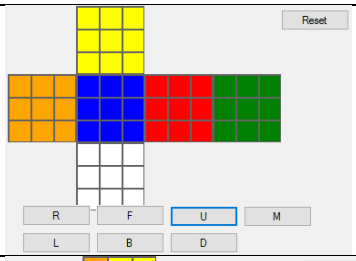
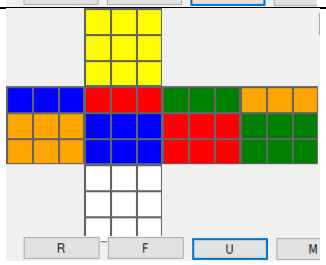
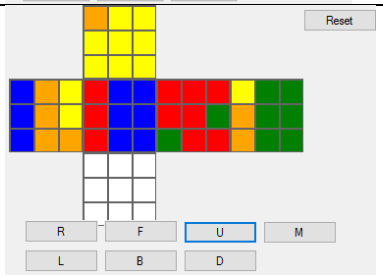
## checkEditingMode

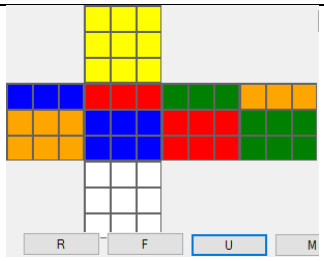
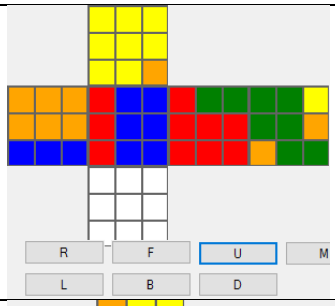
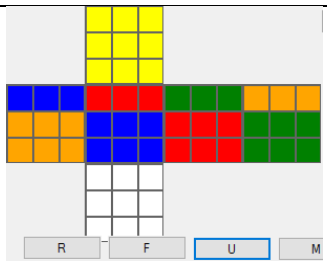
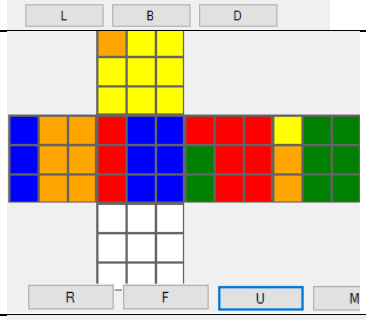
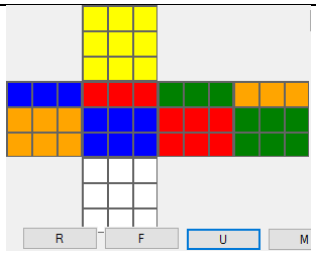
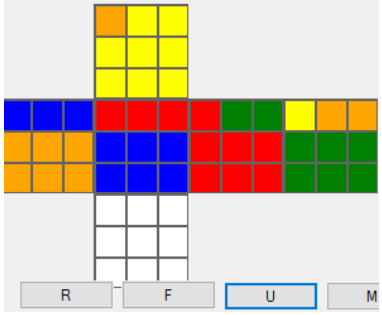
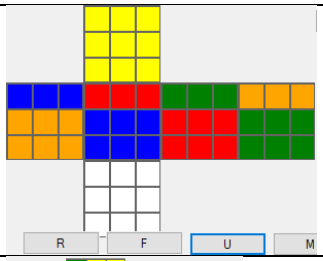
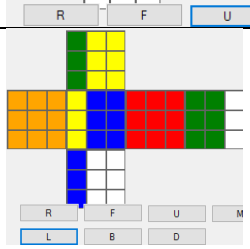
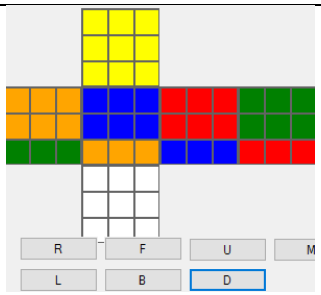
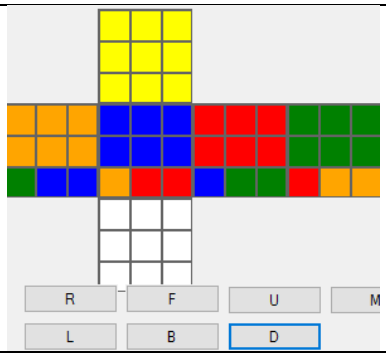
```
1 reference
Public Function checkEditingMode() As Boolean
    Return chkEditingMode.Checked
End Function
End Class
```

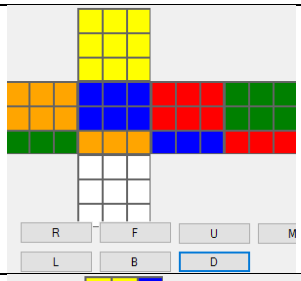
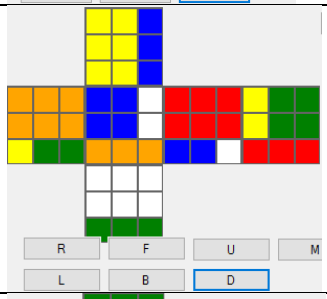
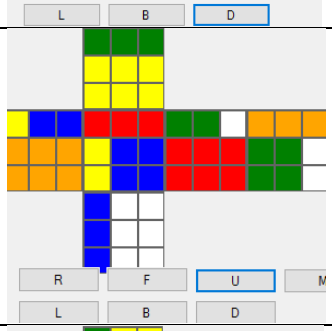
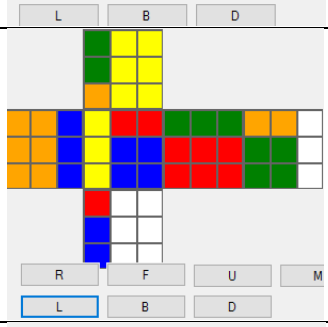
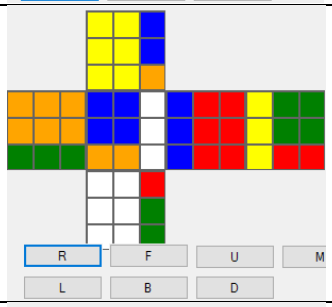
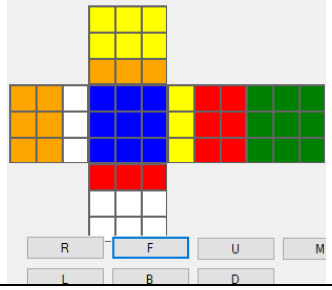
# Testing

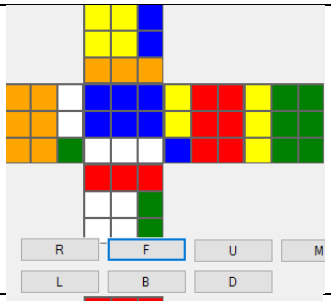
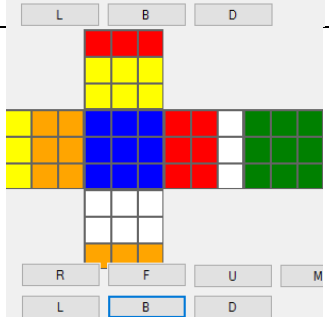
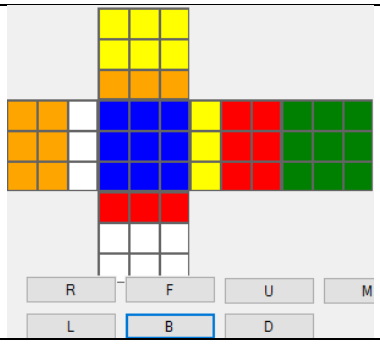
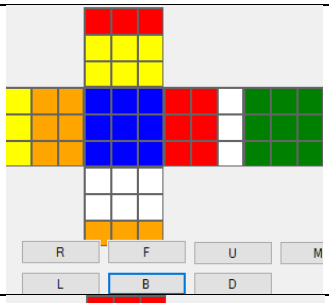
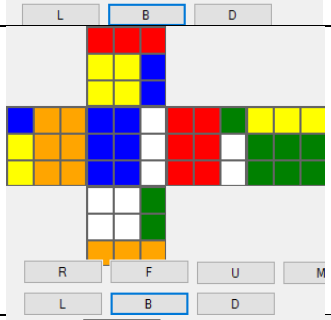
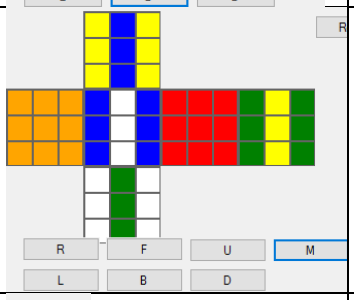
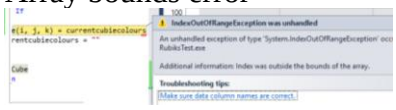
## Test Log

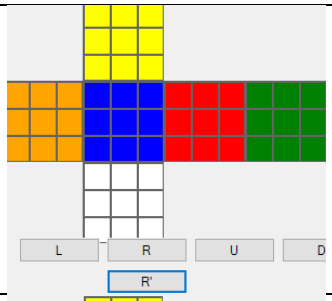
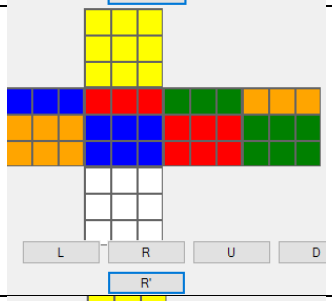
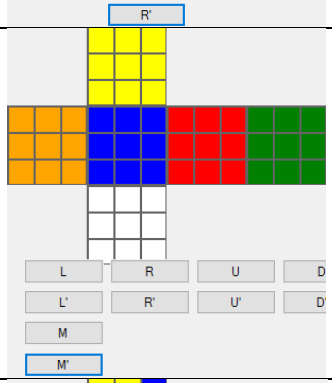
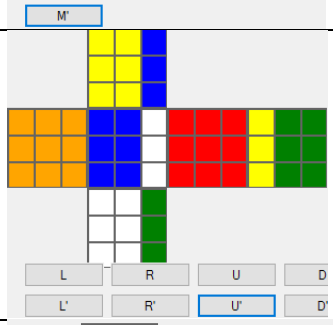
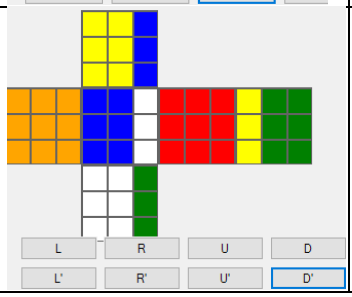
| Test No | Description | Test Data | Expected Results | Pass / Actual Results |
|---|---|---|---|---|
| 1 | Check if the face shows up with 9 facelets. | N/A |  | Pass |
| 2 | The 9 facelets on Face1 change colours between white, yellow, blue, green, red and orange when "Start" button is pressed. | Press "Start" button |  | Pass |
| 3 | The nine faces to align correctly in a net. It looks like I miscalculated, and thought the faces were 50 pixels wide instead of 75. | N/A |  |  |
| 4 | See Test 3. | N/A |  | Pass |
| 5 | When I click on the "Reset" button, the faces should colour themselves by their respective colours. | Press "Reset" button |  |  <br>Array Boundary Error |
| 6 | See Test 5. I changed a line: `Cube(i, j, k).SetCubieType(1) '` | Press "Reset" button |  | Pass |

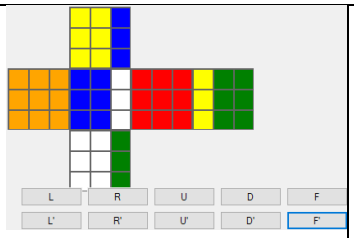
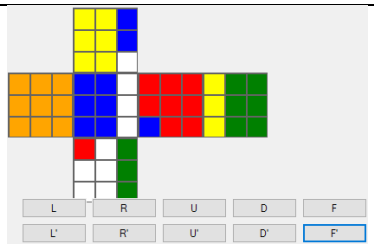
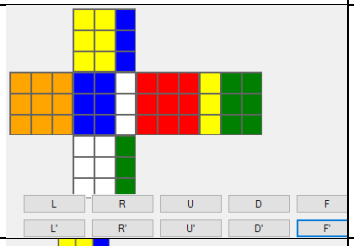
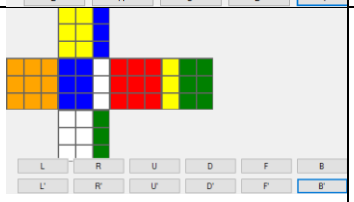
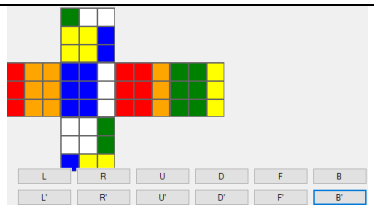
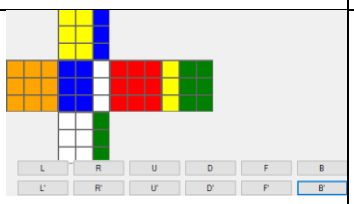
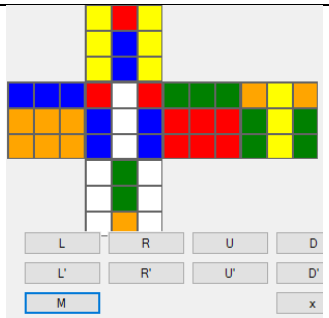
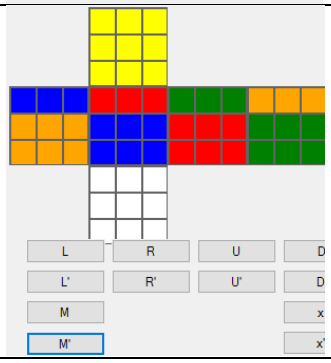
| | | | | |
|---|---|---|---|---|
| 7 | When I press the "R" button, the facelets should mirror how an R move would look like on a 3D cube. | Press "R" button |  |  |
| 8 | See Test 7. | Press "R" button |  | Pass |
| 9 | When pressing the "L" button, it should show the equivalent of turning the left face clockwise. Looks like I called the R subroutine from the form. | Press "L" button |  |  |
| 10 | See Test 9. | Press "L" button |  | Pass |
| 11 | When pressing the "U" button, it should show the equivalent of turning the up face clockwise. I didn't call the subroutine from the form. | Press "U" button |  |  |
| 12 | See Test 11. To deal with this, I will comment every line and decomment them. I have realised that I took the cubies' positions relative to how they are seen when looking at the face. | Press "U" button |  |  |

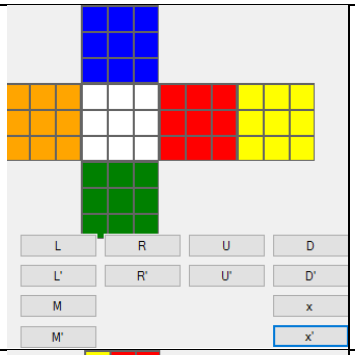
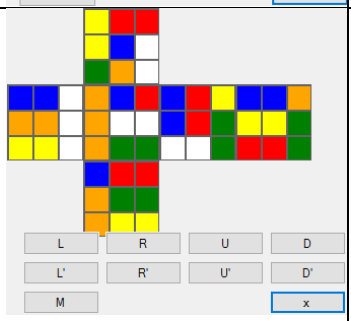
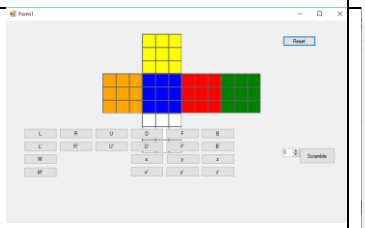
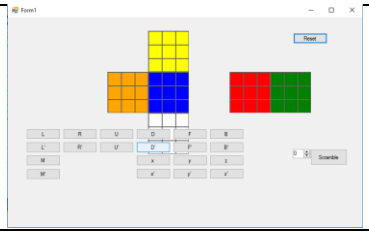
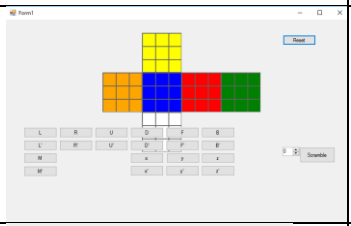
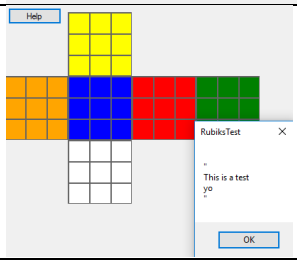
| | | | | |
|---|---|---|---|---|
| 13 | See Test 12. I don't know what happened. | Press "U" button |  |  |
| 14 | See Test 13. I realised I got the rows and columns mixed up so I changed the lines in the moves subroutines and in the Face class. | Press "U" button |  |  |
| 15 | See Test 14. I changed back the lines in the Face class as the rows and columns were being swapped twice. We're getting closer. I think the problem is in the "UL other faces" section. | Press "U" button |  |  |
| 16 | See Test 15. | Press "U" button |  | Pass |
| 17 | I realised from previous tests that the current L subroutine is modelling the anticlockwise move. Therefore, I will change that subroutine. | Press "L" button |  | Pass |
| 18 | When pressing the "D" button, it should show the equivalent of turning the down face clockwise. I have seen this before, so I will change the "DL other faces" section. | Press "D" button |  |  |

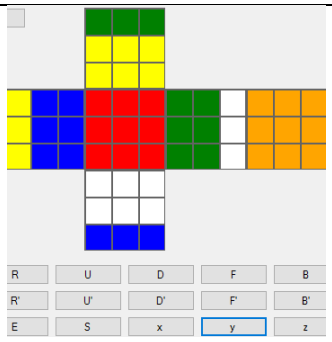
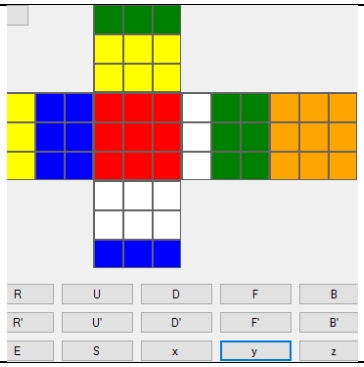
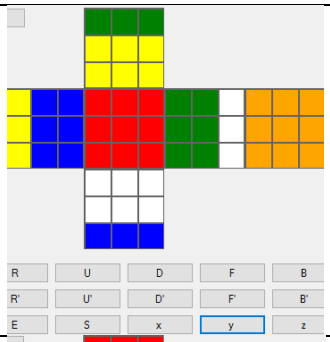
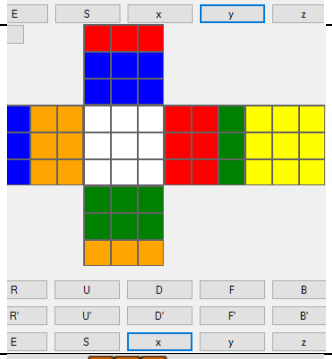
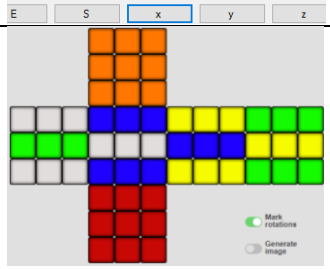
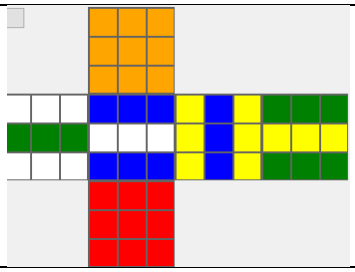
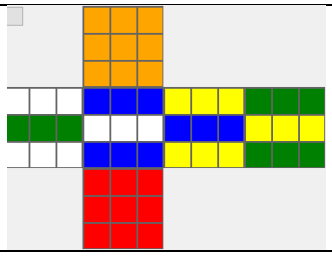
| | | | | |
|---|---|---|---|---|
| 19 | See Test 18. | Press "D" button |  | Pass |
| 19 | To see if the D move works with the down face, I did R then D. I compared my result with the one from a website and it matches. | Press "R" button then "D" button |  | Pass |
| 20 | To see if the U move works with the up face, I did L then U. I compared my result with the one from a website and it matches. | Press "L" button then "U" button |  | Pass |
| 21 | To see if the L move works with the left face, I did U then L. I compared my result with the one from a website and it matches. | Press "U" button then "L" button |  | Pass |
| 22 | To see if the R move works with the right face, I did D then R. I compared my result with the one from a website and it matches. | Press "D" button then "R" button |  | Pass |
| 23 | When pressing the "F" button, it should show the equivalent of turning the front face clockwise. | Press "F" button |  | Pass |

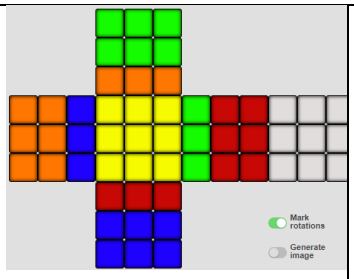
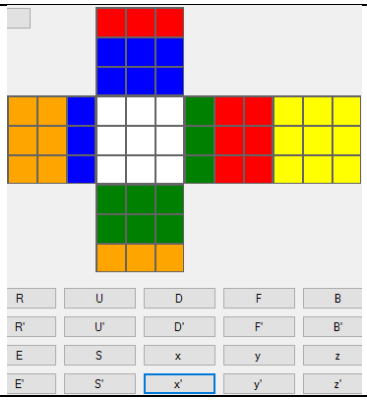
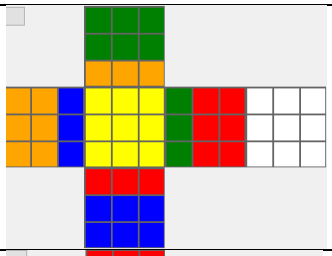
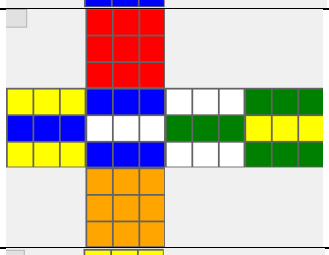
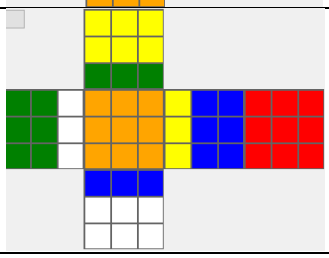
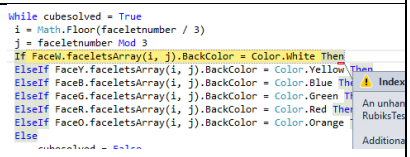
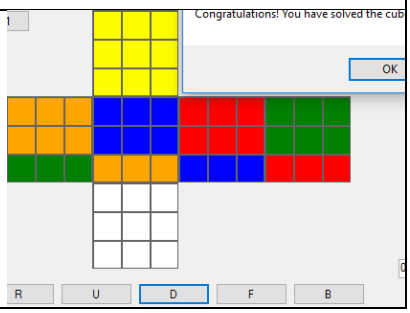
| | | | | |
|---|---|---|---|---|
| 24 | To see if the F move works with the front face, I will do R then F. I will compare my result with the one from a website and it matches. | Press “R” button then “F” button |  | Pass |
| 25 | When pressing the “B” button, it should show the equivalent of turning the back face clockwise. I called the F subroutine from the form when the B button was clicked. | Press “B” button |  |  |
| 26 | See Test 25. | Press “B” button |  | Pass |
| 27 | To see if the B move works with the back face, I will do R then B. I will compare my result with the one from a website and it matches. | Press “R” button then “B” button |  | Pass |
| 28 | When pressing the “M” button, it should show the equivalent of turning the middle column clockwise. | Press “M” button |  | Pass |
| 29 | When clicking on the up arrow on the up-down Scramble control, it should stop at 30. | Press up button on said control | | Pass |
| 30 | See if I can assign a string array to the properties of another with the same size | | Form1 pops up. | Array bounds error  |

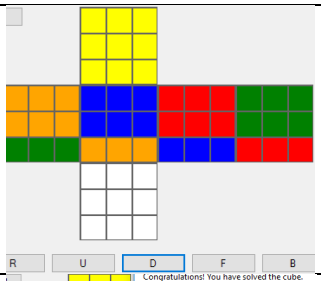
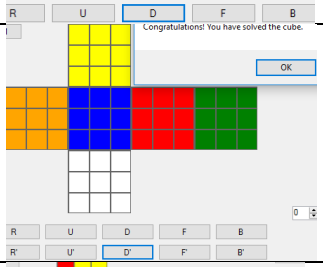
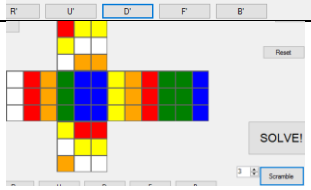
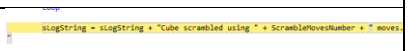
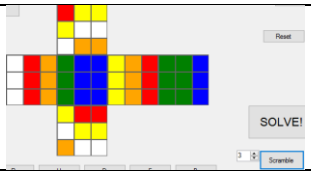
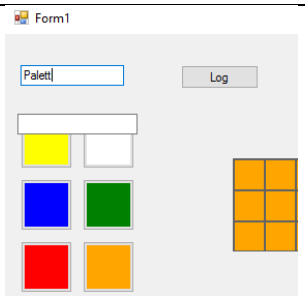
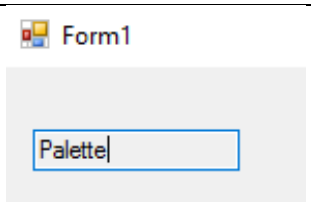
| | | | | |
|---|---|---|---|---|
| 31 | When doing integer division, it looks like the program rounded to the nearest whole number. I added Math.Floor around all integer divisions in my program. | | Form1 pops up. | Pass. |
| 32 | To test the R' subroutine, I will execute R then R' and the cube should be as its default position. | Press R button then R' button |  | Pass |
| 33 | To test the facelets on the right face, I will do U, R, R' which should end up showing the move U. | Press U button, then R button, then R' button |  | Pass |
| 34 | To test the M' subroutine, I will execute M then M' and the cube should be as its default position. | Press M button then M' button |  | Pass |
| 35 | To test the facelets on the up face, I will do R, U, U' which should end up showing the move R. | Press R button, then U button, then U' button |  | Pass |
| 36 | To test the facelets on the down face, I will do R, D, D' which should end up showing the move R. | Press R button, then D button, then D' button |  | |

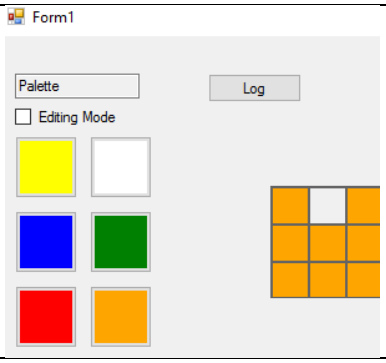
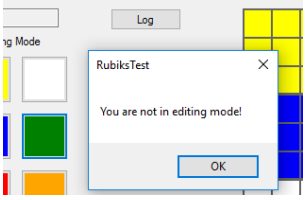
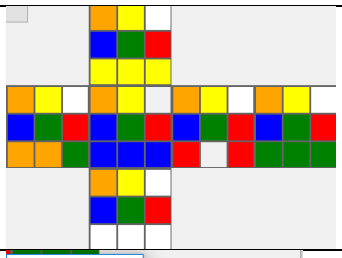
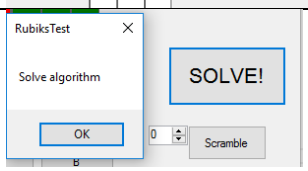
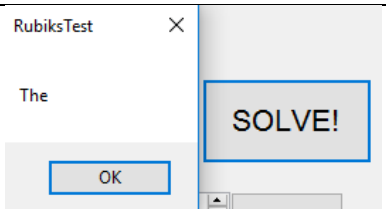
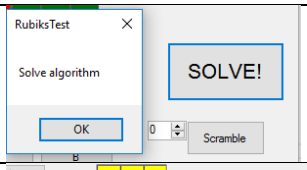
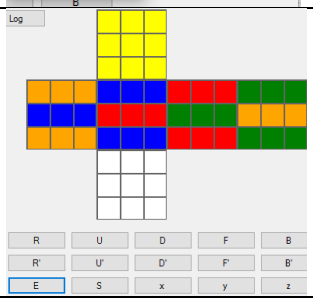
| | | | | |
|---|---|---|---|---|
| 37 | To test the facelets on the front face, I will do R, F, F' which should end up showing the move R. It looks like the problem was in the "UR other faces" section. | Press R button, then F button, then F' button |  |  |
| 38 | See test 37. | Press R button, then F button, then F' button |  | Pass |
| 39 | To test the facelets on the back face, I will do R, B, B' which should end up showing the move R. It looks like it was turning the face clockwise. | Press R button, then B button, then B' button |  |  |
| 40 | See Test 39. | Press R button, then B button, then B' button |  | Pass |
| 41 | To see if the M move works, I executed U then M | Press U button, then M button |  | Pass |
| 42 | See Test 41. When pressing M', it should show the U move | Press M' button |  | Pass |

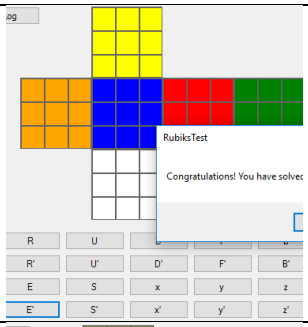
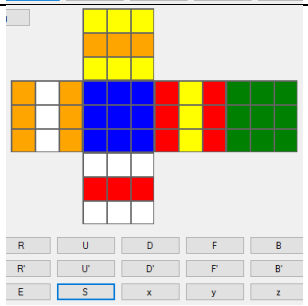
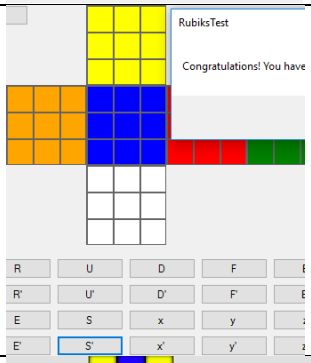
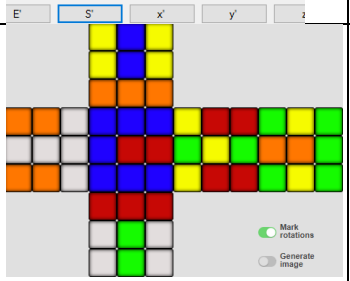
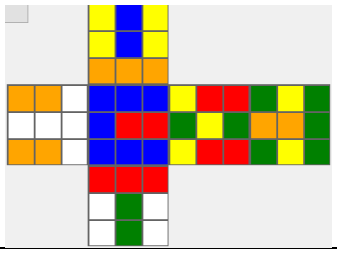
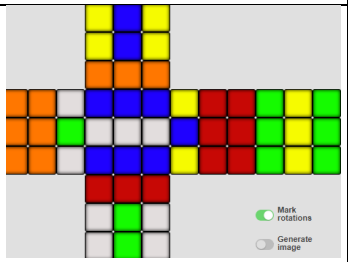
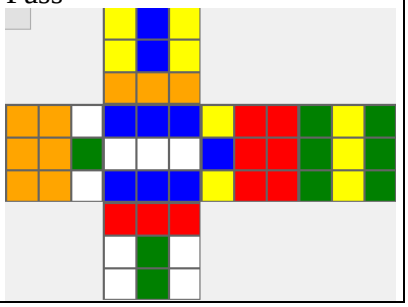
| | | | | |
|---|---|---|---|---|
| 43 | Test the x rotation subroutine | Press "x" button |  | Pass |
| 44 | To test if the x subroutine works for all facelets, I will do a random series of moves (R, U, L, B, D) and then x to see if the result matches the one from a website. I also counted the number of facelets of each colour to check if they add up. | Press "R", "U", "L", "B" and "D" buttons |  | Pass |
| 45 | I changed the faces' coordinates and sizes so that it matches the design UI. | |   | |
| 46 | See Test 45. | |  | Pass |
| 47 | When pressing the "Help" button, a message box should appear with a predefined string. | Press "Help" button |  | There are quotation marks around the text, but this is easily resolvable. |
| 48 | After some messing around, I realised that the x or x' subroutines don't work. | Mess around | | |

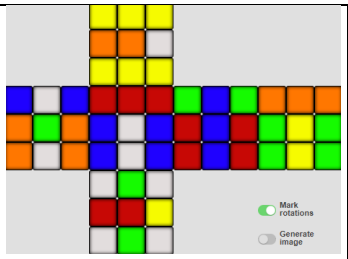
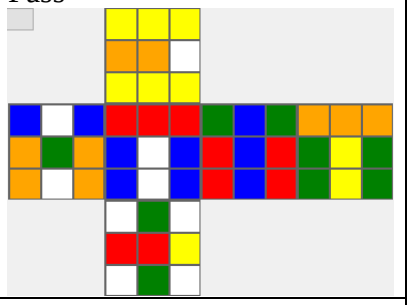
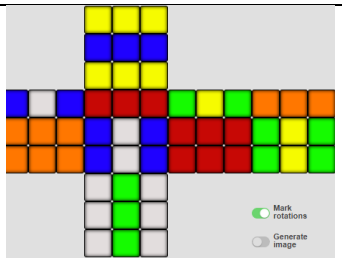
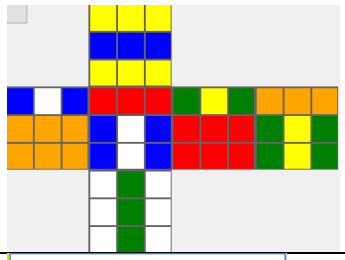
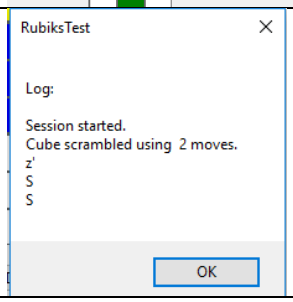
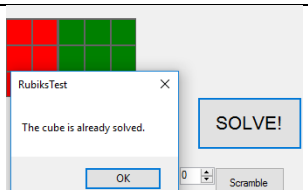
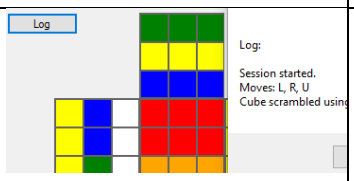
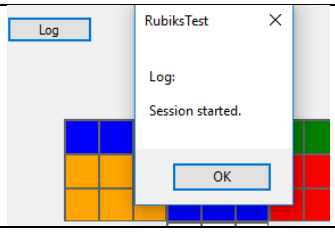
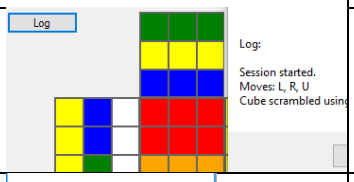
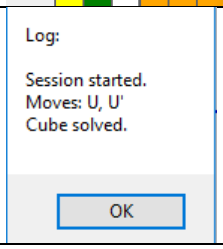
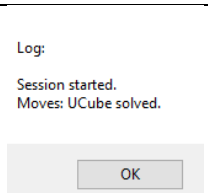
| | | | | |
|---|---|---|---|---|
| 49 | To see if the y subroutine works, I did L then y. | Press "L" button then "y" button |  |  |
| 50 | See Test 49. | |  | Pass |
| 51 | To see if the x subroutine works, I will do U then x. | Press "U" button then "x" button |  | Pass |
| 52 | To see if the z subroutine works, I will do M then z. | Press "M" button then "z" button |  |  |
| 53 | See Test 52. | Press "M" button then "z" button |  | Pass |

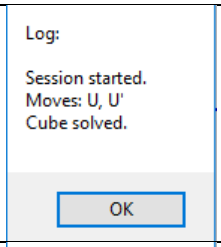
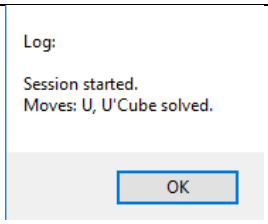
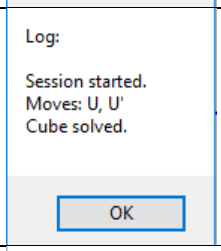
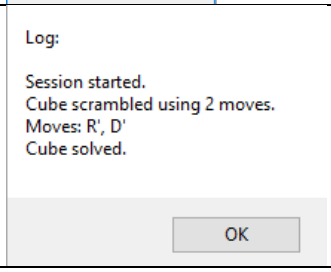
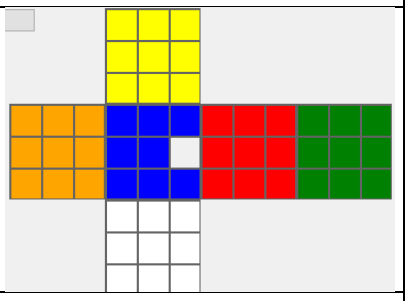
| | | | | |
|---|---|---|---|---|
| 54 | To see if the x’ subroutine works, I will do U then x’. | Press “U” button then “x’” button |  |  |
| 55 | See Test 54. | Press “U” button then “x’” button |  | Pass |
| 56 | To see if the z’ subroutine works, I will do M then z’. | Press “M” button then “z’” button |  | Pass |
| 57 | To see if the y’ subroutine works, I will do L then y’. | Press “L” button then “y’” button |  | Pass |
| 58 | To see if the CheckSolved function works, I pressed a random move. | Press any move button | |  |
| 59 | See Test 58. The message box pops up even when the cube is not solved. | Press any move button | |  |

| | | | | |
|---|---|---|---|---|
| 60 | See Test 59. | Press any move button |  | Pass |
| 61 | See Test 60. | Press Test 60's move's inverse's button |  | Pass |
| 62 | Test if the log works with the scramble subroutine. It looks like it is adding an integer with a string which it doesn't like. | Press "Scramble" button |  |  |
| 63 | Test the scramble subroutine | Press "Scramble" button |  | Pass |
| 64 | Test if the log works. I pressed some random moves, then did a scramble, then reset the cube. | Press moves buttons, the scramble button | | Fail |
| 65 | I found out that the palette textbox is writable, but it would be preferable if not. | Delete character from Palette textbox | |  |
| 66 | See Test 65. | Try to delete character from Palette textbox |  | Pass |
| 67 | When clicking on a palette and then a facelet, it appears that it only works for the colour yellow. | | | |

| | | | | |
|---|---|---|---|---|
| 68 | When clicking on the blue palette and then on an orange facelet, it puts it as grey. | Click on blue palette | |  |
| 69 | I put in a line so that the program reminds the user that they are not in Editing Mode if the tickbox is not ticked. | Press any palette colour |  | Pass |
| 70 | I tested each of the palette colours with all the faces. | |  | Pass |
| 71 | Check if the solve button checks if the cube is solvable and calls the solve subroutine, which for the moment only outputs a textbox saying "Solve algorithm" to say that the subroutine has been called. | Press "Solve" button |  |  |
| 72 | See Test 71 | Press "Solve" button |  | Pass |
| 73 | Test to see if the E subroutine works. | Press "E" button |  | Pass |

| | | | | |
|---|---|---|---|---|
| 74 | Test to see if the E’ subroutine works. | Press “E’” button |  | Pass |
| 75 | Test to see if the S subroutine works. | Press “S” button |  | Pass |
| 76 | Test to see if the S’ subroutine works. | Press “S’” button |  | Pass |
| 77 | To see if the E subroutine properly works, I did M, F then E and compared against a website | Press “M”, “F”, then “E” buttons |  | Pass  |
| 78 | To see if the E subroutine properly works, I copied Test 77 then did E’ and compared against a website. | Press “M”, “F”, “E” then “E’” buttons |  | Pass  |

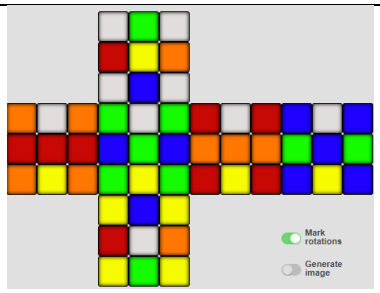
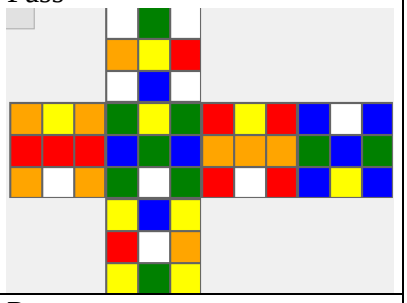
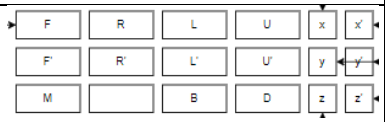
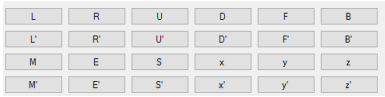
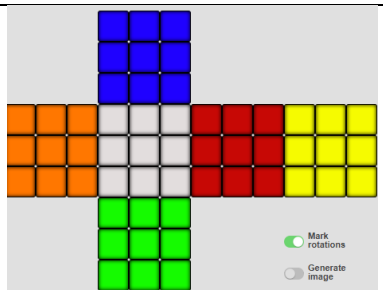
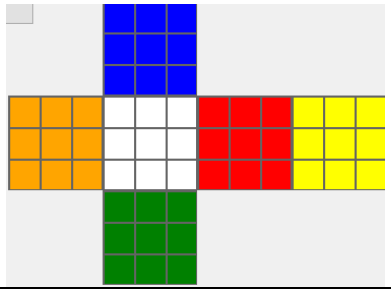
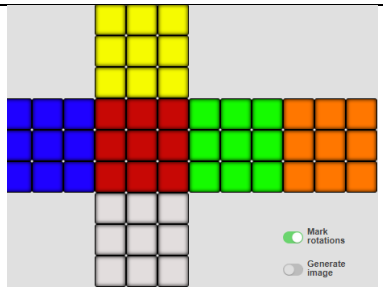
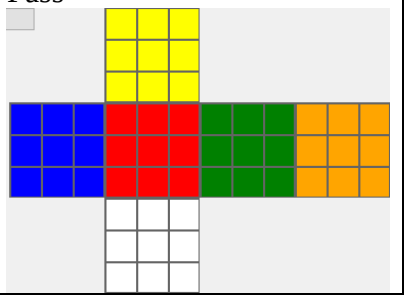
| | | | | |
|---|---|---|---|---|
| 79 | To see if the E' subroutine properly works, I did M, U then S and compared against a website. | Press "M", "U", then "S" buttons |  | Pass  |
| 80 | To see if the S' subroutine properly works, I copied Test 79 then did S' and compared against a website. | Press "M", "U", "S" then "S'" buttons |  | Pass  |
| 81 | When scrambling and then solving the cube, the log should show the scramble, moves made, and the fact that the cube has been solved. | Start a scramble , solve the cube, press "Log" button | |   |
| 82 | Check if the program knows that the cube is solved when the form first pops up. | Press "Solve" button when the form loads |   | Pass |
| 83 | See if the log works with the moves. | Press any move button |   |   |
| 84 | See Test 83. | Press some moves buttons |   | Pass |
| 85 | To see if the program knows if the cube has been solved. | Press "U" then "U'" buttons |   |   |

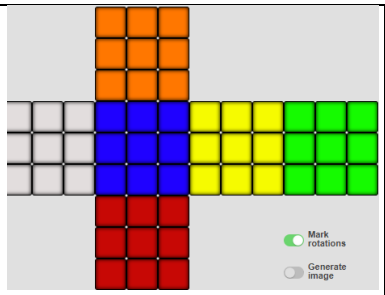
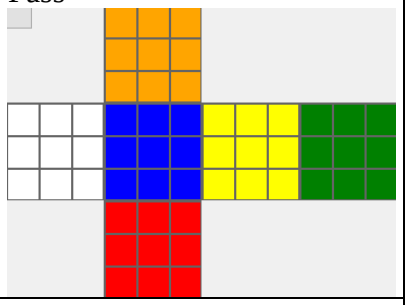
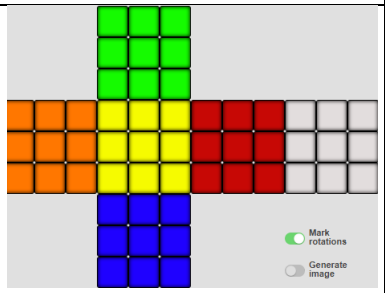
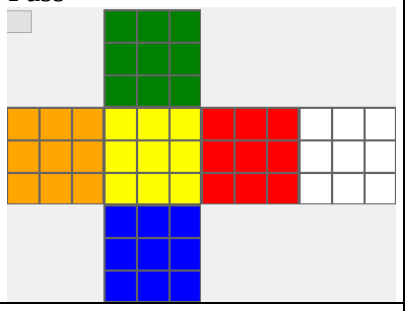
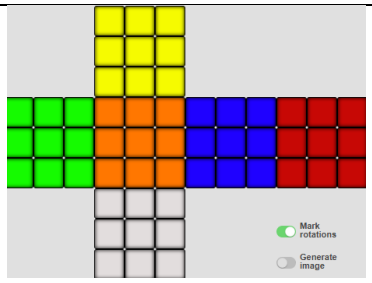
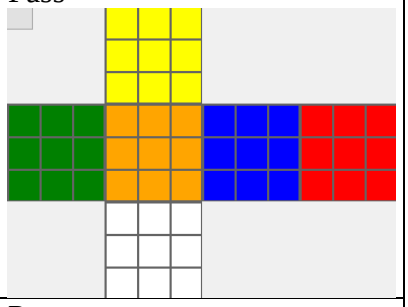
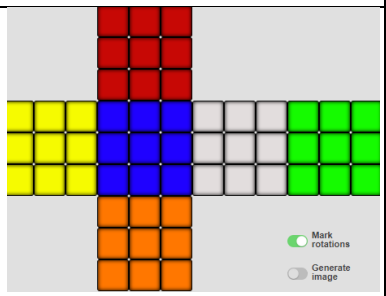
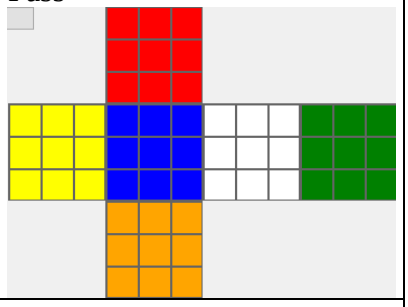
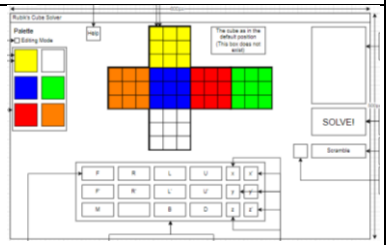
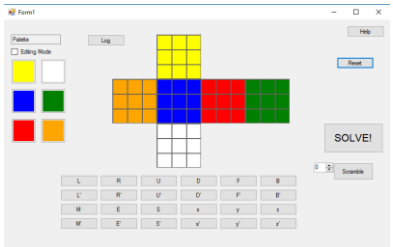
| | | | | |
|---|---|---|---|---|
| 86 | To see if the program outputs the last move before the cube is solved. | Press "U" then "U'" buttons |  |  |
| 87 | See Test 87. | Press "U" then "U'" buttons |  | Pass |
| 88 | See Test 81. | Start a scramble, solve the cube, press "Log" button |  | Pass |
| 89 | Test to see if the program checks if the user is in editing mode when clicking on a facelet. | Click on a facelet. | |  |
| 90 | See Test 89. | Click on a facelet. | | Pass |

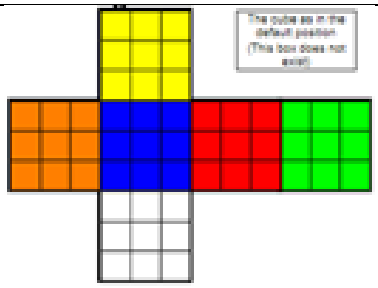
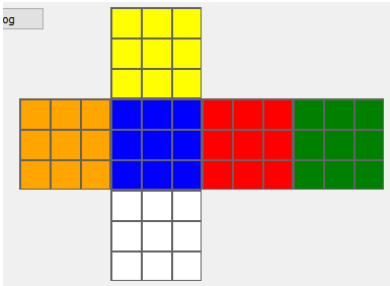
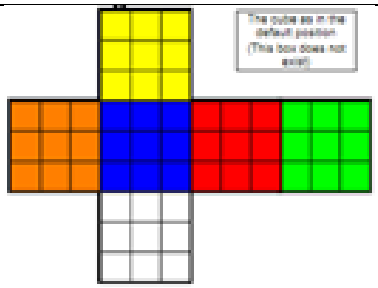
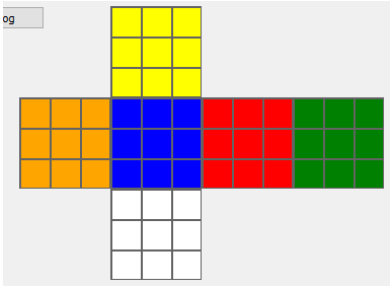
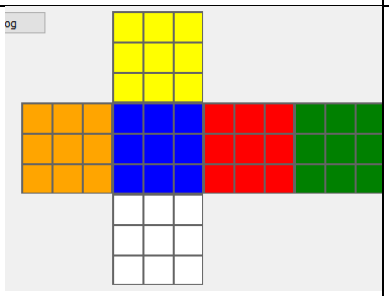
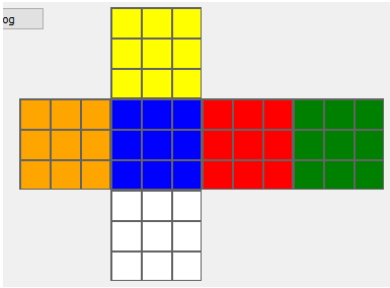
## Objectives Testing

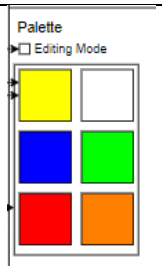
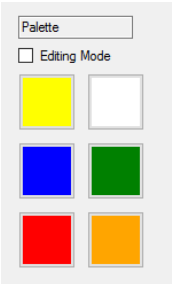
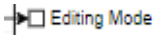
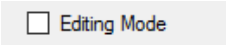
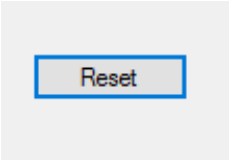
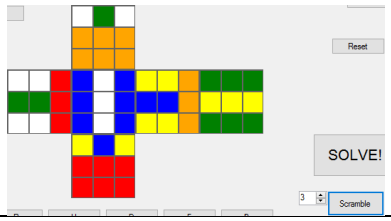
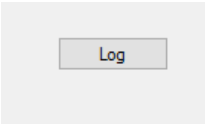
This table is to show which objectives have been met and have passed their tests.

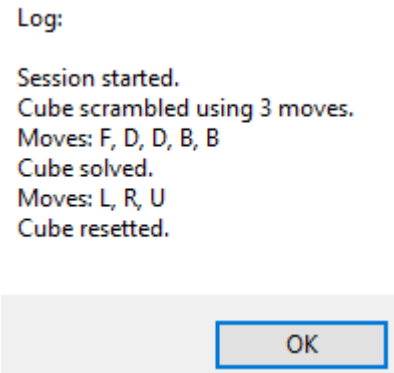
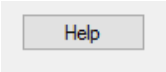
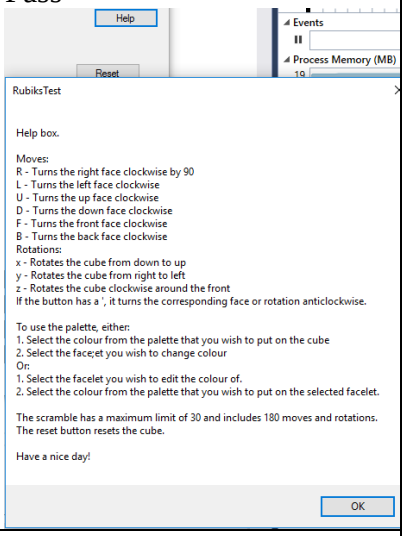
| **Objective No** | **Test Description** | **Test Data** | **Expected Results** | **Pass / Actual Results** |
|---|---|---|---|---|
| 1 | This objective does not need testing as it was worked on and was not met. | N/a | N/a | n/a |

| | | | | |
|---|---|---|---|---|
| 2 | Test every move in a specific order, then compare against a website. | L, R, U, D, F, B, L’, R’, U’, D’, F’, B’, M, E, S, x, y, z, M’, E’, S’, x’, y’, z’ |  | Pass  |
| 3 and 5.7 | When the form loads, the buttons should be neatly arranged and centered, as per the design UI. I have decided to make all of the buttons the same size. | n/a |  | Pass  |
| 4 | Test each of the six rotations. | n/a | n/a | n/a |
| 4 (x) | Check if the x subroutine works. | Press “x” button |  | Pass  |
| 4 (y) | Check if the y subroutine works. | Press “y” button |  | Pass  |

| | | | | |
|---|---|---|---|---|
| 4 (z) | Check if the z subroutine works. | Press “z” button |  | Pass  |
| 4 (x’) | Check if the xa subroutine works. | Press “xa” button |  | Pass  |
| 4 (y’) | Check if the ya subroutine works. | Press “ya” button |  | Pass  |
| 4 (z’) | Check if the za subroutine works. | Press “za” button |  | Pass  |
| 5 | See if the form loads and shows the UI as it is meant to be in the design UI. | n/a |  | Pass  |
| 5.1 | This objective works if objective 5.2 works. | n/a | n/a | Pass |

| | | | | |
|---|---|---|---|---|
| 5.2 | When the form loads, the six faces should arrange themselves as it would look on a normal cube net. | n/a |  | Pass  |
| 5.2.1 | When the form loads and the six faces show, yellow should be on top and white should be at the bottom. | N/a |  | Pass  |
| 5.2.2 and 5.2.3 | I did not have enough time to meet these objectives so they will not be tested. | | | |
| 5.3 | Press any move and press Reset button. This objective works in conjunction with objective 6. | Press any move button, then "Reset" button |  | Pass  |
| 5.4 and 5.5 | I did not animate so these have failed. | n/a | | Fail |

| | | | | |
|---|---|---|---|---|
| 5.6 | See if facelets can change colour when clicked on and tickbox is ticked. | n/a |  | Pass  |
| 5.6.1 | See if the facelets don't change colour when clicked and tickbox is not ticked. | n/a |  | Pass  |
| 6 | This objective works in conjunction with objective 5.3. | n/a | The Design UI did not include this button. | Pass  |
| 7 | Scramble. | Press "Scramble" button with 3 moves. | | Pass  |
| 8 | The "History" button got renamed "Log" because I thought it sounded more convenient. | n/a | The Design UI did not include this button. | Pass  |

| | | | | |
|---|---|---|---|---|
| 8.1 | A new window appears when the “Log” button is pressed. | Do a scramble, solve it manually, do more moves, reset. Press “Log” button. | | Pass<br>  <br> |
| 8.2 and 8.3 | I did not feel that the user would like to access previous sessions. | | | |
| 9 | “Help” button | | The “Help” button is in a different place on the form than on the Design UI.<br> <br> | Pass<br>  <br> |
| 9.1 | A new window appears when “Help” button is pressed. | Press “Help” button. | | Pass<br>  <br> |
| 9.2 | | | | Pass |
| 10 | I did not have enough time to make this, in my opinion. | Press “SOLVE” button. | | Fail |

# Evaluation – TO BE FINISHED BY 19/12/23

1. - - problems/changes
2. - - how well ?

## Evaluating objectives

In this table, I list the objectives and evaluate each one on how well I met it and if there were any issues.

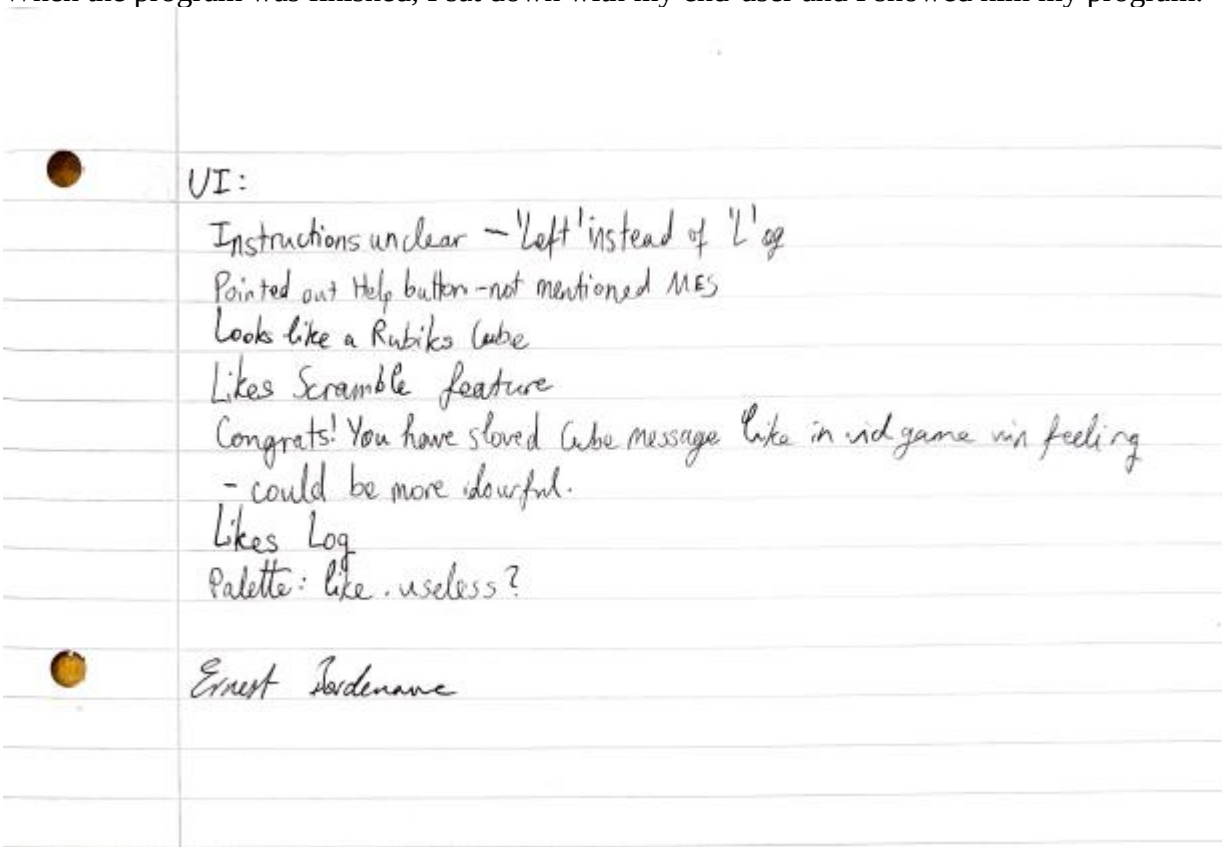
| **Objective** | **Evaluation** |
|---|---|
| 1. The underlying cube | This is a big flop, as I was mainly working on outputting the facelets onscreen, as they are more important, rather than updating the underlying Cube. |
| 1.1 3x3x3 array of cubies | It has been created but wasn't used much, if at all. |
| 1.1.1 Make a Cubie class with a CubieType property. Make sure each Cubie has the right number of colours for its CubieType. Preferably instantiate the cubies at run-time. | Met. I have also instantiated the cubies at run-time so this is a success. |
| 1.1.2 Store the cubies' permutations. | Met. |
| 1.1.3 Make a private Cube made of 27 Cubies and preferably instantiate at run time. | Met. However, it was not instantiated at run time, as it needed to be passed in as a parameter to subroutines in the Moves module, which seems to not be possible with instances/objects not created at design-time. |
| 1.2 6x3x3 array of faces | Failed as not created. I think I didn't have enough time. |
| 2. Create the clockwise and anticlockwise moves as functions and return the new cube state, which will prevent the Cube from being public. | I made all the moves as subroutines, however I did not return the cube's state as I was mainly working on outputting the facelets onscreen than updating the underlying Cube. |
| 3. The buttons on the form have to be neatly arranged and explicit, and there will be a button for each clockwise and anticlockwise move function. | There is indeed a button for each clockwise and anticlockwise move, so this objective has been met. In my opinion, the buttons are neatly arranged, however I will need to get feedback from my end-user. |
| 4. Make six rotation functions, a clockwise and an anticlockwise rotation for each of the three axes. | Met. |
| 4.1 The underlying cube will be updated first before any animation happens. | Fail as I did not focus on the underlying cube, and I ended up doing no animation whatsoever. |
| 5.UI | The general UI follows the design UI, so this objective has been met. |
| 5.1 Make a Face class, which will be composed of 9 Facelets in a 3x3 2D array. | Met. |
| 5.2 Display six Faces in a classic net on a Windows Form. | Met. |
| 5.2.1 Make sure that yellow is above and white is on the bottom, so as to not confuse anyone. | Met. |
| 5.2.2 (Optional) Create the faces at run-time. | Not met. The faces needed to be passed in as parameters from subroutines in the Moves |

| | |
|---|---|
| | module, which seems to not be possible with instances/objects not created at design-time. |
| 5.2.3 (Optional) Recreate Option 2 from ruwix.com. | Not met, as I didn't have enough time. |
| 5.3 Make a default position where each face is only one colour and show it when the Reset button is clicked. | Met. This objective works in conjunction with objective 6. |
| 5.4 Animate each move, both clockwise and anticlockwise. For 180° rotations, I will call and animate the clockwise move twice. Use the sleep subroutine when animating moves. | Not met as I didn't have enough time. Also, there were no buttons for 180° moves or rotations. |
| 5.5 Animate the rotations. The facelets on the far side will be updated first, and the wave will continue in the direction that the cube is being rotated in. Any faces on the axis of rotation will be animated in a clockwise or anticlockwise manner, and will be animated along with the rest of the cube as it rotates. | Not met as I didn't have enough time. |
| 5.6 Make a colour palette so that the user can input individual colours. | Met. This took quite some time to get it to work, but I managed it in the end. |
| 5.6.1 Have a tickbox so the user can say that they are in Editing Mode. | Met. |
| 5.7 Have the move buttons in neat order. Align the clockwise and anticlockwise move buttons with their respective partner. | I have realised that this is repeat of objective 3, so in my opinion it has been met. |
| 6. Make a Reset button which will call an algorithm that resets the state of the cube and facelets. | Met. This objective works in conjunction with objective 5.3. |
| 7. Make a scramble function. It will work in conjunction with a textbox(?) in which the user can input how many moves they want the scrambling function to scramble with. | Met. The textbox ended up being an up-down control, which I didn't know the name of when writing the objectives. This is some defensive programming. |
| 8. A History button will be made. | Met, however it ended up being called the "Log" button, as I felt that that was a more sensible name. |
| 8.1 When it is pressed a new window will appear, which will contain a log of everything that has happened with the cube so far in the session. | Met. |
| 8.2 A text file is to be created to keep track of what has happened. | Not met, as I didn't have the time, and I thought it as too unnecessary compared to other things, such as the palette. |
| 8.3 An integer variable will contain the line number of the file which starts the current session. | Not met as objective 8.2 was not met so the textfile was not made, which meant that no integer variable held the line number. |
| 9. A Help button will be created. | Met. |
| 9.1 When it is pressed, a new window will appear containing instructions on how each button works and describe their relevant moves. | Met. |
| 9.2 The text string in the Help window will be stored in a constant variable. | Met. |

| | |
|---|---|
| 10. The algorithm to solve the cube (Kociemba's algorithm) | Not met as I decided that it would take too long. |

## Feedback from user

When the program was finished, I sat down with my end-user and I showed him my program.



UI:

Instructions unclear – 'Left' instead of 'L' eg

Pointed out Help button – not mentioned MES

Looks like a Rubiks Cube

Likes Scramble feature

Congrats! You have sloved Cube message like in vid game win feeling – could be more idourful.

Likes Log

Palette: like, useless?

Ernest Bardenave

User requirements

1. A reset button to reset the cube back to its original state was already made but is in the user requirements. A reshuffle button is the same as a scrambling button, which will call a scrambling function. The scrambling function will either use a computer algorithm (Kociemba's algorithm) or generate moves at random, depending on how many moves the user wants to scramble the cube with.
2. Animate the moves, both clockwise and anticlockwise, as well as rotations of the whole cube.
3. Have a solve button which calls a solving function which makes use of Kociemba's algorithm.
   3.1 Have a solve box to show the user which moves lead to solving the cube.
   3.2 Erase the moves in the solve box one by one as they are executed by the user.
4. The buttons have to be more explicit in terms of the text that is shown on the buttons, and they will be lined up according to which face of the cube they are moving and in which direction. This also goes with whole cube rotations.
5. There will be a log window which will pop up when a History button is pressed. It will include things like what moves the user has made, any scrambles, and when the solve button has been pressed.
6. Rotate the whole cube in 3D space. This will need to include relevant buttons for each of the three axes in which the cube can be rotated about, both clockwise and anticlockwise. The rotations will also be animated.
7. A Help button is going to be made. When it is pressed, a new window will appear, containing instructions on what the different buttons do.

- Link to user requirements
- Ask for improvements
- summarise
- signed

- Off first impressions, he liked it.

- User requirement 4: He said that the instructions are unclear. I didn't understand what he was saying as I liked my layout, and he said, for example, for the Left button, it should say "Left" instead of "L".
- He didn't know what E and S were, so I pointed out the Help button (user requirement 7), and we realised that it didn't cover E and S, whereas it covered all the others.
- He said that it looks like a Rubik's Cube. He understood the need for the Reset button (user requirement 1) but didn't express an opinion on it.
- He liked the scramble feature.
- After experimenting with the scramble feature, he said he liked the "Congratulations! You have solved the cube" pop-up message. It was like the winning message in a video game and made him feel powerful. He said that it could be made more colourful, because it looks like an error message.
- I told him to click on the Log button (user requirement 5), and he said that he liked it. He asked if he could see what I wrote on the first interview to remind himself of what we discussed.
  I told him to experiment with the palette and I told him about my box-around-the-palette idea, and he said that he likes the palette but that it was a bit useless.
- I asked him to sign it, to prove that he is the same person as my end-user at the beginning.

## Improvements

1. – 3+ suggestions
2. - – rough idea how?
3. - - how it would add to the solution
4. 4. ^ should link to user feedback

This is a list of improvements that could make the program better:
INCLUDE ONE LINKING TO USER FEEDBACK

### Actually use the underlying cube (objectives 1 and 4.1)

This was, in my opinion, the main challenge in this project. However, I didn't get round to it, and it didn't help that much. It would, however, be very beneficial in the solving algorithm, which wasn't made in the end. In order to implement this, I would edit the moves subroutines so that they also deal with the underlying cube as well as updating the facelets on screen.
I also didn't try out the 6x3x3 array of faces, however upon further reflection I have realised that it would be pretty useless anyway and it was only included for experimenting.
Upon further reflection, I don't think the underlying cube was very important, as its main purpose was to satisfy the solving algorithm. I have realised that I could have just checked the facelets to see if the cube was in the G1 state. Checking the underlying cube to check if the algorithm was in G1 would have required some research into the most effective way to search through a 3D array.

### Arrangement of buttons (objectives 3 and 5.7)

Doing this in a separate subroutine was smart, in my opinion, as I would only have had to edit that subroutine to change the buttons' positions, given that the subroutine was in Form1, so the buttons didn't need to be public.
However, my end-user was not very pleased with the arrangement of the buttons. I think I should have asked him to elaborate more.

### Box around palette reflects the selected colour

This box around the palette was in rubiks-cube-solver.com, and I wanted to replicate that if I had the time. However, I found it pretty difficult to place the box behind the palette colours, so I decided to

not do it at all. It would have been nice to have it, so that the user doesn't have to remember which colour they last selected, however the selected palette colour is forgotten when a move happens, so it doesn't matter a lot. I spent quite a lot of time on this, which I could have used for doing other things, but it was nice to experiment.
Any palette edits are not put on the Log, and upon reflection I have realised that it would be nice to have a message like for example: "4 facelets have been changed to green."

### Animation (objectives 5.4 and 5.5)

I didn't have enough time to do the animation. When I first made the moves subroutines, I decided to make the animation later, as I was mainly focusing on actually making the facelets move onscreen when a move button was pressed. Animation wouldn't have really added anything to the properties of the program, but it was in the user requirements when I first interviewed my user.

### The algorithm to solve the cube (Kociemba's algorithm) (objective 10)

I did not have enough time to do this. The first step in the algorithm, G1, wouldn't have required me to update the underlying cube, as I could have just checked the facelets. This is also in the user requirements from when I first interviewed my end-user. I would have to refresh my mind on how the algorithm works.

### Checking if the cube is solved

Upon further refection, I have realised that my cube is only solved when the cube is in the Reset position. This means that if the user solves the cube but another face other than blue is facing front or blue is facing front but yellow is not on top, then the program will not recognise it as solved and will not congratulate the user. This also means that if the user wants to see that message, they will have to do some rotations which feels unnecessary.
To deal with this, I could check each face if all the facelets on it are of the same colour. I would first see what colour the centre facelet is and check if the other 8 facelets on the face are the same colour as the centre facelet.

## Links

Information/Description:

- Writing Code to Solve a Rubik’s Cube | by Brad Hodkinson | Medium

Analysis:

- https://www.youtube.com/watch?v=ZtlMkzix7Bw&feature=youtu.be

Design:

- data structures - How would you represent a Rubik's Cube in code? - Stack Overflow
- language agnostic - How to represent a Rubik's cube - Stack Overflow
- Representing rubik's cube in APL (acm.org)

Computer Puzzling (jaapsch.net)